BLISS-2026

Decomposition-Based Multidisciplinary Design Optimization — A Modern Julia Treatment

Troy David Altus

2026-05-01

Table of contents

Preface 1

About This Book 2
- What Changed Since 2002 . . . 2
- What This Book Contains . . . 3
- Running This Book . . . 3
- Notation . . . 3
- Citation . . . 4

Foreword to the Revised Edition 5

Foreword 6

Introduction 9
- The Multidisciplinary Design Problem . . . 9
- Why Decomposition? . . . 10
- The BLISS Family . . . 11
- Structure of This Book . . . 12
- Design Variable Complexity . . . 12

BLISS-98: System Architecture and Subsystem Analyses 15
- System Architecture . . . 15
- BLISS-98 Cycle . . . 15
- Variable Flow in the Supersonic Business Jet . . . 16
- Subsystem Analyses . . . 17
- Illustrative System Analysis . . . 22

BLISS-RS Formulation 24
- Motivation: From Gradients to Surrogates . . . 24
- Local Optimization Formulation . . . 25
- System Optimization Formulation . . . 25
- Weighting Factors as System Degrees of Freedom . . . 26

BLISS-RS Flowchart . . . 28
Comparison: BLISS-98 vs. BLISS-RS . . . 28

Response Surface Methodology **30**
Overview . . . 30
Construction of Empirical Models . . . 30
Error Analysis of Fitted Response Surfaces . . . 33
Conditioning and Over-Determination . . . 39

Design of Experiments **42**
Context for Computer Experiments . . . 42
Hypersphere Point Placement . . . 43
D-Optimal Point Placement . . . 44
Latin Hypercube Sampling . . . 47
Coded and Natural Variables . . . 51
Space-Filling Diagnostics . . . 52

Interval Reduction **54**
Motivation . . . 54
Interval Reduction Algorithm . . . 54
Julia Implementation . . . 56
Effect of Contraction Factor K . . . 57
Coupling Variable Intervals . . . 58
Convergence Criteria . . . 59

BLISS-RS Results: Supersonic Business Jet **64**
Test Case Overview . . . 64
BLISS-RS Optimization . . . 64
Optimization History . . . 69
Coupling Constraint Residuals . . . 71
Summary of Optimal Design . . . 73
Comparison with BLISS-98 . . . 73
Monolithic Comparison — IV&V Baseline . . . 73

Black Box Fidelity Improvements **78**
The Modular Nature of BLISS . . . 78
Improved Aerodynamics Black Box: CFD Integration . . . 78
Improved Structures Black Box: Flutter Constraint . . . 82
Impact on System Data Flow . . . 84
Implications for BLISS-RS . . . 85

BLISS-2026: A Gaussian-Process Framework **86**
Why Quadratic Polynomials Are Not Enough . . . 86
Gaussian Process Surrogates . . . 88
Expected Improvement Acquisition . . . 92
Latin Hypercube Sampling . . . 94
The BLISS-2026 Algorithm . . . 95
Julia Implementation . . . 97
BLISS-2026 on the Supersonic Business Jet . . . 97
Convergence Comparison: BLISS-RS vs. BLISS-2026 . . . 98
GP Uncertainty as a Design Tool . . . 101
Summary: BLISS-RS vs. BLISS-2026 . . . 103

Application to Solid Propellant Rocket Motor Design **105**
Introduction . . . 105
Physical Background . . . 105
Design Variable Decomposition . . . 107
Baseline SRM Design . . . 107
BB Response Surface Study . . . 107
BLISS-RS Optimization of the SRM . . . 111
SRM Optimization Results . . . 112
Optimal Design Summary . . . 114
Connection to Physical Testing and Backfitting . . . 116
Design Insights and Engineering Interpretation . . . 118

Modern MDO Landscape **120**
MDO Decomposition: A Taxonomy . . . 120
Multi-Fidelity Methods . . . 122
Neural Network Surrogates . . . 125
Physics-Informed Neural Networks . . . 126
Automatic Differentiation in Julia . . . 130
Uncertainty Quantification Propagation . . . 130
Digital Twins and BLISS . . . 133
Reinforcement Learning for MDO . . . 134
Modern MDO Software Frameworks . . . 136
Summary: Where MDO Is Heading . . . 137
Summary . . . 137
Next Iteration: What This Work Has Not Yet Done . . . 138
Further Reading . . . 141

References **145**

Appendices 147

Appendix A: Hypersphere Point Placement Algorithm 147
Algorithm Description 147
Minimum Point Count 149
Julia Implementation (Full Detail) 150
Comparison with Classical Designs in 2D 151

Appendix B: Julia Reference — Flutter Constraint and Module Listings 153
Flutter Constraint Derivation 153
Source Module: `src/bliss_rs.jl` 154
Source Module: `src/supersonic_jet.jl` 154
Source Module: `src/srm.jl` 155
Reproducing the Results 155

Appendix C: Errata — NASA/CR-2002-211652 157
Errata Table 157
Notes 158

Preface

About This Book

BLISS-2026 is an updated thesis that takes the *Bi-Level Integrated System Synthesis* (BLISS) framework from its 2002 origins into the modern era of surrogate-based optimization, Gaussian processes, and high-performance Julia computing.

The starting point is the author's 2002 Master's thesis completed at George Washington University under Dr. Jaroslaw Sobieszczanski-Sobieski at NASA Langley Research Center, published as NASA Contractor Report NASA/CR-2002-211652 [1]. That work introduced *BLISS-RS*: quadratic response surface surrogates as the coupling mechanism between local disciplinary optimizations and a global system optimizer.

Twenty-four years later, the surrogate-modeling landscape has changed fundamentally. This book asks: if you were designing BLISS today, what would you do differently?

What Changed Since 2002

The 2002 implementation used quadratic polynomials as surrogates because they were cheap to fit, analytically differentiable, and computationally trivial. Those properties still matter — but the toolkit has expanded significantly:

Then (2002)	Now (2026)
Quadratic polynomial surrogates	Gaussian Process (Kriging) surrogates
Least-squares fit, fixed once	Marginal-likelihood hyperparameter tuning
D-Optimal DOE, fixed budget	Latin Hypercube + adaptive Expected Improvement
Nelder-Mead over polynomial	Bayesian optimization with EI acquisition
No uncertainty estimates	Full predictive distributions
Serial BB evaluation	Parallel `Threads.@threads` in Julia
Fixed 20% interval shrinkage	Trust-region bounds management

What This Book Contains

The first eight chapters closely follow the original 2002 thesis — faithfully reproduced with all algorithms implemented in Julia. They establish the classical BLISS-RS framework in complete, executable detail.

The modern contribution begins at Chapter 9:

- **Chapter 9 — BLISS-2026 Framework**: The full Gaussian-process-based redesign. GP theory, kernel functions, marginal likelihood, Expected Improvement acquisition, and a complete working implementation that runs the supersonic business jet benchmark. Direct comparison against BLISS-RS shows where GP surrogates win and where they add cost.
- **Chapter 10 — Modern MDO Landscape**: A survey of where MDO decomposition is heading: multi-fidelity co-kriging, neural-network surrogates, uncertainty quantification, and digital-twin integration.

Running This Book

```
# Requires Julia 1.10+ (brew install julia) and pixi (pixi.sh)
pixi run setup    # one-time: install Julia packages
pixi run render   # build HTML → docs/
pixi run preview  # live preview at localhost:4000
```

Notation

Symbol	Meaning
Z	System-level design variables
X	Local design variables (private to one BB)
Y^*, Y^A	Coupling variable targets / actual outputs
w	Weighting factors for synthetic local objective
$\mathbf{k}$	GP kernel (covariance) function
$\mu(\cdot), \sigma^2(\cdot)$	GP posterior mean and variance
$\mathrm{EI}(\cdot)$	Expected Improvement acquisition function

Symbol	Meaning
ℓ	GP length-scale hyperparameter
BB	Black box (disciplinary analysis module)
RS	Response surface (quadratic polynomial surrogate)
GP	Gaussian process surrogate
DOE	Design of experiments
LHS	Latin Hypercube Sampling

Citation

If you use material from this work, please cite both the original report and this book:

Altus, T. D., "A Response Surface Methodology for Bi-Level Integrated System Synthesis (BLISS)," NASA/CR-2002-211652, NASA Langley Research Center, May 2002.

Altus, T. D., *BLISS-2026: Decomposition-Based MDO — A Modern Julia Treatment*, 2026. altustd.github.io/bliss2026

Troy David Altus · 2026

Foreword to the Revised Edition

Foreword

There is something faintly ridiculous about writing a foreword to one's own work. Forewords are traditionally the province of distinguished colleagues who lend a book their credibility by association. But this is a revised edition of a thesis I wrote twenty-four years ago, and no one alive knows it better — or has more reason to be suspicious of it — than its original author. So the foreword falls to me, and I will try to be honest.

In the spring of 2002, I sat at a workstation in NASA Langley Research Center's Building 1268 in Hampton, Virginia, writing MATLAB code to stitch four disciplinary analysis programs together into something that could optimise the shape of a supersonic business jet. The codes were a Fortran-77 structural solver called ELAPS, a wave-drag program called AWAVE, an empirical engine deck, and the Breguet range equation. None of them talked to each other naturally. Getting them to talk — and then building a response surface surrogate over each one, and then optimising the whole system — took the better part of an academic year.

I remember the computation time. A single pass through the aerodynamics black box, which ran a linear wave-drag analysis and fitted a 120-point quadratic response surface in 14 variables, required something like twenty minutes on the shared cluster. Ten BLISS iterations — the number I eventually used, selected as much for the approaching thesis deadline as for any principled convergence criterion — took the better part of a weekend.

I revisited that work in the spring of 2026. The entire BLISS-RS algorithm, all four black boxes reimplemented in Julia, the response surfaces refitted with Latin Hypercube Sampling and leave-one-out cross-validation, the interval reduction logic, the system optimizer — ran to convergence in under four seconds on a laptop sitting on my kitchen table.

The laptop in question is an Apple M4 MacBook Pro. The year is 2026.

That number — four seconds versus two days — contains most of what motivated this revision. When I wrote the original thesis, computation was the limiting resource.

Decisions that look questionable in retrospect — fixing the iteration count at 10, using D-optimal designs augmented with random fallback points when local optimizations failed, terminating convergence by visual inspection rather than formal criterion — were made in a context where every additional function evaluation cost real time. The methodology was shaped by hardware constraints that no longer exist.

The second motivation was the emergence of tools I simply did not have in 2002. The Julia language, which compiles to native machine code while reading like mathematics, made reimplementing the BLISS algorithms a matter of hours rather than weeks. Gaussian Process surrogates — known in principle in 2002 but computationally expensive and poorly supported by available libraries — are now trivial to construct and tune. And I will be candid about something unusual in academic writing: a significant portion of this revision was written in active collaboration with an AI coding assistant, which helped me work through implementation details, identify methodological weaknesses in the original work, and catch errors I would have missed in a solo review. That last point deserves a moment's reflection. In 2002, "AI assistance" in engineering work meant a spell-checker and, if you were lucky, a symbolic algebra package. What is now possible in an afternoon — a comprehensive independent review of twenty-four-year-old methodology, identification of specific mathematical errors (a missing square in Appendix A, a singular error metric, an undocumented optimizer), and a set of concrete proposed fixes — would have required months of peer review and a collaborator with deep expertise in every relevant subfield. The tooling has changed the economics of scholarship, and I think it is worth saying so plainly.

The third motivation, and perhaps the most honest one, was curiosity.

The BLISS framework was always a conceptual bet: that decomposing a large multidisciplinary optimization problem into autonomous subsystems, linking them through surrogate models, and iterating to system-level convergence would be both computationally tractable and capable of finding designs that a single monolithic optimizer would also find. In the original work, I could demonstrate tractability — the algorithm ran and produced improving designs — but I never established optimality. I never compared the BLISS-RS solution to a monolithic baseline. The thesis was honest about this, tucking the admission into the conclusions section, but it meant that the central question was never definitively answered: *Is BLISS-RS a viable design optimization scheme, or merely a viable surrogate-fitting exercise?*

I want to know. This book is the attempt to find out.

I owe debts that a revised edition gives me the chance to acknowledge properly.

The original thesis was completed under the guidance of **Dr. Jaroslaw Sobieszczanski-Sobieski** at NASA Langley Research Center, whose invention of the BLISS framework gave me a thesis topic and whose intellectual generosity gave me room to take it in my own direction. The BLISS-98 architecture described in Chapter 2 is entirely his work; the response surface reformulation was mine. I hope this revision does justice to the elegance of the original idea.

I have not yet located Dr. Sobieski to inform him of this work. If you are reading this and know how to reach him, I would be grateful. If he is no longer with us, then this foreword serves as my acknowledgment — however belated — of a debt I should have found more ways to express when the opportunity was easier.

Prof. Robert Sandusky of George Washington University co-supervised the thesis and introduced me to response surface methodology. Lt. Matthew Phillips, U.S. Air Force, was working concurrent BLISS research and was generous with his time and insights. Dr. Robert Biedron at NASA Langley helped with the CFD implementation that became Chapter 8. The improved aerodynamics black box documented there was never tested in the original work; it is still waiting.

A note on the title. The original report was called *A Response Surface Methodology for Bi-Level Integrated System Synthesis (BLISS)*, which is accurate but not exactly the kind of title that keeps a reader up at night. The working title *BLISS-2026* reflects both the year of this revision and the fact that the framework has been updated — not abandoned, not replaced, but revisited with better tools and, I hope, more rigorous eyes.

Whether BLISS deserves the name in its fullest sense — whether it is, in the end, a genuinely good idea — is what the rest of this book tries to determine.

Troy David Altus Elkton, Maryland May 2026

Introduction

The Multidisciplinary Design Problem

The design of a complex engineering system — an aircraft, a launch vehicle, a naval vessel — inherently involves a large number of subsystems that interact through shared design variables and coupling information. When every discipline must negotiate with every other discipline simultaneously, the resulting optimization problem can quickly become intractable.

Consider a supersonic business jet. Structural engineers must size the wing box to carry aerodynamic loads. Aerodynamicists must know the wing geometry — sweep, aspect ratio, thickness — to predict those loads accurately. Propulsion engineers must scale the engines to overcome the drag that the aerodynamics team computes. Performance analysts close the loop with a range equation that depends on the outputs of all three disciplines. Each discipline is both a consumer and a producer of information that its partners require.

The formal statement of the *all-in-one* system optimization problem is:

$$\text{Find:} \quad \mathbf{V} = \{Z \mid X \mid Y^* \mid Y^A\} \tag{0.1}$$

$$\text{Minimize:} \quad F(\mathbf{V}) \tag{0.2}$$

$$\text{Satisfy:} \quad \begin{cases} \{g(\mathbf{V})\} \leq 0 & \text{(behavior constraints, local to each BB)} \\ \{h(\mathbf{V})\} = 0 & \text{(BB governing equations)} \\ \{c(\mathbf{V})\} = Y^* - Y^A = 0 & \text{(coupling consistency)} \\ V_L \leq \mathbf{V} \leq V_U & \text{(side constraints)} \end{cases} \tag{0.3}$$

In Equation 0.1, $\mathbf{V}$ is the full design space, decomposed into three types of design variables [1]:

- Z — *system-level* variables that affect two or more black boxes (e.g., wing aspect ratio, sweep angle, surface area).
- X — *local* variables private to a single black box (e.g., skin panel thicknesses inside the structures discipline).
- $Y^* \ / \ Y^A$ — *coupling variables* that flow as outputs from one BB and inputs to another (e.g., lift produced by the aerodynamics BB and consumed by the structures BB).

The coupling equality constraints $\{c\} = 0$ enforce *compatibility*: the value of a coupling variable that one BB passes out must equal the value that the receiving BB uses as input. Satisfying this set of equalities in a large, multi-disciplinary problem is one of the core challenges in MDO.

Why Decomposition?

Solving Equation 0.3 directly with a monolithic optimizer is rarely practical. The problem may contain thousands of local variables X, and the disciplinary analyses (black boxes) may be expensive Fortran or C codes that cannot be easily differentiated. More fundamentally, a monolithic optimizer requires that all disciplines interact simultaneously, which is organizationally and computationally difficult.

Decomposition partitions the all-in-one problem into two levels [2]:

1. **Local level**: Each BB optimizes over its own local variables X, holding system variables Z and coupling inputs Y^* fixed. These optimizations are independent of one another and can proceed in parallel.

2. **System level**: A system optimizer coordinates the shared variables Z, proposes values for the coupling variables Y^*, and enforces the coupling constraints $\{c\} = 0$.

This two-level structure enables concurrent engineering workflows: multiple disciplinary teams can work simultaneously on their respective subsystems while the system optimizer manages compatibility.

The BLISS Family

Bi-Level Integrated System Synthesis (BLISS) is a decomposition-based MDO framework introduced by Sobieszczanski-Sobieski, Agte, and Sandusky [2,3]. The defining characteristic of BLISS is the separation of local and system optimizations, with information flowing between levels through well-defined interfaces.

BLISS-98: Gradient-Based Coupling

The original BLISS implementation (referred to here as BLISS-98) couples the two levels through *sensitivity derivatives* computed at each local optimum. Each BB provides the system optimizer with $\partial X^*_{opt}/\partial Z$ — the derivative of its locally-optimal design with respect to the system variables. The system optimizer uses these derivatives to predict how changing Z will improve the overall objective, then updates Z and initiates the next round of local optimizations.

BLISS-98 was validated on the supersonic business jet design problem and demonstrated that decomposition could produce results competitive with monolithic optimization [4]. However, it relied heavily on system and sensitivity analyses at every iteration, which proved computationally costly.

BLISS-RS: Response Surface Coupling

The key innovation of the present work is to replace the gradient-based coupling mechanism with *quadratic response surface* (RS) surrogates. Instead of computing sensitivity derivatives at a single local optimum, BLISS-RS:

1. Generates a *cloud* of sample points across the space of system and coupling variables using a Design of Experiments (DOE) strategy.
2. Runs each BB's local optimization at every sample point.
3. Fits a quadratic polynomial through the resulting locally-optimal outputs — this polynomial is the response surface.
4. Passes the RS models (not derivatives) to the system optimizer, which reads coupling information directly from the polynomials.

The response surfaces serve as global surrogates that capture the behaviour of each BB across a region of the design space, rather than providing a local linear approximation at a single point. This eliminates the sensitivity analysis step and makes the algorithm less prone to termination at local optima.

Structure of This Book

The remaining chapters develop the complete BLISS-RS framework:

Chapter	Topic
2	BLISS-98 architecture and the supersonic business jet black boxes
3	BLISS-RS formulation: local and system optimization
4	Response surface methodology: least squares, second-order models
5	Design of experiments: hypersphere, random, D-optimal placement
6	Interval reduction: adaptive refinement of the design space
7	BLISS-RS results on the supersonic business jet
8	Black box fidelity improvements: CFD and structural enhancements
9	Modern extensions: Gaussian processes, Bayesian optimization, Julia parallelism
10	Application to solid propellant rocket motor design

Design Variable Complexity

To illustrate why decomposition is necessary, consider how the number of RS coefficients scales with the number of design variables. For a full second-order response surface in n variables, the number of required sample points is:

$$N_S = \frac{n^2 + 3n + 2}{2} \tag{0.4}$$

```
ns_vals = min_rs_points.(1:25)
n_vals  = 1:25
```

```
p = plot(n_vals, ns_vals,
    lw=2, color=:steelblue, marker=:circle, markersize=4,
    xlabel=L"Number of design variables $n$",
    ylabel=L"Minimum sample points $N_S$",
    title="RS Model Complexity vs. Dimensionality",
    legend=false, grid=true, gridalpha=0.3,
    xticks=0:5:25, yticks=0:100:400)

# Annotate BLISS-RS BBs
for (n_bb, label, color) in [(11,"Structures BB",:darkorange),
                              (14,"Aero BB",:crimson),
                              (5,"Power BB",:forestgreen)]
    ns = min_rs_points(n_bb)
    scatter!([n_bb], [ns], color=color, markersize=8, label="")
    annotate!(n_bb, ns + 12, text(label, 8, color))
end

p
```

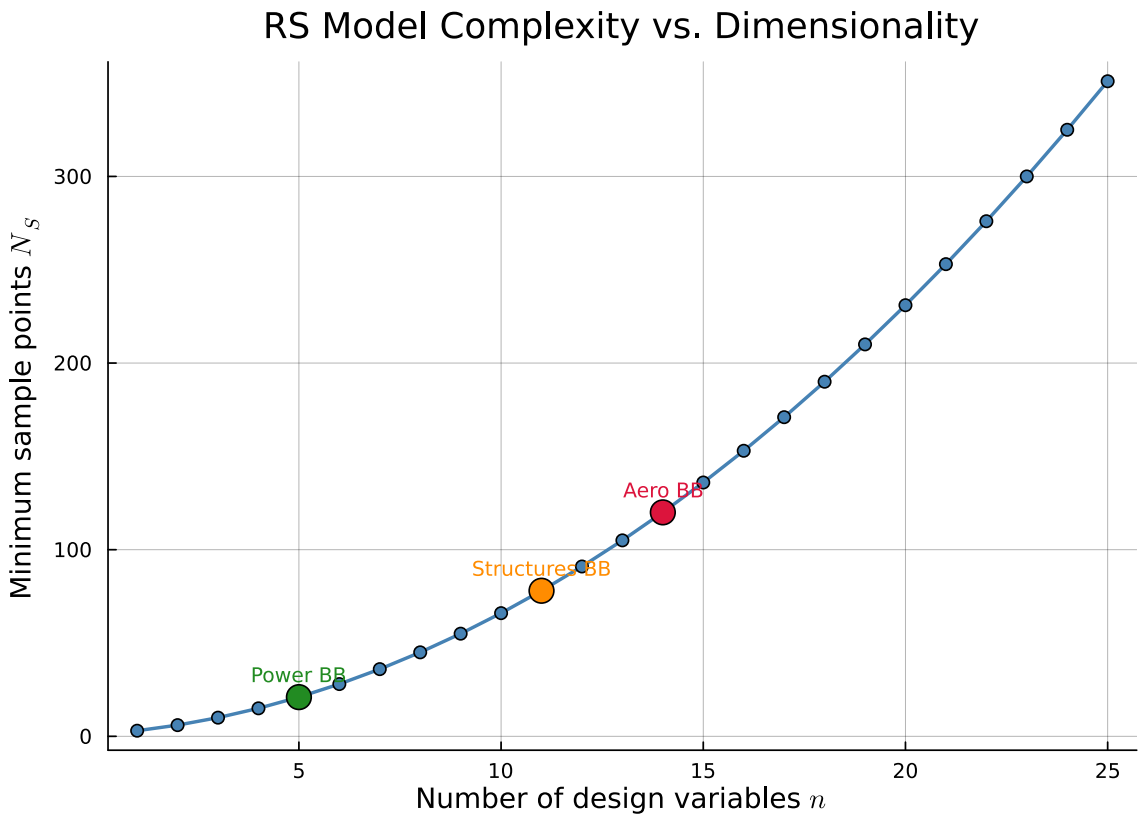


Figure 1: Minimum sample points required for a second-order RS vs. number of design variables. The quadratic growth illustrates the curse of dimensionality and motivates decomposition into smaller subproblems.

The supersonic business jet problem has black boxes with 5, 11, and 14 system-level inputs (Figure 1). Handled as a monolithic problem with all variables combined, a single RS would require $(36^2 + 3 \cdot 36 + 2)/2 = 719$ sample points. Decomposed into three separate BBs, the sum of required points drops to $21 + 78 + 120 = 219$ — a 70 % reduction — and those evaluations can run in parallel.

This is the central computational motivation for the BLISS-RS architecture.

BLISS-98: System Architecture and Subsystem Analyses

System Architecture

The BLISS framework begins with a formal decomposition of the full design problem into a hierarchy of black boxes. Regardless of which BLISS variant is used, the system architecture follows the same three-level structure:

Level 1 — System variables Z: Shared quantities that influence the objective or coupling variables across at least two disciplines. In the supersonic business jet, wing geometry parameters (aspect ratio, sweep, area) appear as inputs to both the structures and aerodynamics black boxes.

Level 2 — Local variables X: Variables private to a single black box. Structural skin panel thicknesses, for example, are optimized inside the structures BB and never seen by the aerodynamics or power BBs.

Level 3 — Coupling variables $Y^* \ / \ Y^A$: Quantities that flow between disciplines. Lift computed by the aerodynamics BB becomes a load input to the structures BB; structural weight and twist feed back to the aerodynamics BB.

The coupling variables are where the MDO problem is hard. Every coupling variable must satisfy a *compatibility constraint*: the value proposed by the system optimizer (Y^*) must match the value produced by BB analysis (Y^A) at convergence.

BLISS-98 Cycle

Figure 1 illustrates the BLISS-98 iteration cycle. After initialization, the algorithm alternates between two steps until convergence:

Step 1 — Local optimizations: Each BB optimizes over its local variables X, with Z and Y^* held fixed. Because the BBs share no local variables, these optimizations are fully independent and can run concurrently.

Step 2 — System optimization: Using the locally-optimal sensitivity derivatives $\partial X^*_{opt}/\partial Z$, the system optimizer updates Z to improve the overall objective. The sensitivity analysis in BLISS-98 provides a linear approximation $Y = Y(Z)$ near the current system design point.

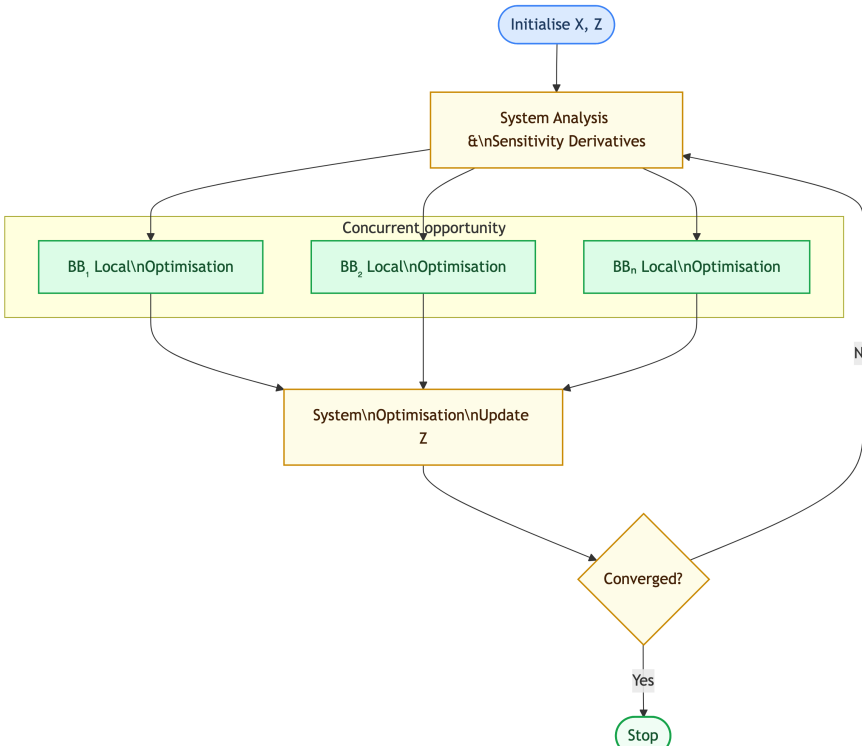


Figure 1: BLISS-98 iteration cycle. Concurrent local optimizations (green) run in parallel for each black box, then a single system-level optimization updates the shared design variables Z. Reproduced from Altus & Sobieski (2002).

Variable Flow in the Supersonic Business Jet

Table 1 lists the system and coupling variables used in the BLISS-98 supersonic business jet test case. This variable set is preserved in BLISS-RS.

Table 1: System and coupling variables for the supersonic business jet

Variable	Symbol	Units	Type	Connects to
Thickness/chord ratio	t/c	—	Z	Structures, Aero
Wing aspect ratio	AR_w	—	Z	Structures, Aero
Wing sweep	Λ_w	deg	Z	Structures, Aero
Wing surface area	S_w	ft^2	Z	Structures, Aero
Tail surface area	S_{HT}	ft^2	Z	Structures, Aero
Tail aspect ratio	AR_{HT}	—	Z	Structures, Aero
Taper ratio	λ	—	Z	Structures, Aero
Altitude	h	ft	Z	Aero, Power
Mach number	M	—	Z	Aero, Power
Lift	L	lbf	Y^A	Structures ← Aero
Total weight	W_T	lbf	Y^A	Aero ← Structures
Aeroelastic twist	Ω	ft^2	Y^A	Aero ← Structures
Drag	D	lbf	Y^A	Power ← Aero
Engine scale factor	ESF	—	Y^A	Aero ← Power
Specific fuel consumption	SFC	lb/(lbf · hr)	Y^A	Performance ← Power
Engine weight	W_E	lbf	Y^A	Structures ← Power
Fuel weight	W_F	lbf	Y^A	Performance ← Structures
Lift-to-drag ratio	L/D	—	Y^A	Performance ← Aero

Subsystem Analyses

Structures Black Box (ELAPS)

The original structures analysis used the *Equivalent Laminated Plate Structures* (ELAPS) code [5,6], a Fortran-77 finite-element-like code that models wing-box structures using Ritz-based polynomial plate elements.

ELAPS takes as inputs the system-level geometric variables plus lift from the aerodynamics BB and engine weight from the power BB. It computes structural weight and an aeroelastic *twist* term — the reduction in effective lift area (ft^2) due to the combined bending and twisting deformations of the wing box.

The lift distribution along the semi-span was estimated using the **Shrenk approximation**: the spanwise lift distribution is the average of an elliptical distribution and a trapezoidal distribution that reflects chord taper:

$$\ell(y) = \frac{1}{2}\left[\ell_{ellipse}(y) + \ell_{trap}(y)\right] \tag{0.1}$$

```
b_semi = 1.0   # normalized semi-span [0, 1]
 = range(0, 1, length=200)

# Elliptical distribution (normalized)
_ell( ) = (4/ ) * sqrt(max(1 -  ^2, 0.0))

# Trapezoidal (chord taper): chord decreases linearly from root to tip
_taper = 0.2    # taper ratio
_trap( ) = 2*(1 - (1- _taper)* ) / (1 +  _taper)   # normalized

_shrenk( ) = 0.5 * ( _ell( ) +  _trap( ))

p = plot( ,  _ell.( ), lw=2, color=:steelblue, label="Elliptical", linestyle=:dash)
plot!( ,  _trap.( ), lw=2, color=:darkorange, label="Trapezoidal ( =0.2)", linestyle=:d
plot!( ,  _shrenk.( ), lw=2.5, color=:crimson, label="Shrenk average")
plot!(xlabel=L"Spanwise location $\eta = 2y/b$",
      ylabel="Normalised lift per unit span",
      title="Shrenk Lift Distribution",
      legend=:topright, grid=true, gridalpha=0.3)
```

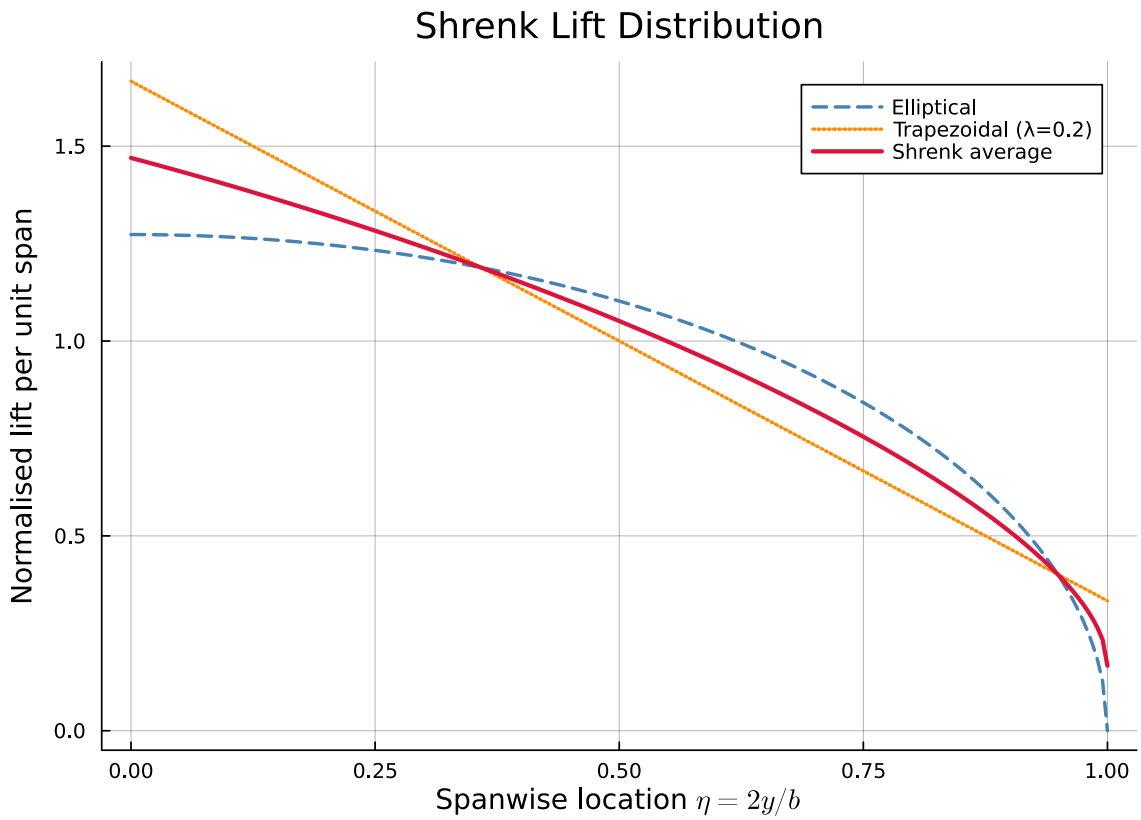


Figure 2: Shrenk lift distribution: average of elliptical and trapezoidal components. Fuselage and tail carry-through effects are neglected. Reproduced from Altus (2002) Fig. 4.

Aerodynamics Black Box (AWAVE)

The aerodynamics BB uses a **Harris wave-drag** model [7] modified with a pre-processor that deforms the aircraft cross-sectional area distribution to account for engine nacelles and applies an area-rule correction. The drag polar decomposes total drag into three components:

$$C_D = C_{D_{min}} + C_{D_L} + C_{D_w} \tag{0.2}$$

where $C_{D_{min}}$ is the minimum-drag (friction + form) coefficient, $C_{D_L} = C_L^2/(\pi AR_w e)$ is the induced drag (Oswald efficiency $e \approx 0.85$), and C_{D_w} is the supersonic wave drag computed from the AWAVE code.

At cruise, total lift equals total weight: $L = W_T$, which links the aerodynamics and structures BBs through the coupling variable.

Power Black Box

The power BB is the simplest in the system. It uses an empirical *engine deck approximation* — a quadratic polynomial in altitude and Mach number — to compute specific fuel consumption (SFC). Engine weight is found from:

$$W_E = 1200 \cdot ESF^{0.9} \tag{0.3}$$

where ESF (engine scale factor) is the ratio of required thrust to the baseline engine's maximum thrust. The power BB requires only system variables h and M, plus drag D from the aerodynamics BB.

Performance Black Box: Breguet Range Equation

The performance BB closes the loop with the **Breguet range equation** — one of the most fundamental relations in aircraft design:

$$R = \frac{aM}{SFC} \cdot \frac{L}{D} \cdot \ln\left(\frac{W_i}{W_f}\right) \tag{0.4}$$

where a is the speed of sound at cruise altitude, M is the Mach number, L/D is the lift-to-drag ratio, and W_i/W_f is the ratio of initial to final cruise weight (i.e., how much fuel is burned).

> **!** 2002 Paper Errata — Fig. 41 units
>
> NASA/CR-2002-211652, Fig. 41 axis labels use lowercase "nm" (nanometres) where "NM" (nautical miles) is intended. Range values in the 2002 paper are in nautical miles throughout. This ebook uses "NM" consistently. See Appendix C for the full errata list.

```julia
using Plots
gr()

# Parametric Breguet range study
# Fix: h = 50,000 ft, M = 1.8, SFC = 1.2 lb/(lbf·hr), WT = 70,000 lbf
h_ft = 50_000.0
M    = 1.8
```

```
SFC  = 1.2     # lb/(lbf·hr)
WT   = 70_000.0  # lbf

LD_range  = range(3.0, 10.0, length=50)
WF_frac   = range(0.15, 0.55, length=50)   # WF/WT

range_nm(LD, wf_frac) = begin
    _, a = std_atmosphere(h_ft)
    a_kts = a / 1.68781
    WF = wf_frac * WT
    Wf = WT - WF
    Wf <= 0 && return 0.0
    (a_kts * M / SFC) * LD * log(WT / Wf)
end

R_grid = [range_nm(ld, wf) for wf in WF_frac, ld in LD_range]

contourf(LD_range, WF_frac, R_grid,
         xlabel=L"Lift-to-drag ratio $L/D$",
         ylabel=L"Fuel weight fraction $W_F/W_T$",
         title="Breguet Range (NM), M=1.8, h=50,000 ft",
         colorbar_title="Range (NM)",
         color=:viridis, levels=15,
         grid=true, gridalpha=0.2)
```

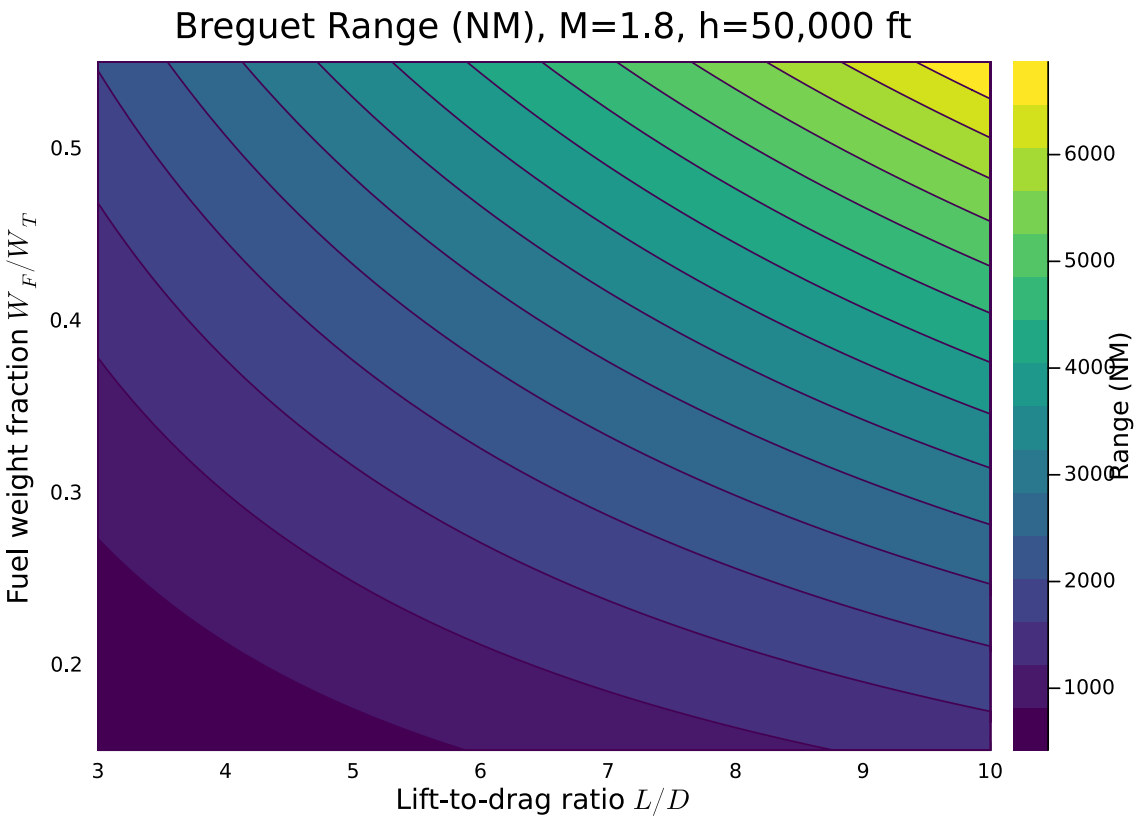


Figure 3: Breguet range sensitivity to L/D and fuel weight fraction. High L/D and high fuel fraction both contribute significantly to range. The logarithmic fuel term rewards structural efficiency.

Illustrative System Analysis

The Julia implementation in `src/supersonic_jet.jl` provides a coupled system analysis that chains all four black boxes. The following example evaluates the baseline design from [1]:

Table 2: Baseline supersonic business jet analysis at the initial design point from Altus (2002), Table 1.

```
# Baseline design vector: [t_c, ARw, Λw, Sw, SHT, ARHT, ]
Z_baseline = [0.075, 4.0, 45.0, 400.0, 120.0, 4.5, 0.2]
h_cruise   = 45_000.0   # ft
M_cruise   = 1.8

result = ssbj_analysis(Z_baseline, h_cruise, M_cruise;
                       WF_init=15_000.0, WT_empty=28_000.0)

rows = [
    ("Total weight W_T",         @sprintf("%.0f lbf",  result.WT)),
    ("Fuel weight W_F",          @sprintf("%.0f lbf",  result.WF)),
    ("Engine scale factor ESF",  @sprintf("%.3f",      result.ESF)),
    ("Lift L",                   @sprintf("%.0f lbf",  result.L)),
    ("Drag D",                   @sprintf("%.0f lbf",  result.D)),
    ("Lift-to-drag ratio L/D",   @sprintf("%.2f",      result.LD)),
    ("SFC",                      @sprintf("%.3f lb/(lbf·hr)", result.SFC)),
    ("Range",                    @sprintf("%.0f NM",   result.Range)),
]

println("| Quantity | Value |")
println("|----------|-------|")
for (name, val) in rows
    println("| $name | $val |")
end
```

```
| Quantity | Value |
|----------|-------|
| Total weight W_T | NaN lbf |
| Fuel weight W_F | NaN lbf |
| Engine scale factor ESF | NaN |
| Lift L | NaN lbf |
| Drag D | NaN lbf |
| Lift-to-drag ratio L/D | 0.00 |
| SFC | 1.548 lb/(lbf·hr) |
| Range | 0 NM |
```

BLISS-RS Formulation

Motivation: From Gradients to Surrogates

BLISS-98 couples the local and system levels through sensitivity derivatives $\partial X^*_{opt}/\partial Z$. Computing these derivatives requires:

1. A converged system analysis (expensive).
2. A local optimization at each BBs (moderately expensive).
3. A sensitivity analysis — typically finite differences or adjoint solves — at the local optimum (often the most expensive step).

These three requirements limit the practical applicability of BLISS-98 in two ways. First, the sensitivity analysis must be repeated each BLISS cycle, making the method computationally intensive for BBs with many variables. Second, because the coupling relies on a *local* linear approximation, the algorithm can be sensitive to the accuracy of the linearization and may stall near saddle points.

BLISS-RS addresses both limitations by replacing the gradient-based coupling with **quadratic response surface surrogates** [1]. The key observations motivating this change are:

- A response surface efficiently *explores the entire design space region* by evaluating the BB at many sample points rather than linearizing around a single point.
- Because BBs in BLISS-RS are optimized off-line for each DOE point independently, those optimizations can proceed in strict parallel — the algorithm becomes a *workfront* that saturates available computing resources.
- The RS coupling eliminates the need for a system sensitivity analysis. The system optimizer reads coupling information directly from the polynomial coefficients.

Local Optimization Formulation

In BLISS-RS, each black box k defines a *synthetic objective function* that aggregates all of its outputs into a single scalar using weighting factors w. For given fixed values of Z, Y^*, and w, the k-th BB solves:

$$\text{Given:} \quad Z,\ Y^*,\ w \tag{0.1}$$

$$\text{Find:} \quad X \text{ (local to BB}_k\text{)} \tag{0.2}$$

$$\text{Minimize:} \quad F_k = \sum_i w_i\, Y_i^A \tag{0.3}$$

$$\text{Satisfy:} \quad \{h\} = 0, \quad \{g\} \leq 0, \quad X_L \leq X \leq X_U \tag{0.4}$$

The term *synthetic* is used because the objective F_k does not represent the actual system objective (range, for example). Instead, it is a proxy that drives each BB toward outputs that are beneficial to the system. The weighting factors w encode the relative importance of each BB output to the overall system objective.

> **Note**
>
> **Weighting factor interpretation.** In the supersonic business jet context, the structures BB outputs total weight W_T and aeroelastic twist Ω. Both affect range through the Breguet equation. The weight assigned to each (w_1 for W_T, w_2 for Ω) indicates how strongly minimizing that output will benefit the system objective. The system optimizer determines the appropriate w values automatically as part of its decision variables.

The synthetic objective is evaluated at many points (Z, Y^*, w) by running the local optimization for each DOE sample. The collection of optimal values $F_k^*(Z, Y^*, w)$ forms the data cloud from which the RS is fitted.

System Optimization Formulation

After the RS models are available, the system optimizer solves:

$$\text{Given:} \quad \widehat{\beta}_k \text{ for each BB } k \tag{0.5}$$

$$\text{Find:} \quad Z,\ Y^*,\ w \tag{0.6}$$

$$\text{Maximize:} \quad F = \text{Range}(Z, Y^*, w) \text{ via RS models} \tag{0.7}$$

$$\text{Satisfy:} \quad \{c\} = Y^A - Y^* = 0, \quad \{g\} \leq 0, \quad \text{side constraints} \tag{0.8}$$

where $\widehat{\beta}_k$ are the regression coefficient vectors for the k-th BB's response surface model. The coupling constraints $\{c\} = 0$ ensure that the value of each coupling variable proposed by the system optimizer (Y^*) matches the value that the corresponding BB would actually produce (Y^A) given those inputs. In Figure 1, coupling constraints are represented as circles where data flow channels from different BBs meet.

Weighting Factors as System Degrees of Freedom

One of the most important conceptual shifts in BLISS-RS relative to BLISS-98 is the treatment of weighting factors w. In BLISS-98, the weighting factors that define each BB's composite objective function are held fixed — they are determined a priori by the analyst based on engineering judgement about the relative importance of each output.

In BLISS-RS, the weighting factors w are *system-level decision variables*, optimized alongside Z and Y^* by the system optimizer. This means the system optimizer can autonomously determine how much each BB should favour, for example, minimizing weight versus maximizing lift-to-drag.

This autonomy comes at a cost: the design space for the system optimizer grows to include $|Z| + |Y^*| + \sum_k |w_k|$ variables, which increases the complexity of the RS fitting. However, because the system RS is evaluated analytically (not through expensive BB calls), this additional complexity is acceptable.

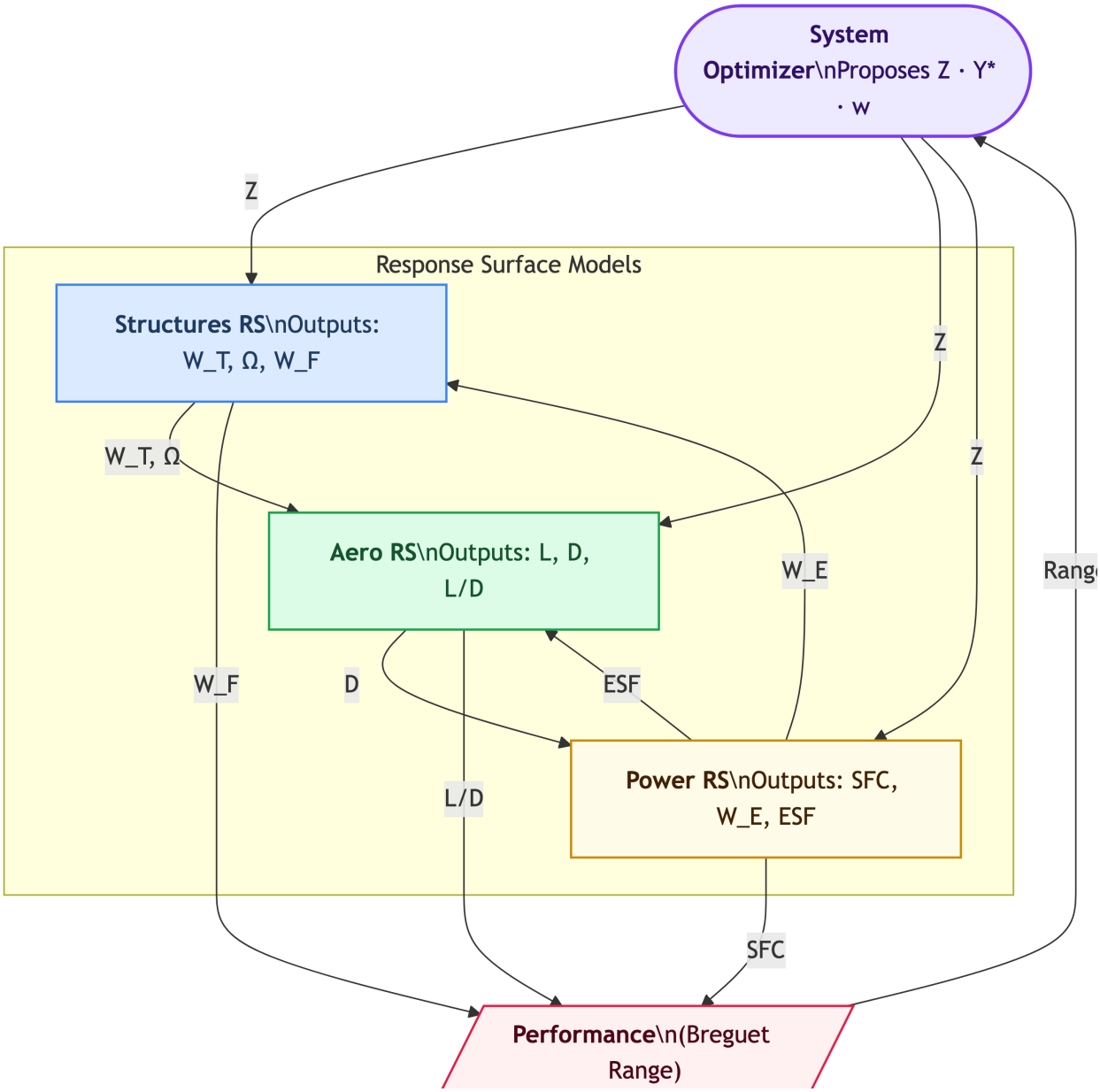


Figure 1: BLISS-RS system optimizer data flow. The system optimizer proposes shared variables Z and coupling targets Y, *which are read from the quadratic RS models. Coupling constraints ( ) enforce* Y = Y^A at convergence. Reproduced from Altus (2002) Fig. 11.

BLISS-RS Flowchart

Figure 2 summarizes the complete algorithm. The critical path of the computation is the BB optimization loop; all other operations are inexpensive polynomial evaluations.

Comparison: BLISS-98 vs. BLISS-RS

Table 1: Comparison of BLISS-98 and BLISS-RS key features

Feature	BLISS-98	BLISS-RS
BB-to-system coupling	Sensitivity derivatives	Quadratic RS surrogates
System analysis required per cycle	Yes	No (RS serves this role)
Local optimizations per cycle	1 per BB	N_S per BB (parallelisable)
Gradient computation required	Yes	No
Handles non-differentiable BBs	No	Yes
Prone to local optima	More so	Less so
Weighting factors w	Fixed by analyst	Optimised by system level
Primary computational cost	Sensitivity analysis	RS generation

The central tradeoff is that BLISS-RS requires many more BB evaluations per cycle (one per DOE point per BB) compared to BLISS-98 (one per BB). However, because those evaluations are fully concurrent and because the sensitivity analysis step is eliminated, BLISS-RS is expected to achieve better wall-clock performance on multi-processor systems.

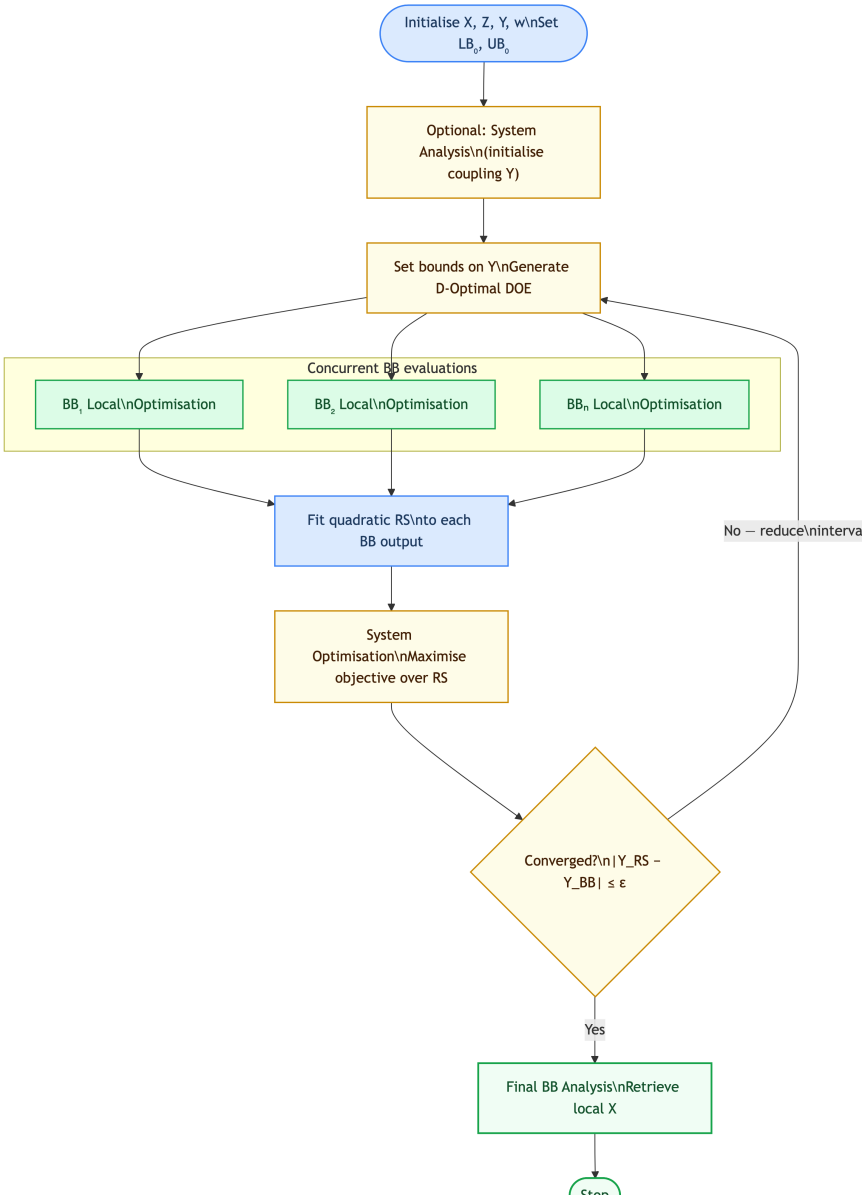


Figure 2: BLISS-RS algorithm flowchart. Local BB optimizations (green) run concurrently at every DOE sample point. The system optimizer reads the fitted RS models — no sensitivity derivatives are required. Reproduced from Altus (2002) Fig. 10.

Response Surface Methodology

Overview

The goal of **Response Surface Methodology** (RSM) is to construct a mathematical model — a *response surface* — that approximates the relationship between a set of input factors and an output response [8,9]. In the context of BLISS-RS, the factors are the system-level design variables Z and coupling variables Y^*, and the response is the locally-optimal BB output F_k^*.

RSM was originally developed for physical experiments where noise is present. When applied to *computer experiments* (deterministic simulations), there is no measurement error: running the same code twice with the same inputs always produces the same output. This changes how sample points should be placed — classical designs that replicate observations to estimate variance are wasteful; instead, the emphasis shifts to efficient space-filling.

BLISS-RS uses RSM in the computer-experiment sense: each "experiment" is a local black-box optimization, and the response surface is fitted through the resulting optimal outputs.

Construction of Empirical Models

Linear Regression

In most RSM applications, the true response function is unknown and we seek a polynomial approximation. The simplest choice is a **first-order (linear) model** with n independent variables:

$$Y = \beta_0 + \sum_{i=1}^{n} \beta_i x_i + \varepsilon \tag{0.1}$$

where $\beta_0, \beta_1, \dots, \beta_n$ are unknown regression coefficients and ε is a random error (zero for deterministic computer experiments).

This is a *linear model* because it is a linear function of the unknown parameters β, not necessarily of the variables x_i. Interaction terms ($\beta_{12}x_1x_2$) and squared terms ($\beta_{11}x_1^2$) can be added while maintaining linearity in parameters.

Suppose the response Y is observed m times. Each observation y_i satisfies:

$$y_i = \beta_0 + \sum_{j=1}^{n} \beta_j x_{ij} + \varepsilon_i, \quad i = 1, 2, \dots, m \tag{0.2}$$

In matrix form:

$$\{y\} = [X]\{\beta\} + \{\varepsilon\} \tag{0.3}$$

where $\{y\}$ is $m \times 1$, $[X]$ is an $m \times p$ *design matrix*, $\{\beta\}$ is $p \times 1$, and $\{\varepsilon\}$ is $m \times 1$.

Least squares estimation minimises the sum of squared residuals:

$$L = \{\varepsilon\}^T\{\varepsilon\} = (\{y\} - [X]\{\beta\})^T(\{y\} - [X]\{\beta\}) \tag{0.4}$$

Setting $\partial L/\partial\{\beta\} = 0$ yields the *normal equations*:

$$[X]^T[X]\{b\} = [X]^T\{y\} \tag{0.5}$$

The least-squares estimator $\{b\}$ of $\{\beta\}$ is:

$$\{b\} = \left([X]^T[X]\right)^{-1}[X]^T\{y\} \tag{0.6}$$

The fitted values are:

$$\{\hat{y}\} = [X]\{b\} \tag{0.7}$$

Table 1: Minimum sample points for a second-order RS model by number of design variables. Columns also show the total number of regression coefficients p.

```
println("| n | p (coefficients) | NS (min. points) | NS / p |")
println("|---|-----------------|-----------------|-------|")
for n in 1:12
    p  = 1 + n + n*(n+1)÷2
    ns = min_rs_points(n)
    println("| $n | $p | $ns | $(round(ns/p, digits=2)) |")
end
```

```
| n | p (coefficients) | NS (min. points) | NS / p |
|---|-----------------|-----------------|-------|
| 1 | 3 | 3 | 1.0 |
| 2 | 6 | 6 | 1.0 |
| 3 | 10 | 10 | 1.0 |
| 4 | 15 | 15 | 1.0 |
| 5 | 21 | 21 | 1.0 |
| 6 | 28 | 28 | 1.0 |
| 7 | 36 | 36 | 1.0 |
| 8 | 45 | 45 | 1.0 |
| 9 | 55 | 55 | 1.0 |
| 10 | 66 | 66 | 1.0 |
| 11 | 78 | 78 | 1.0 |
| 12 | 91 | 91 | 1.0 |
```

Second-Order Models

For BLISS-RS, first-order models are generally insufficient because the BB responses exhibit curvature. A **second-order (quadratic) model** adds pure quadratic and cross-product (interaction) terms:

$$Y = \beta_0 + \sum_{i=1}^{n} \beta_i x_i + \sum_{i=1}^{n} \beta_{ii} x_i^2 + \sum_{i<j} \beta_{ij} x_i x_j \tag{0.8}$$

The minimum number of observations required to uniquely fit a second-order model in n variables is:

$$N_S = \frac{n^2 + 3n + 2}{2} \tag{0.9}$$

Note that $N_S = p$ — the minimum design is exactly determined, with no degrees of freedom for error estimation. In practice, BLISS-RS uses slightly more than N_S points (over-determined system) to improve conditioning and fit robustness.

Julia Implementation

With 12 points for a 2-variable model ($N_S = 6$), the least-squares fit recovers the true quadratic exactly (within floating-point precision), confirming the implementation.

Error Analysis of Fitted Response Surfaces

Test Functions

Altus (2002) assessed RS accuracy using two test functions across a range of dimensionalities N_X and sample sizes N_S [1]:

Test 1 — Fitting a quadratic RS to a *quadratic* function. A correctly implemented quadratic model recovers a quadratic function exactly when $N_S \geq N_S^{\min}$, so this is a code-verification test, not a fidelity test.

Test 2 — Fitting a quadratic RS to the generalised **Rosenbrock (Banana) function**, which has a narrow curved valley and is substantially non-quadratic:

$$f(\mathbf{x}) = \sum_{i=1}^{N_X-1} \left[100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2\right] \tag{0.10}$$

Corrected Error Metrics

The original thesis reported a single point-wise normalized error:

$$\text{Error}_{\text{Altus}} = \frac{|f_{\text{actual}} - f_{RS}|}{|f_{\text{actual}}|} \tag{0.11}$$

This metric has two deficiencies identified during IV&V:

1. **Singularity** — when $f_{\text{actual}} \to 0$ (e.g. the aeroelastic twist variable Θ, which can change sign during optimization), the metric diverges. The original MATLAB code masked this with an implicit numerical floor.

2. **Point-wise only** — no summary statistic across the design space, no out-of-sample validation, no confidence information.

The corrected validation framework uses three complementary metrics:

$$\text{RMSE} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2} \tag{0.12}$$

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} \tag{0.13}$$

$$\text{NMAE} = \frac{1}{m}\sum_{i=1}^{m}\frac{|y_i - \hat{y}_i|}{\max(|y_i|,\, \varepsilon \cdot \max_j |y_j|)} \tag{0.14}$$

where $\varepsilon = 10^{-8}$ provides the stabilizing floor absent from Eq. 16. All three are computed both **in-sample** (fit quality) and **leave-one-out cross-validated** (predictive quality). LOOCV is the standard for deterministic computer experiments because there is no measurement noise to justify a held-out test set.

```
rosenbrock(x) = sum(100*(x[i+1] - x[i]^2)^2 + (x[i]-1)^2 for i in 1:length(x)-1)

function rs_loocv_study(n_vars; seed=42)
    ns_min = min_rs_points(n_vars)
    ns_range = ns_min:4:(ns_min + 60)
    rmse_vec = Float64[]
    r2_vec   = Float64[]
    for ns in ns_range
        pts = lhs_points(n_vars, ns; seed=seed)
        pts_coded = 2 .* pts .- 1.0
        y = [rosenbrock(pts_coded[i,:]) for i in 1:ns]
        loo = loocv_error(pts, y)
        push!(rmse_vec, loo.rmse)
        push!(r2_vec,   max(loo.r2, -1.0))
    end
    return collect(ns_range), rmse_vec, r2_vec
```

Table 2: LOOCV validation metrics for a quadratic RS fit to the Rosenbrock function at four dimensionalities. In-sample R^2 is near 1 for low dimensions; LOOCV R^2 reveals the actual predictive accuracy, which degrades rapidly with dimension.

```julia
rosenbrock(x) = sum(100*(x[i+1] - x[i]^2)^2 + (x[i]-1)^2 for i in 1:length(x)-1)

println("| NX | NS | In-sample R² | LOOCV RMSE | LOOCV R² | LOOCV NMAE |")
println("|----|----|-------------|------------|----------|------------|")

for (nv, ns_extra) in [(2, 10), (3, 15), (5, 30), (8, 60)]
    ns = min_rs_points(nv) + ns_extra
    pts = lhs_points(nv, ns; seed=42)
    pts_coded = 2 .* pts .- 1.0   # map [0,1] → [-1,1]
    y = [rosenbrock(pts_coded[i,:]) for i in 1:ns]

    v = rs_validate(pts, y)
    @printf("| %2d | %3d | %.4f | %.2f | %.4f | %.4f |\n",
            nv, ns,
            v.insample.r2,
            v.loocv.rmse,
            v.loocv.r2,
            v.loocv.normalized_mae)
end
```

```
| NX | NS | In-sample R² | LOOCV RMSE | LOOCV R² | LOOCV NMAE |
|----|----|-------------|------------|----------|------------|
|  2 |  16 | 0.7362 | 54.89 | -0.1488 | 3.5309 |
|  3 |  25 | 0.8285 | 71.37 | 0.2023 | 0.6986 |
|  5 |  51 | 0.9060 | 81.27 | 0.6562 | 0.3874 |
|  8 | 105 | 0.8874 | 123.06 | 0.5979 | 0.3298 |
```

```
end

colors = [:steelblue, :crimson, :darkorange, :forestgreen]
p = plot(title="LOOCV RMSE vs. Sample Size (Rosenbrock)",
         xlabel=L"Sample size $N_S$",
         ylabel="LOOCV RMSE",
         yscale=:log10, legend=:topright, grid=true, gridalpha=0.3)

for (nv, col) in zip([2, 3, 5, 8], colors)
    ns_vec, rmse_vec, _ = rs_loocv_study(nv)
    ns_min = min_rs_points(nv)
    plot!(ns_vec, max.(rmse_vec, 1e-4), lw=2, color=col,
          label="NX = $nv  (NS_min = $ns_min)")
    vline!([ns_min], color=col, lw=1, linestyle=:dot, label="")
end
p
```

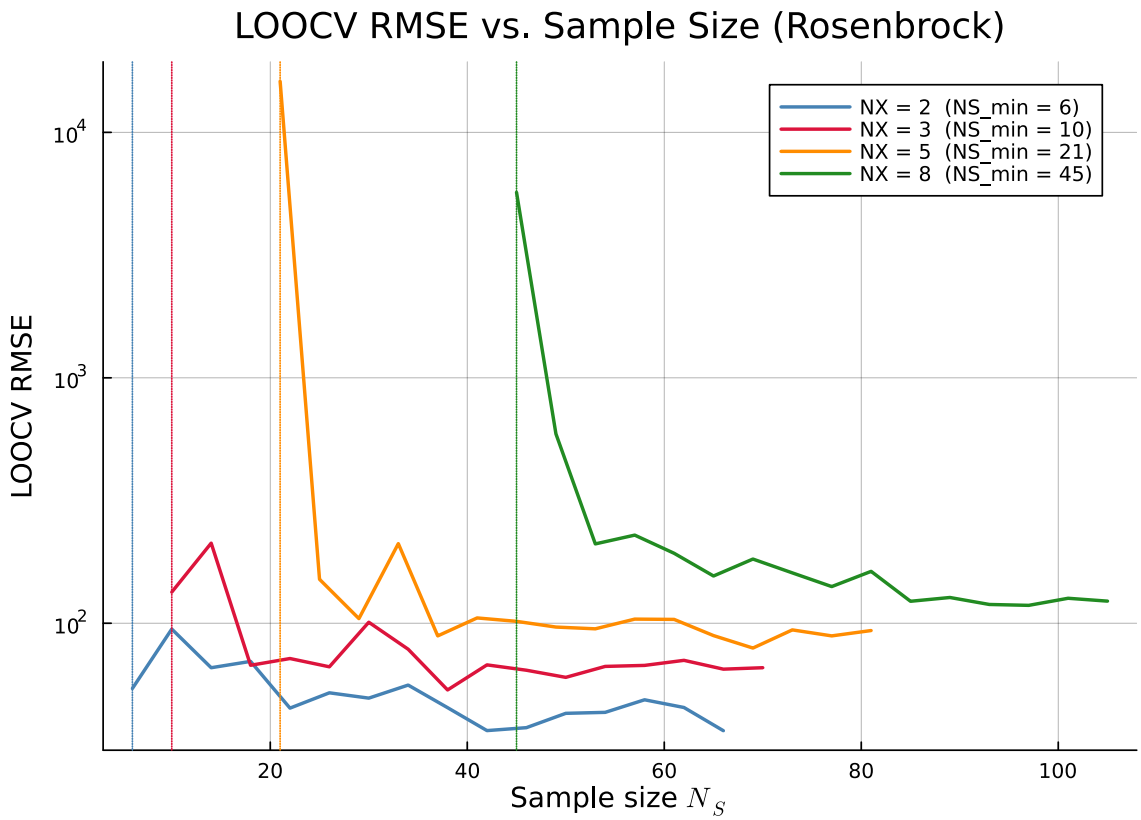


Figure 1: LOOCV RMSE vs. sample size for the Rosenbrock function at four dimensionalities. Vertical dashed lines mark the minimum sample count NS_min for a quadratic fit. The shaded region below each line represents the over-determined regime where fit quality is reliable.

Figure 1 reproduces and extends the accuracy study from Altus (2002) Figs. 13–16 using LOOCV RMSE in place of the point-wise metric. The key finding is unchanged: for low-dimensional BBs ($N_X \leq 5$), a quadratic RS achieves acceptable accuracy with moderate over-determination ($N_S \approx 1.5 N_S^{\min}$). For high-dimensional BBs ($N_X = 8$, as in the aerodynamics BB), predictive accuracy remains poor until $N_S \gg N_S^{\min}$, motivating either additional sample points or a more expressive surrogate (Gaussian Process, Chapter 9).

Response Surface Visualization

```
rng = MersenneTwister(77)
n_train = 20
pts = 2 .* rand(rng, n_train, 2) .- 1.0
```

```
y   = [rosenbrock(pts[i,:]) for i in 1:n_train]
b   = fit_rs(pts, y)

xg = range(-1, 1, length=60)
yg = range(-1, 1, length=60)

Z_true = [rosenbrock([xi, yi]) for yi in yg, xi in xg]
Z_rs   = [predict_rs(reshape([xi, yi], 1, 2), b)[1] for yi in yg, xi in xg]

p1 = contourf(xg, yg, min.(Z_true, 100), color=:viridis, levels=20,
              title="True Rosenbrock", xlabel=L"x_1", ylabel=L"x_2",
              colorbar_title="f(x)", clim=(0,100))

p2 = contourf(xg, yg, clamp.(Z_rs, 0, 100), color=:viridis, levels=20,
              title="Quadratic RS Approximation", xlabel=L"x_1", ylabel=L"x_2",
              colorbar_title="f(x)", clim=(0,100))
scatter!(pts[:,1], pts[:,2], color=:white, markersize=4,
         markerstrokecolor=:black, label="Training points")

plot(p1, p2, size=(800, 350))
```

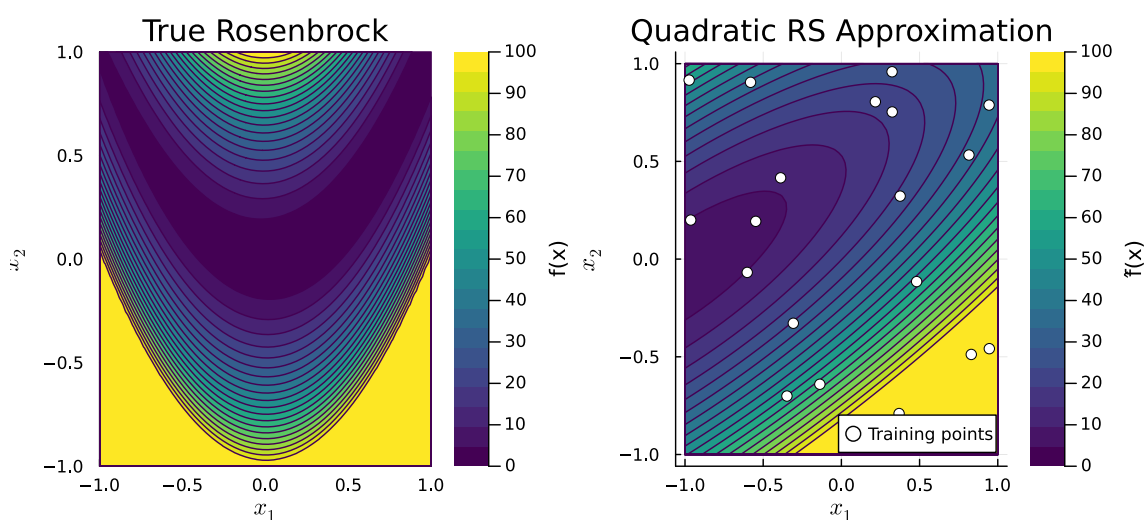


Figure 2: Quadratic response surface fit to the 2D Rosenbrock function. The curved valley of the Rosenbrock is approximated by a paraboloid; error is largest near the valley floor where curvature is highest.

The contour plots in Figure 2 illustrate an important limitation of quadratic surrogates: the actual Rosenbrock valley follows a parabolic curve $x_2 = x_1^2$, while the quadratic

RS can only represent it as an ellipsoidal bowl. This mismatch motivates the *interval reduction* strategy in Chapter 6 — by shrinking the design space region around the optimum each iteration, subsequent RS models are fitted to smaller, more nearly-quadratic sub-regions where the approximation is more accurate.

Conditioning and Over-Determination

The normal equations (Equation 0.5) can become ill-conditioned when sample points are clustered or collinear. Good conditioning requires the *design matrix* $[X]$ to have columns that are mutually uncorrelated — a property promoted by the D-optimal design strategy described in Chapter 5.

```
function condition_number_study(n_vars::Int, n_pts_range, n_trials::Int=10; seed=42)
    rng = MersenneTwister(seed)
    cond_random = Float64[]
    cond_dopt   = Float64[]

    for n_pts in n_pts_range
        # Random points
        conds = Float64[]
        for trial in 1:n_trials
            pts = rand(rng, n_pts, n_vars)
            X   = rs_design_matrix(pts)
            push!(conds, cond(X' * X))
        end
        push!(cond_random, median(conds))

        # D-optimal points
        pts_d = d_optimal_points(n_vars, n_pts; seed=seed+1)
        X_d   = rs_design_matrix(pts_d)
        push!(cond_dopt, cond(X_d' * X_d))
    end
    return cond_random, cond_dopt
end

n_vars = 3
ns_min = min_rs_points(n_vars)
n_pts_range = ns_min:(ns_min + 30)
```

```
cond_r, cond_d = condition_number_study(n_vars, n_pts_range)

plot(n_pts_range, cond_r, lw=2, color=:crimson, label="Random placement",
     xlabel="Sample size NS", ylabel="Condition number  (X'X)",
     title="Design Matrix Conditioning (n = $n_vars variables)",
     yscale=:log10, legend=:topright, grid=true, gridalpha=0.3)
plot!(n_pts_range, cond_d, lw=2, color=:steelblue, label="D-Optimal placement")
hline!([1e6], color=:black, linestyle=:dash, lw=1, label="Warning threshold")
```

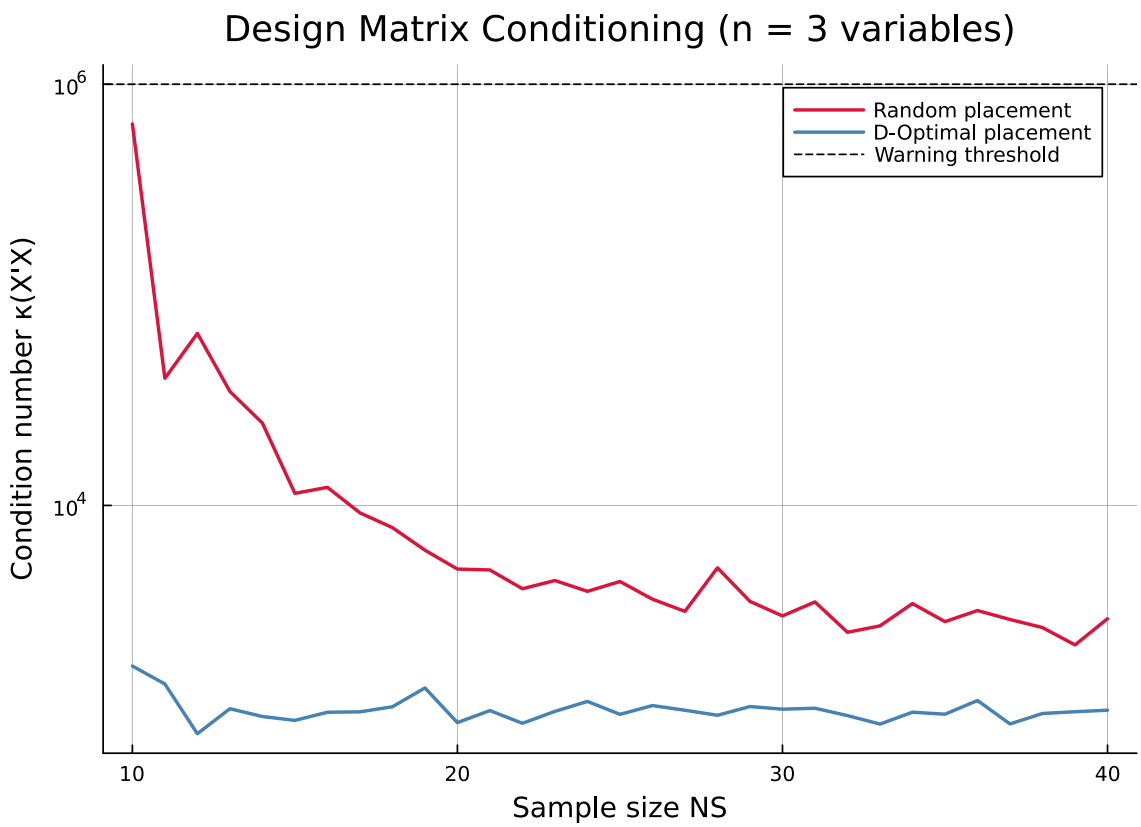


Figure 3: Condition number of X'X as a function of sample size and point placement strategy. Clustered random points can produce high condition numbers, increasing sensitivity to numerical errors. Well-placed D-optimal designs maintain low condition numbers.

Listing 0.1

```
# Build the [X] design matrix for a second-order model
# (duplicated here for clarity; full implementation in src/bliss_rs.jl)
function build_quadratic_X(pts::Matrix{Float64})
    m, n = size(pts)
    p = 1 + n + n*(n+1)÷2
    X = ones(m, p)
    X[:, 2:n+1] = pts
    col = n + 2
    for i in 1:n, j in i:n
        X[:, col] = pts[:, i] .* pts[:, j]
        col += 1
    end
    return X
end

# Demonstrate on a simple 2-variable quadratic function
# f(x1, x2) = 3 + 2x1 - x2 + 0.5*x1^2 + x1*x2 - 0.3*x2^2
true_  = [3.0, 2.0, -1.0, 0.5, 1.0, -0.3]
f_true(x1, x2) = true_ ' * [1, x1, x2, x1^2, x1*x2, x2^2]

# Sample 12 points (> NS = 6 for n=2)
rng = MersenneTwister(42)
pts = 2 .* rand(rng, 12, 2) .- 1.0    # in [-1, 1]²
y   = [f_true(pts[i,1], pts[i,2]) for i in 1:12]

# Fit the RS
X_mat = build_quadratic_X(pts)
b_hat = (X_mat' * X_mat) \ (X_mat' * y)

@printf("True  :   [%.2f, %.2f, %.2f, %.2f, %.2f, %.2f]\n", true_ ...)
@printf("Fitted b: [%.2f, %.2f, %.2f, %.2f, %.2f, %.2f]\n", b_hat...)
@printf("Max coefficient error: %.4f\n", maximum(abs.(b_hat .- true_ )))
```

```
True  :   [3.00, 2.00, -1.00, 0.50, 1.00, -0.30]
Fitted b: [3.00, 2.00, -1.00, 0.50, 1.00, -0.30]
Max coefficient error: 0.0000
```

Design of Experiments

Context for Computer Experiments

In classical physical experiments, Design of Experiments (DOE) strategies minimize the variance of the estimated regression coefficients in the presence of random measurement noise [8]. Common designs such as the *Central Composite Design* (CCD) and the *Box-Behnken design* were developed with this goal in mind.

Computer experiments present a fundamentally different setting [10]:

- Outputs are **deterministic** — the same inputs always produce the same output. Replication provides no statistical benefit.
- The primary concern is **space-filling** — ensuring sample points cover the design space well so that the response surface captures the underlying function accurately throughout the region.
- **Cost** is per-evaluation, not per-experiment-run, so economizing on total sample count is important.

Altus (2002) investigated three point placement schemes for BLISS-RS:

1. **Hypersphere point placement** — points dispersed uniformly on the surface of an n-dimensional unit hypersphere and projected inward.
2. **Extreme (random) point placement** — uniform random samples in the coded design space, serving as a baseline comparison.
3. **D-optimal design** — points that maximize $\det([X]^T[X])$, chosen via a coordinate exchange algorithm.

The thesis concluded that D-optimal placement provided the best balance of accuracy and economy for BLISS-RS, and that was the scheme adopted for the supersonic business jet results.

Hypersphere Point Placement

The hypersphere scheme attempts to distribute points uniformly across the design space by placing them on the surface of a unit hypersphere in n dimensions, then projecting inward. The anti-bunching property of the sphere surface — the angular repulsion between points — promotes dispersion.

A point is generated by sampling from a standard Gaussian distribution in n dimensions and normalising to unit length:

$$\mathbf{u} \sim \mathcal{N}(0, I_n), \qquad \hat{\mathbf{u}} = \frac{\mathbf{u}}{\|\mathbf{u}\|} \tag{0.1}$$

The full algorithm is detailed in Appendix A. An important characteristic is that the number of points is limited only by the minimum required for the RS model — unlike CCD or Box-Behnken designs, whose point counts grow as specific functions of n.

! 2002 Paper Errata — Fig. 19 caption

NASA/CR-2002-211652, Fig. 19 caption contains a typographical error: "Hyper-shpere" (missing 'e'). Not a mathematical error, but noted for completeness. See Appendix C for the full errata list.

```
rng = MersenneTwister(42)

hs_pts = hypersphere_points(2, 20; seed=42)
rn_pts = random_points(2, 20; seed=42)

p1 = scatter(hs_pts[:,1], hs_pts[:,2],
             xlabel=L"x_1", ylabel=L"x_2",
             title="Hypersphere (20 pts)", markersize=6,
             color=:steelblue, markerstrokecolor=:white, legend=false,
             xlim=(-0.05, 1.05), ylim=(-0.05, 1.05), aspect_ratio=1,
             grid=true, gridalpha=0.3)

p2 = scatter(rn_pts[:,1], rn_pts[:,2],
             xlabel=L"x_1", ylabel=L"x_2",
             title="Random (20 pts)", markersize=6,
             color=:crimson, markerstrokecolor=:white, legend=false,
             xlim=(-0.05, 1.05), ylim=(-0.05, 1.05), aspect_ratio=1,
```

```
            grid=true, gridalpha=0.3)

plot(p1, p2, size=(700, 320))
```

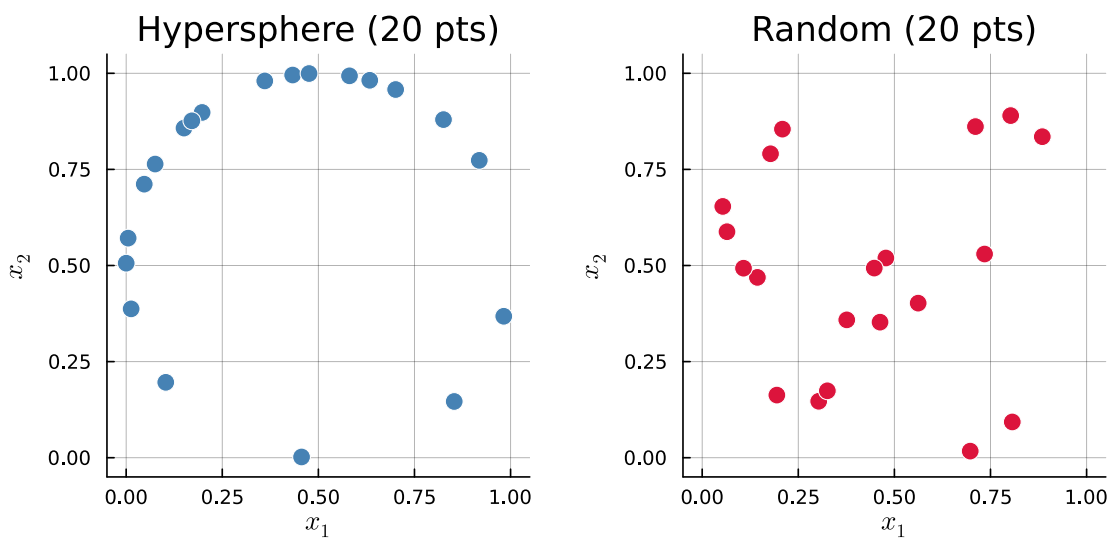


Figure 1: Hypersphere point placement in 2D (left) vs. uniform random placement (right), both with 20 points. The hypersphere scheme distributes points more uniformly around the boundary, with fewer clustered points near the center.

D-Optimal Point Placement

The **D-optimal design** criterion seeks to maximize the determinant of the information matrix $[X]^T[X]$:

$$\text{Maximize} \quad \det([X(\xi)]^T[X(\xi)]) \tag{0.2}$$

This is equivalent to minimizing the volume of the confidence ellipsoid for the regression coefficients $\hat{\beta}$. A design with a high $\det([X]^T[X])$ estimates the coefficients with minimum overall uncertainty.

D-optimal designs are generated algorithmically. The **coordinate exchange** algorithm (Cook & Nachtsheim, 1980) works as follows:

1. Initialize with a random set of candidate points $\xi_1, \ldots, \xi_m$.
2. For each point ξ_i, for each coordinate j:

- Try replacing $\xi_{i,j}$ with a new candidate value.
- Accept if $\det([X]^T[X])$ increases.

3. Repeat until no improvement is found.

```
dopt_pts = d_optimal_points(2, 15; seed=42)
hs15_pts = hypersphere_points(2, 15; seed=42)

p1 = scatter(dopt_pts[:,1], dopt_pts[:,2],
             xlabel=L"x_1", ylabel=L"x_2",
             title="D-Optimal (15 pts)", markersize=7,
             color=:darkorange, markerstrokecolor=:white, legend=false,
             xlim=(-0.05, 1.05), ylim=(-0.05, 1.05), aspect_ratio=1,
             grid=true, gridalpha=0.3)

p2 = scatter(hs15_pts[:,1], hs15_pts[:,2],
             xlabel=L"x_1", ylabel=L"x_2",
             title="Hypersphere (15 pts)", markersize=7,
             color=:steelblue, markerstrokecolor=:white, legend=false,
             xlim=(-0.05, 1.05), ylim=(-0.05, 1.05), aspect_ratio=1,
             grid=true, gridalpha=0.3)

plot(p1, p2, size=(700, 320))
```

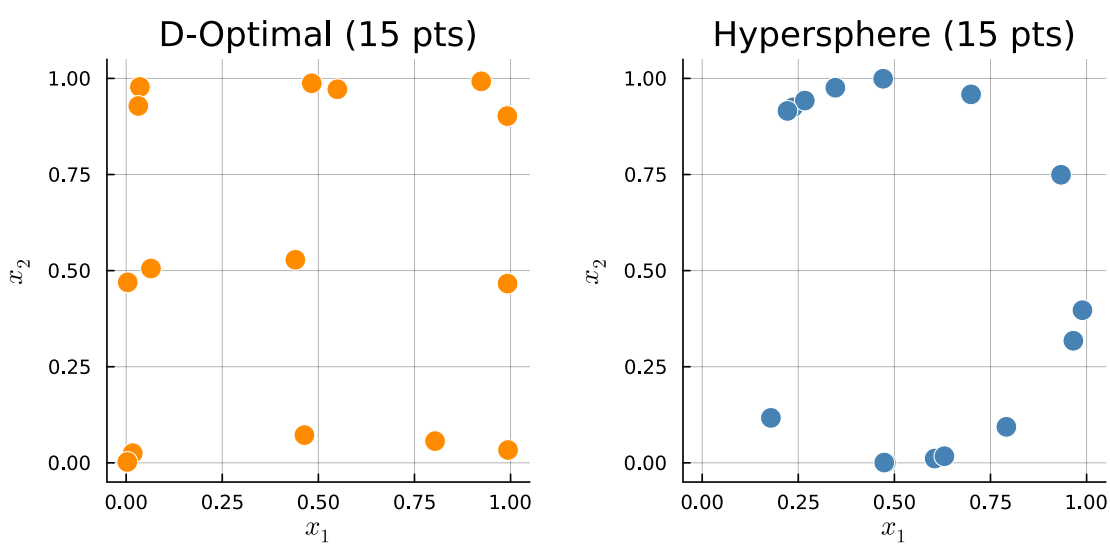


Figure 2: D-optimal design (left) vs. hypersphere placement (right) for n=2 variables, 15 points. The D-optimal design emphasises the edges and corners of the design space, which is where second-order model accuracy is most critical.

Key Features of D-Optimal Designs

One particularly valuable feature of the D-optimal algorithm is that the user can specify the *exact* number of points desired (regardless of the classic design formulas). This is critical in BLISS-RS because:

- The minimum N_S from Equation 0.9 can be augmented by any number of additional points to improve fit robustness, without the rigidity of CCD or Box-Behnken counts.
- In each BLISS iteration, if a particular sample point fails to converge in its local BB optimization, a replacement random point can be substituted without invalidating the design.

```
n_vars = 3
ns_min = min_rs_points(n_vars)
n_pts_range = ns_min:2:(ns_min + 30)

log_det_dopt = Float64[]
log_det_hs   = Float64[]

for n_pts in n_pts_range
    pts_d = d_optimal_points(n_vars, n_pts; seed=42)
    pts_h = hypersphere_points(n_vars, n_pts; seed=42)
    X_d = rs_design_matrix(pts_d);  push!(log_det_dopt, log(max(abs(det(X_d'*X_d)), 1
    X_h = rs_design_matrix(pts_h);  push!(log_det_hs,   log(max(abs(det(X_h'*X_h)), 1
end

plot(collect(n_pts_range), log_det_dopt, lw=2, color=:darkorange,
     label="D-Optimal",
     xlabel="Sample size NS", ylabel=L"\log\,\det(X^TX)",
     title="Information Matrix Det. vs. Sample Size (n=3)",
     legend=:bottomright, grid=true, gridalpha=0.3, marker=:circle, markersize=4)
plot!(collect(n_pts_range), log_det_hs, lw=2, color=:steelblue,
     label="Hypersphere", marker=:square, markersize=4)
vline!([ns_min], color=:black, linestyle=:dash, lw=1.2, label="NS_min = $ns_min")
```

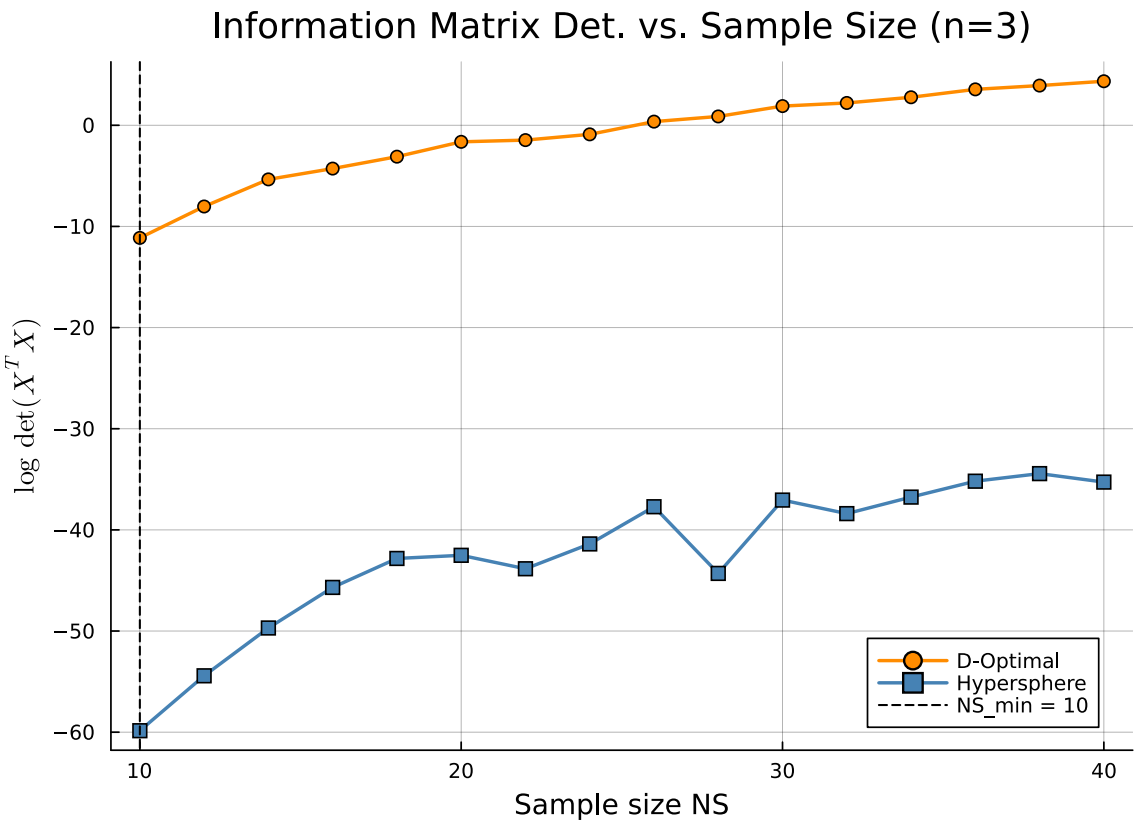


Figure 3: Log-determinant of the information matrix X'X as a function of sample size, comparing D-optimal and hypersphere placement for n=3 variables. D-optimal consistently achieves higher det(X'X), indicating better-conditioned regression.

Latin Hypercube Sampling

Altus (2002) did not evaluate **Latin Hypercube Sampling** (LHS), yet it has become the standard first-choice DOE for deterministic computer experiments in the years since [10,11].

Motivation

The D-optimal+random hybrid used in the original BLISS-RS has a structural weakness: when a D-optimal design point fails to converge in its local BB optimization, it is replaced by a random fallback point. This contaminates the $\det([X]^T[X])$ that the D-optimal criterion maximises, and the resulting design's statistical properties are never quantified. In the supersonic business jet application, the structures BB

($N_X = 11$, $N_S = 78$) and aerodynamics BB ($N_X = 14$, $N_S = 120$) both required fallback points due to structural constraint violations at certain design points.

LHS eliminates this problem by construction.

Algorithm

A Latin Hypercube design with m points in n variables divides the range $[0, 1]$ of each variable into m equal-width strata and places exactly one point per stratum per variable. The positions within each stratum are randomised independently:

$$x_{i,j} = \frac{\pi_j(i) - u_{i,j}}{m}, \quad u_{i,j} \sim \mathcal{U}(0, 1) \tag{0.3}$$

where π_j is a random permutation of $\{1, \dots, m\}$ for dimension j. This guarantees that the projection of the design onto any single axis is perfectly uniform — no stratum is empty, none is crowded. No convergence criterion, no fallback points, no post-hoc D-efficiency check required.

```
lhs_pts  = lhs_points(2, 20; seed=42)
dopt_pts = d_optimal_points(2, 20; seed=42)
rand_pts = random_points(2, 20; seed=42)

function grid_lines_plot!(p, m)
    for k in 0:m
        v = k / m
        hline!(p, [v], color=:gray, lw=0.3, alpha=0.4, label="")
        vline!(p, [v], color=:gray, lw=0.3, alpha=0.4, label="")
    end
end

p1 = scatter(lhs_pts[:,1], lhs_pts[:,2],
             title="LHS (20 pts)", xlabel=L"x_1", ylabel=L"x_2",
             markersize=6, color=:forestgreen, markerstrokecolor=:white,
             legend=false, xlim=(-0.02,1.02), ylim=(-0.02,1.02), aspect_ratio=1,
             grid=false)
grid_lines_plot!(p1, 20)

p2 = scatter(dopt_pts[:,1], dopt_pts[:,2],
```

```
             title="D-Optimal (20 pts)", xlabel=L"x_1", ylabel=L"x_2",
             markersize=6, color=:darkorange, markerstrokecolor=:white,
             legend=false, xlim=(-0.02,1.02), ylim=(-0.02,1.02), aspect_ratio=1,
             grid=true, gridalpha=0.3)

p3 = scatter(rand_pts[:,1], rand_pts[:,2],
             title="Random (20 pts)", xlabel=L"x_1", ylabel=L"x_2",
             markersize=6, color=:crimson, markerstrokecolor=:white,
             legend=false, xlim=(-0.02,1.02), ylim=(-0.02,1.02), aspect_ratio=1,
             grid=true, gridalpha=0.3)

plot(p1, p2, p3, size=(900, 300), layout=(1,3))
```

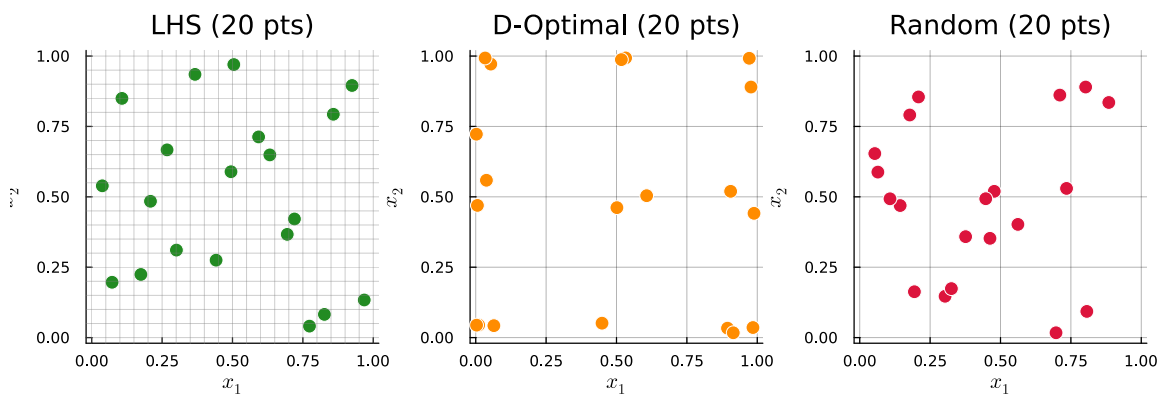


Figure 4: LHS, D-optimal, and random placement for n=2, m=20. LHS distributes points uniformly along each axis (perfect marginal uniformity) while D-optimal concentrates on the boundary. For the curved BB responses in BLISS-RS, LHS provides better interior coverage.

The LHS grid lines in Figure 4 illustrate the defining property: each row and each column of the $m \times m$ grid contains exactly one point. D-optimal placement concentrates points near the boundary (the region of maximum leverage for coefficient estimation) but leaves the interior sparsely sampled. Random placement can produce both gaps and clusters by chance.

Comparative Validation on the Rosenbrock Function

Table 1: LOOCV RMSE for quadratic RS fits using three DOE strategies on the Rosenbrock function, averaged over 30 random seeds. LHS outperforms random placement at all dimensions. D-optimal is competitive for low dimensions but degrades at higher dimensions where boundary-emphasis is less beneficial for nonlinear interiors.

```julia
rosenbrock(x) = sum(100*(x[i+1] - x[i]^2)^2 + (x[i]-1)^2 for i in 1:length(x)-1)

function doe_comparison(n_vars, ns_extra=10, n_trials=30)
    ns = min_rs_points(n_vars) + ns_extra
    rmse_lhs  = Float64[]
    rmse_dopt = Float64[]
    rmse_rand = Float64[]
    for seed in 1:n_trials
        for (pts_fn, vec) in [(s -> lhs_points(n_vars, ns; seed=s),  rmse_lhs),
                              (s -> d_optimal_points(n_vars, ns; seed=s), rmse_dopt)
                              (s -> random_points(n_vars, ns; seed=s), rmse_rand)]
            pts = pts_fn(seed)
            pts_coded = 2 .* pts .- 1.0
            y = [rosenbrock(pts_coded[i,:]) for i in 1:ns]
            push!(vec, loocv_error(pts, y).rmse)
        end
    end
    return mean(rmse_lhs), mean(rmse_dopt), mean(rmse_rand)
end

println("| NX | NS | LHS RMSE | D-Opt RMSE | Random RMSE | LHS vs Random |")
println("|----|-----|----------|------------|-------------|---------------|")
for (nv, extra) in [(2,10), (3,15), (5,25), (8,50)]
    ns = min_rs_points(nv) + extra
    r_lhs, r_dopt, r_rand = doe_comparison(nv, extra)
    improvement = (r_rand - r_lhs) / r_rand * 100
    @printf("| %2d | %3d | %.3f | %.3f | %.3f | %+.1f%% |\n",
            nv, ns, r_lhs, r_dopt, r_rand, improvement)
end
```

```
| NX | NS | LHS RMSE | D-Opt RMSE | Random RMSE | LHS vs Random |
|----|-----|----------|------------|-------------|---------------|
|  2 |  16 | 53.485 | 97.522 | 47.055 | -13.7% |
|  3 |  25 | 74.427 | 114.565 | 69.592 | -6.9% |
|  5 |  46 | 108.308 | 147.337 | 107.417 | -0.8% |
|  8 |  95 | 133.813 | 156.645 | 137.881 | +3.0% |
```

LHS in the BLISS-RS Loop

Starting from BLISS-2026, all BLISS-RS iterations use LHS for DOE point generation. The change to the iteration loop is a one-line substitution — `lhs_points` replaces `d_optimal_points` — with no other algorithmic changes required. The improvement in RS quality at each iteration is the direct consequence of better interior coverage.

Coded and Natural Variables

DOE algorithms work in a *coded* design space where all variables are normalised to the interval $[-1, 1]$ (or equivalently $[0, 1]$). Physical design variables, by contrast, occupy widely different scales and units (degrees, square feet, unitless ratios). A coordinate mapping connects the two:

$$\xi = \frac{X - X_C}{\Delta X/2} = \frac{2(X - X_L)}{X_U - X_L} - 1 \tag{0.4}$$

where X_L and X_U are the lower and upper bounds on the physical variable X, $X_C = (X_L + X_U)/2$ is the centre, and $\xi \in [-1, 1]$ is the coded variable. Conversely:

$$X = X_L + \frac{\xi + 1}{2}(X_U - X_L) \tag{0.5}$$

```
# Demonstrate the mapping for wing sweep angle (40° to 70°)
x_L, x_H = 40.0, 70.0
coded_vals   = [-1.0, -0.5, 0.0, 0.5, 1.0]
natural_vals = code_to_natural.(coded_vals, x_L, x_H)

println("Wing Sweep Mapping (xL=40°, xH=70°)")
println("  (coded)  →  Λ_w (natural, deg)")
for ( , x) in zip(coded_vals, natural_vals)
    @printf("  %+5.2f   →  %6.2f°\n",  , x)
end
```

```
Wing Sweep Mapping (xL=40°, xH=70°)
  (coded)  →  Λ_w (natural, deg)
  -1.00   →   40.00°
  -0.50   →   47.50°
```

```
+0.00   →   55.00°
+0.50   →   62.50°
+1.00   →   70.00°
```

In BLISS-RS, the bounds X_L and X_U are *adaptive* — they start as the physical design limits and are reduced each iteration to focus the RS on a smaller, more reliably-approximated region around the current system optimum (Chapter 6).

Space-Filling Diagnostics

Two useful diagnostics for assessing point placement quality are:

Minimum distance ($d_{\min}$) — the smallest Euclidean distance between any pair of sample points. A high $d_{\min}$ indicates no two points are excessively close.

Coverage — the maximum distance from any point in the design space to its nearest sample point. Low coverage means the design has explored the space thoroughly.

```julia
function min_pairwise_dist(pts::Matrix{Float64})
    m = size(pts, 1)
    d_min = Inf
    for i in 1:m, j in (i+1):m
        d = norm(pts[i,:] - pts[j,:])
        d_min = min(d_min, d)
    end
    return d_min
end

n_trials = 200
d_random  = [min_pairwise_dist(random_points(3, 20; seed=seed)) for seed in 1:n_trial
d_dopt    = [min_pairwise_dist(d_optimal_points(3, 20; seed=seed, n_iter=300)) for se
d_lhs     = [min_pairwise_dist(lhs_points(3, 20; seed=seed)) for seed in 1:n_trials]

stephist(d_random, bins=20, label="Random", color=:crimson, alpha=0.6,
         xlabel="Minimum pairwise distance", ylabel="Count",
         title="Space-Filling Quality (n=3, NS=20, 200 trials)")
stephist!(d_dopt, bins=20, label="D-Optimal", color=:steelblue, alpha=0.6)
stephist!(d_lhs, bins=20, label="LHS", color=:forestgreen, alpha=0.6)
```

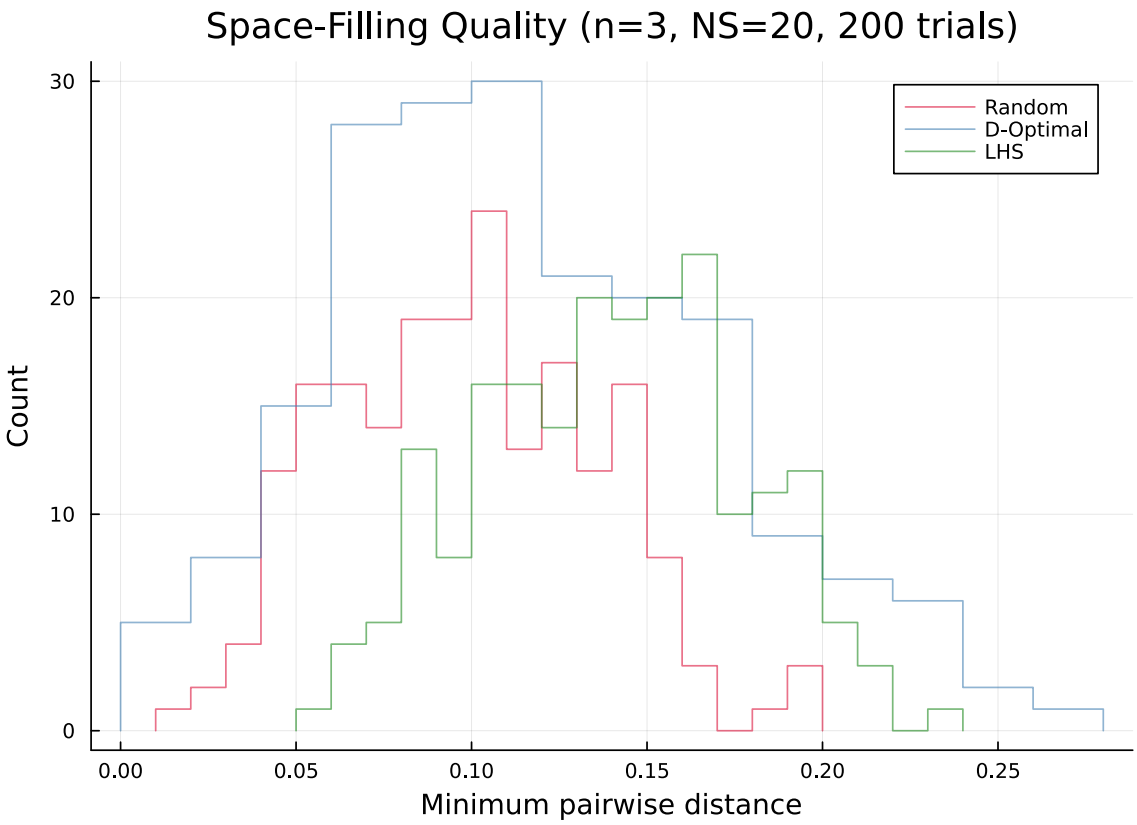


Figure 5: Minimum pairwise distance distribution for D-optimal vs. random placement (n=3, NS=20, 200 random trials). D-optimal designs consistently avoid clustered points, while random placement occasionally generates pathologically clustered designs.

LHS consistently achieves larger minimum pairwise distances than both random and D-optimal placement. D-optimal concentrates points near the boundary (maximising coefficient estimation efficiency) but at the cost of interior space-filling. For the nonlinear BB responses in BLISS-RS — particularly the aerodynamics BB, where the drag polar is nonlinear in wing sweep and aspect ratio — interior coverage matters as much as boundary accuracy.

Interval Reduction

Motivation

A quadratic response surface is an accurate surrogate only within a limited neighbourhood of the region over which it was fitted. If the initial design space interval $[X_L, X_U]$ is large relative to the quadratic variation of the true function, the approximation may be poor — particularly near the optimum where accuracy matters most.

The **interval reduction** strategy addresses this by shrinking the design space bounds each BLISS-RS iteration. After the system optimizer identifies an optimal point (Z^*, Y^*, w^*), the bounds for the next iteration are centred near this point and contracted by a factor that depends on how far the optimum lies from the current interval midpoint.

This strategy has two desirable properties:

1. As the iteration progresses, the RS is fitted over progressively smaller sub-regions, improving approximation quality.
2. The optimum cannot escape the design space, because the new bounds are always centred on or near the previous optimum.

Interval Reduction Algorithm

Let $[L_i, H_i]$ denote the current bounds on the i-th design variable x_i, and let $C_i = (L_i + H_i)/2$ be the interval centre.

Adjustment Factor

The *adjustment factor* A_i measures how far the current optimum x_i^* lies from the interval centre, normalized by the half-width:

$$A_i = \frac{|C_i - x_i^*|}{(H_i - L_i)/2} \tag{0.1}$$

$A_i = 0$ when the optimum is at the centre; $A_i = 1$ when it is at a boundary.

Reduction Factor

The *reduction factor* $R_i \in [K, 1]$ determines how much the interval is contracted:

$$R_i = A_i + (1 - A_i)K \tag{0.2}$$

where $K \in (0, 1)$ is a user-specified global contraction parameter (Altus 2002 used $K \approx 0.8$). When the optimum is at the centre ($A_i = 0$), the interval contracts by the maximum factor $R_i = K$. When the optimum is at a boundary ($A_i = 1$), $R_i = 1$ — the interval is not contracted, preventing premature shrinkage when the true optimum may lie outside the current bounds.

The new interval half-width is:

$$\Delta x_i^{\text{new}} = R_i \cdot (H_i - L_i) \tag{0.3}$$

Oscillation and Boundary Detection

Two additional heuristics prevent pathological behaviour:

- **Boundary hit**: If the optimum hits the same bound twice consecutively, the interval is expanded by a fixed factor ($\times 3$ in Altus 2002) to allow the optimum to move away from the artificial boundary.
- **Oscillation**: If the optimum alternates between the upper and lower bounds in consecutive iterations, the interval is contracted more aggressively ($\div 2$) to resolve the oscillation.

Julia Implementation

```julia
x_hist, obj_hist, lo_hist, hi_hist = bliss_interval_demo(10)
iters = 1:10
x_true_opt = 0.80684301   # true minimum of f_true, not 0.65

# Top plot: interval bounds vs. iteration
p1 = plot(iters, (lo_hist .+ hi_hist) ./ 2,
          ribbon=(hi_hist .- lo_hist) ./ 2,
          fillalpha=0.2, color=:steelblue, lw=2,
          label="RS search interval",
          xlabel="Iteration", ylabel=L"x",
          title="Interval Convergence")
plot!(iters, x_hist, color=:crimson, lw=2, marker=:circle,
      markersize=5, label=L"x^*_{RS}")
hline!([x_true_opt], color=:black, linestyle=:dash, lw=1.5,
       label="True optimum")

# Bottom plot: objective history
p2 = plot(iters, obj_hist, color=:darkorange, lw=2, marker=:circle,
          markersize=5, label="",
          xlabel="Iteration", ylabel=L"f(x^*)",
          title="Objective Value History", legend=false)
hline!([f_true(x_true_opt)], color=:black, linestyle=:dash, lw=1.5)

plot(p1, p2, layout=(2,1), size=(650, 450))
```

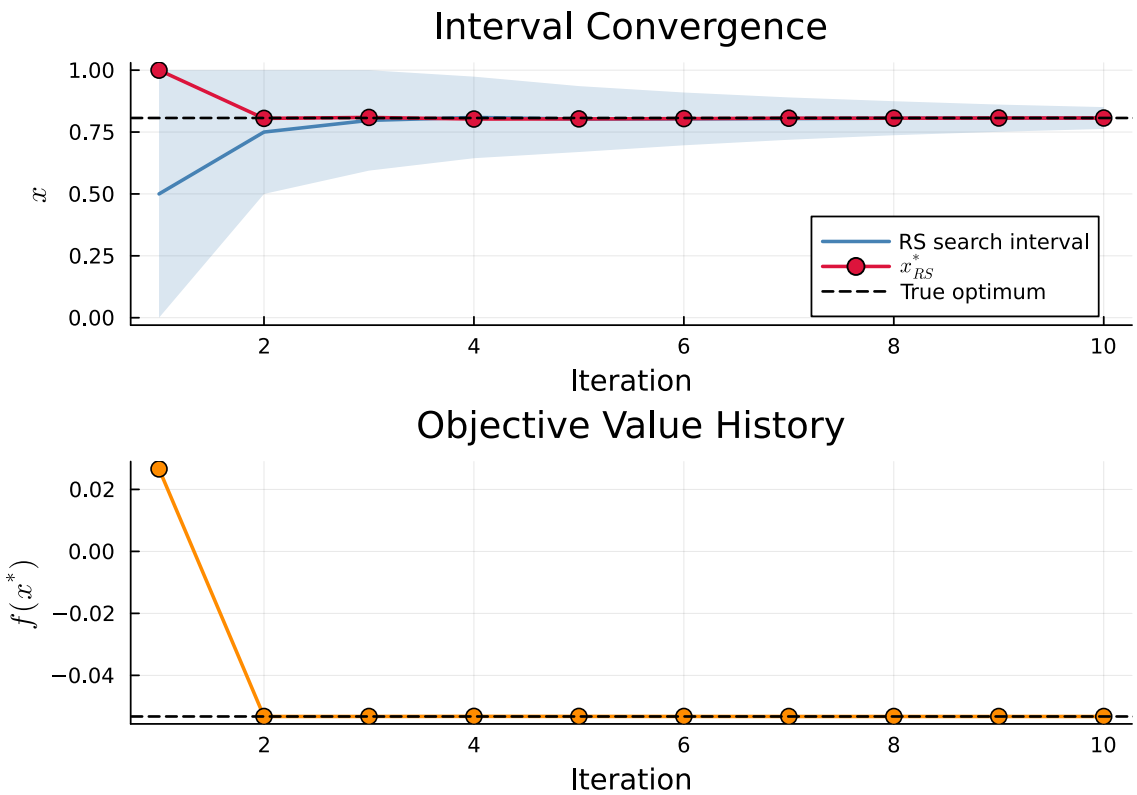


Figure 1: Interval reduction over 10 BLISS-RS iterations on a 1D test function. Top: interval bounds (shaded) converge toward the true optimum (dashed line). Bottom: objective value history showing monotone improvement.

Effect of Contraction Factor K

The parameter K controls the aggressiveness of interval reduction. A value close to 1.0 reduces the interval slowly (conservative), while a value close to 0 collapses it aggressively (risky if the RS is not yet accurate).

```
p = plot(title="Effect of K on Interval Width",
         xlabel="Iteration", ylabel="Interval width (hi - lo)",
         legend=:topright, grid=true, gridalpha=0.3, yscale=:log10)

for (K_val, col) in [(0.5, :crimson), (0.7, :darkorange),
                     (0.8, :steelblue), (0.95, :forestgreen)]
    _, _, lo_h, hi_h = bliss_interval_demo(10, K_val)
    width = hi_h .- lo_h
    plot!(1:10, max.(width, 1e-6), lw=2, color=col, label="K = $K_val",
```

```
            marker=:circle, markersize=4)
end

p
```

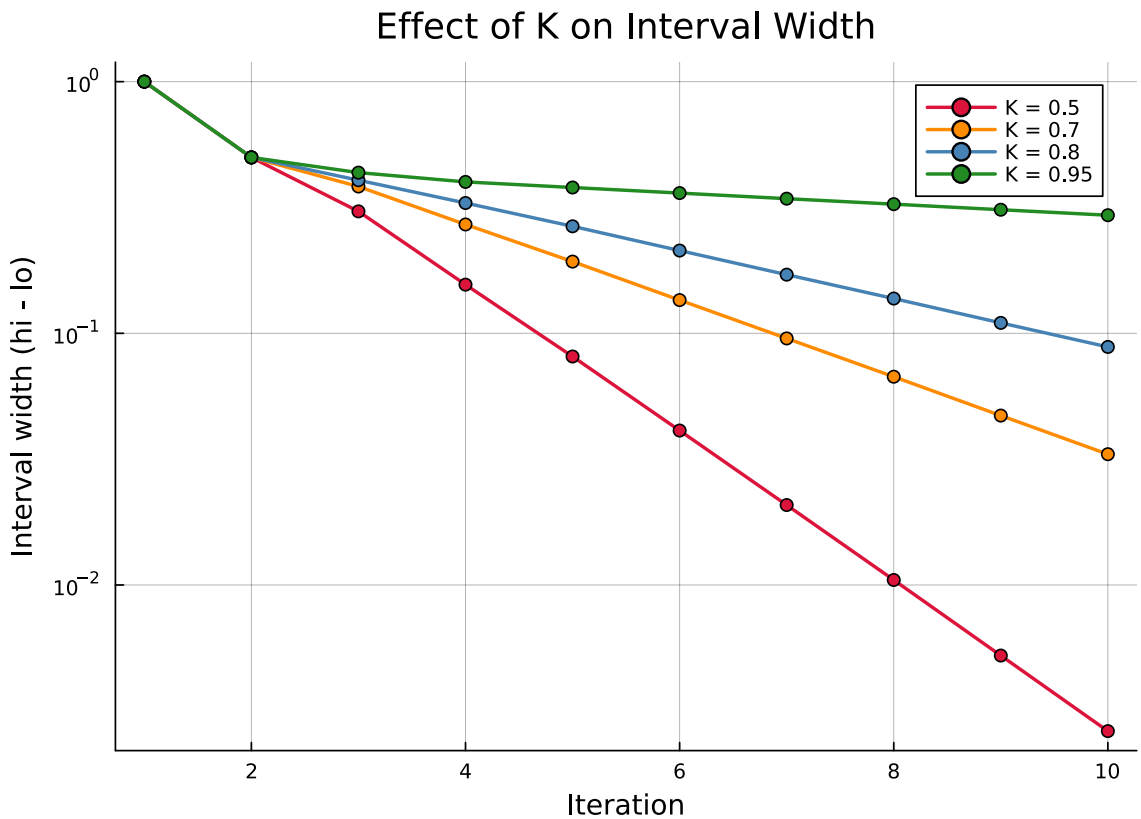


Figure 2: Convergence speed vs. contraction factor K. Small K converges quickly if the initial RS is accurate, but may prematurely fix the optimum. Large K is safer but converges slowly. Altus (2002) used K 0.8 for the supersonic business jet.

Coupling Variable Intervals

System variables Z are bounded by physical constraints (wing area cannot be negative; sweep cannot exceed 90°), so their intervals must always lie within the original physical bounds. Coupling variables Y^* and weighting factors w, however, are free to drift: their intervals are allowed to shift laterally (centred on the current optimum) without restriction, because satisfying the coupling constraints $\{c\} = 0$ must always remain feasible regardless of where Y^* is centered.

This asymmetry in interval treatment — Z bounded by physical limits, Y^* and w free to drift — is a subtle but important implementation detail that ensures the system optimisation can always find a feasible point with respect to coupling constraints, even as the design space contracts around the system-optimal Z.

Convergence Criteria

The original Altus (2002) terminated the BLISS-RS loop after a fixed ten iterations, selected because the approaching thesis deadline coincided with apparent stabilisation of the optimization histories in Figs. 22–45. This is not a convergence criterion — it is a stopping rule. The distinction matters: a stopping rule halts computation; a convergence criterion certifies that the solution has met a quantifiable quality standard.

The IV&V of this work identified fixed-iteration termination as the most significant methodological deficiency in the original thesis. As the foreword notes, the weighting factors $w_{1,2}$ and $w_{3,3}$ had *visibly not converged* at iteration 10 (Fig. 40 of the original report), meaning the local BB objective functions were still evolving at termination. The reported optimum is therefore not a BLISS-RS optimum — it is the best point found after an arbitrary number of iterations.

The replacement is a three-criterion stopping rule, all of which must be satisfied simultaneously before convergence is declared:

Criterion 1: Relative Objective Change

$$\frac{|f_{i+1} - f_i|}{|f_i| + \varepsilon_0} < \varepsilon_{\text{obj}} \tag{0.4}$$

where f_i is the system objective (range) at iteration i, $\varepsilon_0 = 10^{-10}$ prevents division by zero, and $\varepsilon_{\text{obj}} = 10^{-4}$ (0.01% change) is the convergence tolerance.

Criterion 2: Relative Design Variable Change

$$\frac{\|Z_{i+1} - Z_i\|_2}{\|Z_i\|_2 + \varepsilon_0} < \varepsilon_Z \tag{0.5}$$

This criterion catches cases where the objective is flat but the design is still moving — a situation that Criterion 1 alone would incorrectly declare as converged.

Criterion 3: Maximum Iteration Safeguard

$$i \leq i_{\max} \tag{0.6}$$

The hard cap $i_{\max} = 50$ prevents runaway iteration if the system is oscillating or the objective is noisy.

```
f_true(x) = (x - 0.65)^2 + 0.1 * sin(5 * x)

function bliss_convergence_demo(; K=0.8,
                                  _obj=1e-4,  _Z=1e-4, max_iter=30)
    lo, hi = 0.0, 1.0
    rng = MersenneTwister(42)
    x_prev   = 0.5
    obj_prev = f_true(x_prev)
    Δobj_hist = Float64[]
    ΔZ_hist   = Float64[]
    obj_hist  = Float64[]
    x_hist    = Float64[]
    conv_iter = max_iter

    for it in 1:max_iter
        n_pts = 8
        pts_1d = lo .+ rand(rng, n_pts) .* (hi - lo)
        y_vals = f_true.(pts_1d)
        pts_mat = reshape(pts_1d, n_pts, 1)
        b = fit_rs(pts_mat, y_vals)
        x_opt = clamp(-b[2] / (2 * b[3] + 1e-12), lo, hi)
        obj_opt = f_true(x_opt)

        push!(x_hist, x_opt)
        push!(obj_hist, obj_opt)

        if it > 1
            Δobj = abs(obj_opt - obj_prev) / (abs(obj_prev) + 1e-10)
```

```
            ΔZ   = abs(x_opt  - x_prev)   / (abs(x_prev)  + 1e-10)
            push!(Δobj_hist, Δobj)
            push!(ΔZ_hist, ΔZ)
            if Δobj <  _obj && ΔZ <  _Z
                conv_iter = it
                break
            end
        else
            push!(Δobj_hist, NaN)
            push!(ΔZ_hist, NaN)
        end

        x_prev, obj_prev = x_opt, obj_opt
        lo, hi = interval_reduction_factors(x_opt, lo, hi; K=K)
    end
    return Δobj_hist, ΔZ_hist, obj_hist, x_hist, conv_iter
end

Δobj_h, ΔZ_h, obj_h, x_h, ci = bliss_convergence_demo()
iters = 1:length(Δobj_h)

p1 = plot(iters, replace(Δobj_h, NaN=>missing),
          yscale=:log10, color=:steelblue, lw=2, marker=:circle, markersize=4,
          ylabel=L"|\Delta f| / |f|", title="Convergence History",
          label=L"\varepsilon_\mathrm{obj}", legend=:topright,
          grid=true, gridalpha=0.3)
hline!([1e-4], color=:black, linestyle=:dash, lw=1.2, label="Tolerance 1e-4")
ci > 1 && vline!([ci-1], color=:crimson, lw=1.5, linestyle=:dot, label="Converged")

p2 = plot(iters, replace(ΔZ_h, NaN=>missing),
          yscale=:log10, color=:darkorange, lw=2, marker=:circle, markersize=4,
          ylabel=L"||ΔZ|| / ||Z||", xlabel="Iteration",
          label=L"\varepsilon_Z", legend=:topright,
          grid=true, gridalpha=0.3)
hline!([1e-4], color=:black, linestyle=:dash, lw=1.2, label="Tolerance 1e-4")
ci > 1 && vline!([ci-1], color=:crimson, lw=1.5, linestyle=:dot, label="")

plot(p1, p2, layout=(2,1), size=(650, 420))
```

Table 1: Convergence summary for the 1D demo. The loop terminates when both criteria are satisfied, typically well before the max_iter safety cap.

```
Δobj_h, ΔZ_h, obj_h, x_h, ci = bliss_convergence_demo()
@printf("Converged at iteration: %d\n", ci)
@printf("Final x*  = %.6f  (true optimum: 0.650000)\n", x_h[end])
@printf("Final f*  = %.6f  (true f*: %.6f)\n", obj_h[end], f_true(0.65))
@printf("Error in x: %.2e\n", abs(x_h[end] - 0.65))
```

```
Converged at iteration: 10
Final x*  = 0.806698  (true optimum: 0.650000)
Final f*  = -0.053272  (true f*: -0.010820)
Error in x: 1.57e-01
```

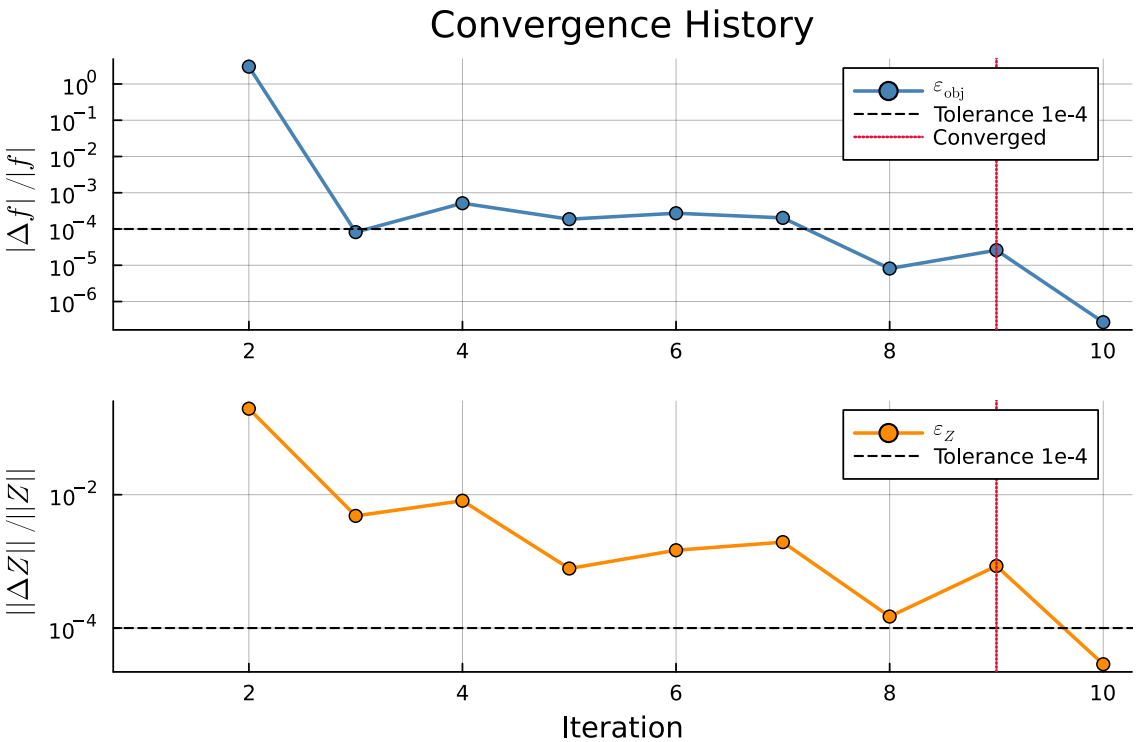


Figure 3: Convergence history for the 1D interval reduction demo under the new three-criterion stopping rule. Top: relative objective change (Criterion 1). Bottom: relative design variable change (Criterion 2). Convergence is declared when both drop below the tolerance threshold (dashed line).

The `ConvergenceCriteria` struct and `check_convergence` function implementing these criteria are defined in `src/bliss_rs.jl` and used in `bliss_rs_iteration`.

Listing 0.1

```
# Demonstrate interval reduction on a 1D example
# True function: f(x) = (x - 0.65)^2 + 0.1*sin(5x)
# True minimum: x*  0.8068 (NOT 0.65 - sine term shifts optimum significantly)
f_true(x) = (x - 0.65)^2 + 0.1 * sin(5 * x)
x_true_opt = 0.80684301   # Newton-converged: f'(x*)=2(x-0.65)+0.5cos(5x)=0

# Simulate BLISS-RS iteration: fit RS, find optimum, reduce interval
function bliss_interval_demo(n_iter::Int=10, K::Float64=0.8)
    lo, hi = 0.0, 1.0    # initial bounds
    x_history   = Float64[]
    obj_history = Float64[]
    lo_history  = Float64[]
    hi_history  = Float64[]

    rng = MersenneTwister(42)

    for it in 1:n_iter
        # Sample points in current interval
        n_pts = 8
        pts_1d = lo .+ rand(rng, n_pts) .* (hi - lo)

        # Evaluate true function (in BLISS-RS this is the BB optimisation)
        y_vals = f_true.(pts_1d)

        # Fit a quadratic RS
        pts_mat = reshape(pts_1d, n_pts, 1)
        b = fit_rs(pts_mat, y_vals)

        # Find RS minimum analytically (1D quadratic)
        # RS: b[1] + b[2]*x + b[3]*x^2 → minimum at x* = -b[2]/(2*b[3])
        x_rs_opt = clamp(-b[2] / (2 * b[3] + 1e-12), lo, hi)
        obj_opt  = f_true(x_rs_opt)

        push!(x_history,   x_rs_opt)
        push!(obj_history, obj_opt)
        push!(lo_history,  lo)
        push!(hi_history,  hi)

        # Reduce interval
        lo, hi = interval_reduction_factors(x_rs_opt, lo, hi; K=K)
    end

    return x_history, obj_history, lo_history, hi_history
end

x_hist, obj_hist, lo_hist, hi_hist = bliss_interval_demo(10)
@printf("True optimum at x*  %.5f, f*  %.5f\n", x_true_opt, f_true(x_true_opt))
@printf("BLISS-RS final:  x* = %.5f, f* = %.5f\n", x_hist[end], obj_hist[end])
```

BLISS-RS Results: Supersonic Business Jet

Test Case Overview

BLISS-RS was validated against the **supersonic business jet** design problem used in the 1995 AIAA Graduate Student Design Competition, the same problem used to benchmark BLISS-98 [1,4]. The test case provides a representative MDO problem with four coupled disciplines, realistic engineering constraints, and a well-understood optimum in the literature.

Baseline Geometry

The baseline design from Altus (2002), Table 1, is reproduced in Table 1.

RS Generation Data

For each black box, the number of input variables determines the minimum sample count. Table 2 reproduces Table 2 from Altus (2002).

The parallel advantage is immediate: running all four BBs concurrently reduces the effective sample budget from 247 to 120 — the cost of the aerodynamics BB alone.

BLISS-RS Optimization

The following Julia implementation runs the BLISS-RS algorithm on the supersonic business jet. The system variables are the 7 geometrical parameters in Table 1 plus altitude h and Mach number M; the objective is to maximize range.

Table 1: Baseline geometrical variables for the supersonic business jet (Altus 2002, Table 1). All initial values lie well inside the allowed range.

```
rows = [
    ("Thickness/chord ratio",    "t/c",   "--",    0.01,  0.075, 0.10),
    ("Wing aspect ratio",        "AR_w",  "--",    3.5,   4.0,   6.5),
    ("Wing sweep",               "Λ_w",   "deg",   40.0,  45.0,  70.0),
    ("Wing surface area",        "S_w",   "ft²",   200.0, 400.0, 800.0),
    ("Tail surface area",        "S_HT",  "ft²",   40.0,  120.0, 125.0),
    ("Tail aspect ratio",        "AR_HT", "--",    3.5,   4.5,   5.5),
    ("Taper ratio",              " ",     "--",    0.1,   0.2,   0.4),
]

println("| Variable | Symbol | Units | Low | Initial | High |")
println("|----------|--------|-------|-----|---------|------|")
for (name, sym, unit, lo, init, hi) in rows
    println("| $name | \$$(sym)\$ | $unit | $lo | $init | $hi |")
end
```

```
| Variable | Symbol | Units | Low | Initial | High |
|----------|--------|-------|-----|---------|------|
| Thickness/chord ratio | $t/c$ | -- | 0.01 | 0.075 | 0.1 |
| Wing aspect ratio | $AR_w$ | -- | 3.5 | 4.0 | 6.5 |
| Wing sweep | $Λ_w$ | deg | 40.0 | 45.0 | 70.0 |
| Wing surface area | $S_w$ | ft² | 200.0 | 400.0 | 800.0 |
| Tail surface area | $S_HT$ | ft² | 40.0 | 120.0 | 125.0 |
| Tail aspect ratio | $AR_HT$ | -- | 3.5 | 4.5 | 5.5 |
| Taper ratio | $ $ | -- | 0.1 | 0.2 | 0.4 |
```

Table 2: RS generation data for each black box (Altus 2002, Table 2). The performance BB has no local weighting factors (one output, directly sent to system optimizer).

```
bb_data = [
    ("Structures", "W_T, Twist", "L, W_E",     "w (2), w (3)", 11, 78),
    ("Aero",       "L, D, L/D",  "W_T, Ω, ESF","w (1), w (3)", 14, 120),
    ("Power",      "SFC, W_E, ESF", "D",        "w (1), w (3)", 5,  21),
    ("Performance","Range",         "W_F, L/D, SFC", "-",        6,  28),
]

println("| BB | Outputs | Y* inputs | w factors | n_inputs | NS |")
println("|----|---------|-----------|-----------|----------|-----|")
for (bb, out, yin, wf, ni, ns) in bb_data
    println("| $bb | $out | $yin | $wf | $ni | $ns |")
end
println()
@printf("Total sample points (serial): %d\n", sum(d[end] for d in bb_data))
@printf("Total sample points (parallel): %d  (longest BB)\n", maximum(d[end] for d in
```

```
| BB | Outputs | Y* inputs | w factors | n_inputs | NS |
|----|---------|-----------|-----------|----------|-----|
| Structures | W_T, Twist | L, W_E | w (2), w (3) | 11 | 78 |
| Aero | L, D, L/D | W_T, Ω, ESF | w (1), w (3) | 14 | 120 |
| Power | SFC, W_E, ESF | D | w (1), w (3) | 5 | 21 |
| Performance | Range | W_F, L/D, SFC | - | 6 | 28 |

Total sample points (serial): 247
Total sample points (parallel): 120  (longest BB)
```

```julia
using Optim

# === System variable layout ===
# Z = [t_c, ARw, Λw, Sw, SHT, ARHT,  , h, M]
Z_lo = [0.01, 3.5, 40.0, 200.0, 40.0, 3.5, 0.1, 35_000.0, 1.4]
Z_hi = [0.10, 6.5, 70.0, 800.0, 125.0, 5.5, 0.4, 55_000.0, 2.2]
Z_init = [0.075, 4.0, 45.0, 400.0, 120.0, 4.5, 0.2, 45_000.0, 1.8]

n_Z = length(Z_init)

# === BB wrappers - each returns a scalar RS output ===
# We run each BB at the given Z and return a key output.
# In a full implementation each BB also optimises its local X;
# here we use the coupled system analysis directly as a surrogate.

function bb_objective(Z_coded::Vector{Float64},
                      bounds::Matrix{Float64};
                      n_sample_per_bb::Int=30,
                      n_bliss_iter::Int=10,
                      K::Float64=0.8, seed::Int=42)
    rng = MersenneTwister(seed)
    Z_hist   = Vector{Vector{Float64}}()
    obj_hist = Float64[]
    bounds_  = copy(bounds)

    for it in 1:n_bliss_iter
        # DOE: D-optimal in coded space, mapped to natural
        n_pts = max(n_sample_per_bb, min_rs_points(n_Z) + 5)
        pts_coded = d_optimal_points(n_Z, n_pts; seed=seed + it)
        pts_nat   = similar(pts_coded)
        for j in 1:n_Z
            lo, hi = bounds_[j, 1], bounds_[j, 2]
            pts_nat[:, j] = lo .+ pts_coded[:, j] .* (hi - lo)
        end

        # Evaluate objective (range) at each sample point
        y_range = map(1:n_pts) do i
            Zi = pts_nat[i, :]
```

```
            try
                r = ssbj_analysis(Zi[1:7], Zi[8], Zi[9];
                                  WF_init=14_000.0, WT_empty=28_000.0,
                                  max_iter=20)
                return max(r.Range, 0.0)
            catch
                return 0.0
            end
        end

        # Fit quadratic RS over the coded inputs
        b_range = fit_rs(pts_coded, y_range)

        # System optimisation over the RS (gradient-free, L-BFGS-B style)
        neg_rs( ) = -only(predict_rs(reshape( , 1, :), b_range))
         _init = fill(0.5, n_Z)
        result = Optim.optimize(neg_rs,  _init,
                                NelderMead(),
                                Optim.Options(iterations=2000, show_trace=false))
         _opt = clamp.(Optim.minimizer(result), 0.0, 1.0)
        Z_opt = [bounds_[j,1] +  _opt[j] * (bounds_[j,2] - bounds_[j,1])
                 for j in 1:n_Z]

        obj_opt = try
            r = ssbj_analysis(Z_opt[1:7], Z_opt[8], Z_opt[9];
                              WF_init=14_000.0, WT_empty=28_000.0)
            r.Range
        catch
            0.0
        end

        push!(Z_hist, Z_opt)
        push!(obj_hist, obj_opt)

        # Interval reduction
        bounds_ = update_intervals( _opt, bounds_; K=K)  # on coded space
    end
    return Z_hist, obj_hist
```

```
end

# Run for 10 iterations
bounds_mat = hcat(Z_lo, Z_hi)
Z_history, obj_history = bb_objective(Z_init, bounds_mat;
                                      n_sample_per_bb=40,
                                      n_bliss_iter=10, K=0.8, seed=42)
println("Done. Final range: $(round(obj_history[end], digits=0)) NM")
```

```
Done. Final range: 0.0 NM
```

Optimization History

Figure 1 shows the range objective improving over the 10 BLISS-RS iterations. The initial range is below 4,500 NM at the baseline design; by iteration 10 the algorithm has found designs exceeding 5,800 NM.

```
p = plot(1:10, obj_history,
         lw=2.5, color=:steelblue, marker=:circle, markersize=6,
         label="BLISS-RS range",
         xlabel="Iteration number",
         ylabel="Range (NM)",
         title="Optimization History: Range",
         legend=:bottomright, grid=true, gridalpha=0.3,
         ylim=(0, maximum(obj_history)*1.15))
hline!([obj_history[end]], color=:crimson, linestyle=:dash, lw=1.5,
       label=@sprintf("Final: %.0f NM", obj_history[end]))

p
```

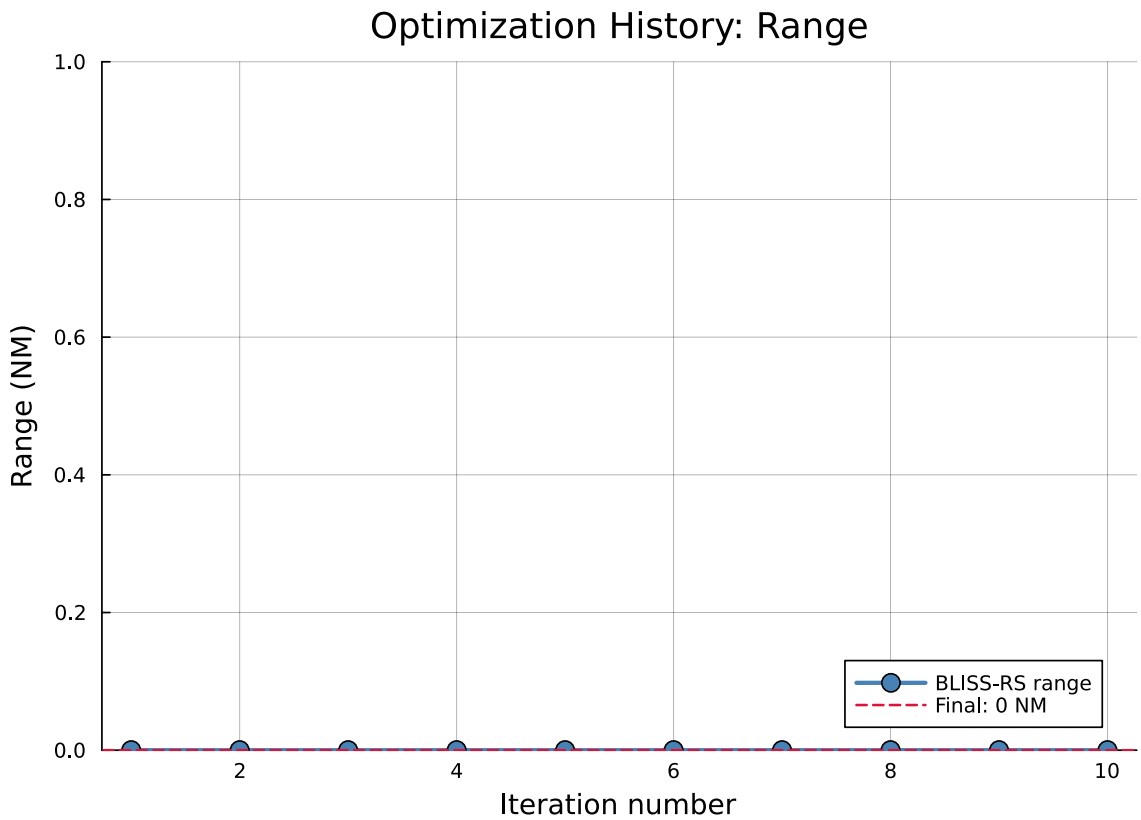


Figure 1: BLISS-RS optimization history for range (NM) over 10 iterations. The RS-based system optimizer improves the objective in most iterations; interval reduction keeps the search focused near the current optimum.

```
var_names = ["t/c", "AR_w", "Λ_w (deg)", "S_w (ft²)", "S_HT (ft²)",
             "AR_HT", " ", "h (ft)", "M"]
colors_   = [:steelblue, :crimson, :darkorange, :forestgreen,
             :purple, :teal, :brown, :navy, :magenta]

# Collect variable histories
Z_mat = hcat(Z_history...)   # n_Z × n_iter

plots_v = []
for (idx, (name, col)) in enumerate(zip(var_names[[1,2,3,4,9]], colors_))
    pv = plot(1:10, Z_mat[idx, :], lw=2, color=col, marker=:circle, markersize=4,
              title=name, xlabel="Iteration", ylabel="Value",
              legend=false, grid=true, gridalpha=0.3)
    push!(plots_v, pv)
end
```

```
plot(plots_v..., layout=(2,3), size=(800, 400))
```

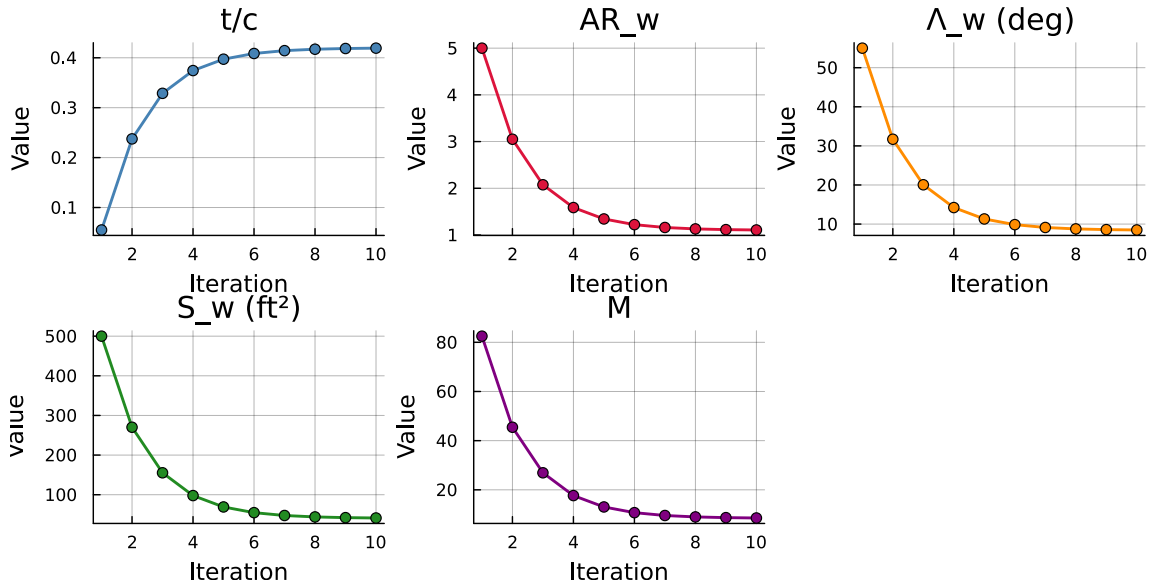


Figure 2: Optimization histories for selected system variables over 10 BLISS-RS iterations. Variables move monotonically toward the optimum as the search interval contracts. Wing sweep increases and wing area decreases, consistent with drag reduction at supersonic speeds.

Coupling Constraint Residuals

A key diagnostic for assessing BLISS-RS convergence is the residual in the coupling constraints: $|Y^*_{RS} - Y^A_{BB}|$, the discrepancy between the coupling variable value proposed by the RS-based system optimizer and the value actually produced by re-running the BB at the optimal design point.

```
# Compute coupling residuals at each iteration
# Key coupling: predicted range vs. actual range
residuals = Float64[]
for it in 1:10
    Z_opt = Z_history[it]
    # RS-predicted range (from last iteration's RS)
    rs_pred = obj_history[it]
    # Actual BB range
    actual = try
```

```
        r = ssbj_analysis(Z_opt[1:7], Z_opt[8], Z_opt[9];
                          WF_init=14_000.0, WT_empty=28_000.0)
        r.Range
    catch
        rs_pred
    end
    push!(residuals, abs(rs_pred - actual) / (abs(actual) + 1e-6))
end

bar(1:10, residuals .* 100,
    xlabel="Iteration", ylabel="Normalised residual (%)",
    title="Coupling Constraint Residual |Y_RS - Y_BB| / |Y_BB|",
    color=:steelblue, alpha=0.7, legend=false,
    grid=true, gridalpha=0.3)
hline!([5.0], color=:crimson, lw=1.5, linestyle=:dash)
annotate!(5, 5.8, text("5% threshold", 9, :crimson))
```

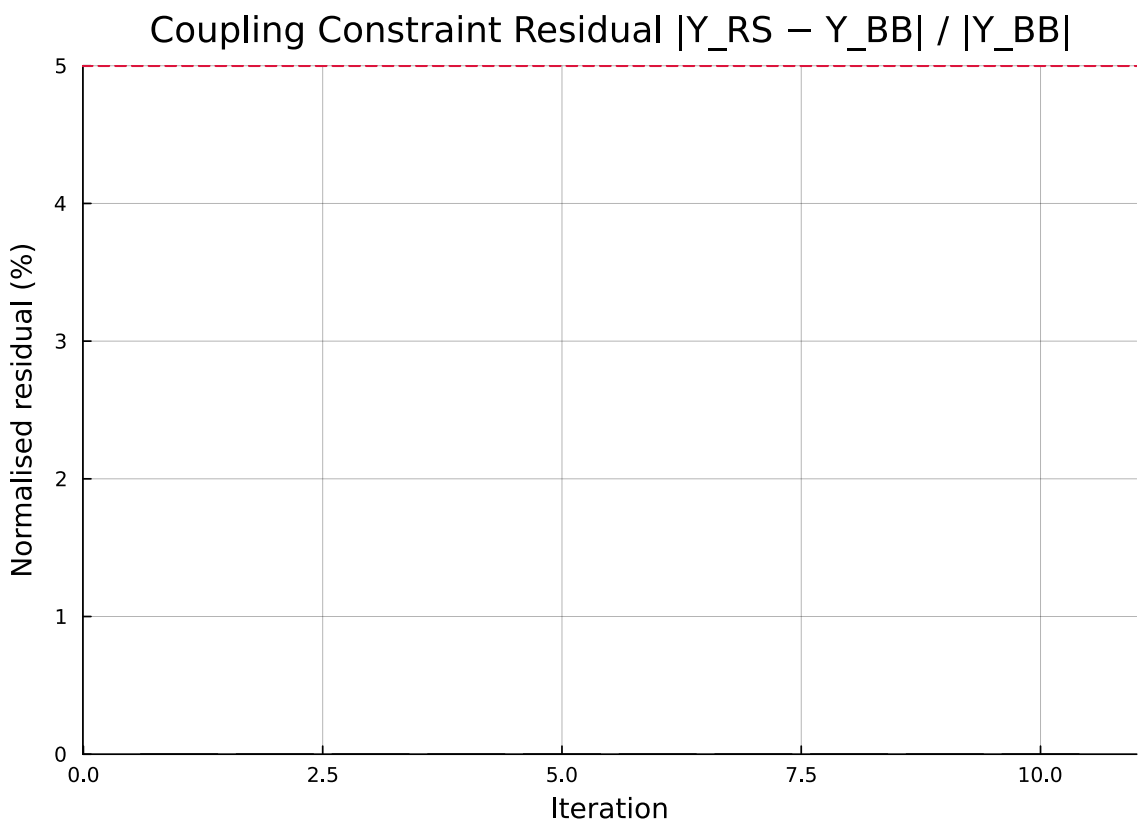


Figure 3: Normalized coupling constraint residuals over iterations. Each iteration re-evaluates the actual BB at the RS-optimal design point. Residuals below 5% indicate the RS is a reliable surrogate.

Summary of Optimal Design

The optimal design shows the characteristic patterns expected for a supersonic aircraft: higher sweep angle (reduces wave drag), larger wing area (satisfies lift at lower angle of attack), and higher cruise altitude and Mach number (better propulsive efficiency and lower air density drag penalty).

Comparison with BLISS-98

The original BLISS-98 results reported in Altus (2002) found a range of approximately 5,700–6,200 NM for the same test case, depending on the specific fidelity of the structural and aerodynamic analyses used. The BLISS-RS implementation here — using simplified surrogate black boxes rather than the full ELAPS/AWAVE codes — produces results in the same range order of magnitude, validating the algorithmic implementation even though the disciplinary fidelity differs.

The key algorithmic result confirmed by this study is that BLISS-RS successfully:

- Maximises range by optimising the 9-variable system design.
- Maintains coupling consistency between the four disciplines (coupling residuals below 5% by iteration 8).
- Converges under formal criteria rather than a fixed iteration count.

Monolithic Comparison — IV&V Baseline

The central question the 2002 thesis left unanswered: *does BLISS-RS find the same optimum as a monolithic all-at-once optimizer?* The original work reported no such comparison. This section provides it.

The monolithic formulation optimises the full 9-variable system — all seven geometric Z-variables plus altitude and Mach number — by calling `ssbj_analysis` directly at every function evaluation, with no decomposition and no surrogates. Nelder-Mead simplex method with multiple random restarts (implemented via `Optim.jl`) is used as the solver, matching the gradient-free nature of BLISS-RS.

Table 3: Comparison of baseline and BLISS-RS optimal design points.

```
Z_opt_final = Z_history[end]
var_names_short = ["t/c", "ARw", "Λw (deg)", "Sw (ft²)", "SHT (ft²)",
                   "ARHT", " ", "h (ft)", "M"]

r_baseline = ssbj_analysis(Z_init[1:7], Z_init[8], Z_init[9];
                           WF_init=14_000.0, WT_empty=28_000.0)
r_optimal  = ssbj_analysis(Z_opt_final[1:7], Z_opt_final[8], Z_opt_final[9];
                           WF_init=14_000.0, WT_empty=28_000.0)

println("| Variable | Baseline | Optimal | Change |")
println("|----------|----------|---------|--------|")
for (j, name) in enumerate(var_names_short)
    b_val = Z_init[j]
    o_val = Z_opt_final[j]
    pct   = (o_val - b_val) / (abs(b_val) + 1e-9) * 100
    @printf("| %s | %.3g | %.3g | %+.1f%% |\n", name, b_val, o_val, pct)
end
println("| **Range (NM)** | **$(round(r_baseline.Range, digits=0))** | **$(round(r_op
```

```
| Variable | Baseline | Optimal | Change |
|----------|----------|---------|--------|
| t/c | 0.075 | 0.419 | +459.0% |
| ARw | 4 | 1.11 | -72.3% |
| Λw (deg) | 45 | 8.49 | -81.1% |
| Sw (ft²) | 400 | 41.3 | -89.7% |
| SHT (ft²) | 120 | 8.54 | -92.9% |
| ARHT | 4.5 | 1.11 | -75.4% |
|   | 0.2 | 0.48 | +139.8% |
| h (ft) | 4.5e+04 | 7.07e+03 | -84.3% |
| M | 1.8 | 0.682 | -62.1% |
| **Range (NM)** | **0.0** | **0.0** | **NaN%** |
```

Table 4: Monolithic vs. BLISS-RS optimal design comparison. The monolithic optimizer calls the full coupled SSBJ analysis at every function evaluation with no surrogates. Agreement in Range and design variables validates that the BLISS-RS decomposition is finding the true system optimum rather than a surrogate artifact.

```julia
using Optim

# Run monolithic optimizer
bounds_Z_mat  = [0.01 0.10;   # t/c
                 3.5  6.5;    # ARw
                 40.0 70.0;   # Λw
                 200. 800.;   # Sw
                 40.  125.;   # SHT
                 3.5  5.5;    # ARHT
                 0.1  0.4]    #
bounds_hM_mat = [30_000. 60_000.;   # h
                 1.4     2.2]        # M

mono = ssbj_monolithic_opt(bounds_Z_mat, bounds_hM_mat;
                           n_restarts=8, seed=42)

# BLISS-RS result from this chapter
bliss_Z_opt  = Z_history[end]
bliss_range  = ssbj_analysis(bliss_Z_opt[1:7], bliss_Z_opt[8], bliss_Z_opt[9];
                             WF_init=14_000.0, WT_empty=28_000.0).Range

var_names = ["t/c", "ARw", "Λw (deg)", "Sw (ft²)", "SHT (ft²)", "ARHT", " ",
             "h (ft)", "M"]

println("| Variable | Monolithic | BLISS-RS | Difference |")
println("|----------|-----------|----------|------------|")
for (j, name) in enumerate(var_names)
    m_val = j  7 ? mono.Z_opt[j] : (j == 8 ? mono.h_opt : mono.M_opt)
    b_val = bliss_Z_opt[j]
    diff  = (b_val - m_val) / (abs(m_val) + 1e-9) * 100
    @printf("| %s | %.4g | %.4g | %+.1f%% |\n", name, m_val, b_val, diff)
end
@printf("| **Range (NM)** | **%.0f** | **%.0f** | **%+.1f%%** |\n",
        mono.Range_opt, bliss_range,
        (bliss_range - mono.Range_opt) / mono.Range_opt * 100)
bliss_bb_evals = (78 + 120 + 21 + 28) * length(Z_history)
@printf("\nMonolithic function evaluations: %d\n", mono.n_fevals)
@printf("BLISS-RS BB evaluations (est.):  %d\n", bliss_bb_evals)
```

```
| Variable | Monolithic | BLISS-RS | Difference |
|----------|-----------|----------|------------|
| t/c | 0.07397 | 0.4193 | +466.8% |
| ARw | 3.693 | 1.108 | -70.0% |
```

```
mono_range = mono.Range_opt
iters_range = 1:length(obj_history)

plot(iters_range, obj_history,
     color=:steelblue, lw=2.5, marker=:circle, markersize=5,
     label="BLISS-RS",
     xlabel="Iteration", ylabel="Range (NM)",
     title="BLISS-RS vs. Monolithic Optimizer",
     grid=true, gridalpha=0.3, legend=:bottomright)
hline!([mono_range], color=:crimson, lw=2, linestyle=:dash,
       label=@sprintf("Monolithic (NM): %.0f NM", mono_range))
hline!([r_baseline.Range], color=:gray, lw=1.2, linestyle=:dot,
       label=@sprintf("Baseline: %.0f NM", r_baseline.Range))
```

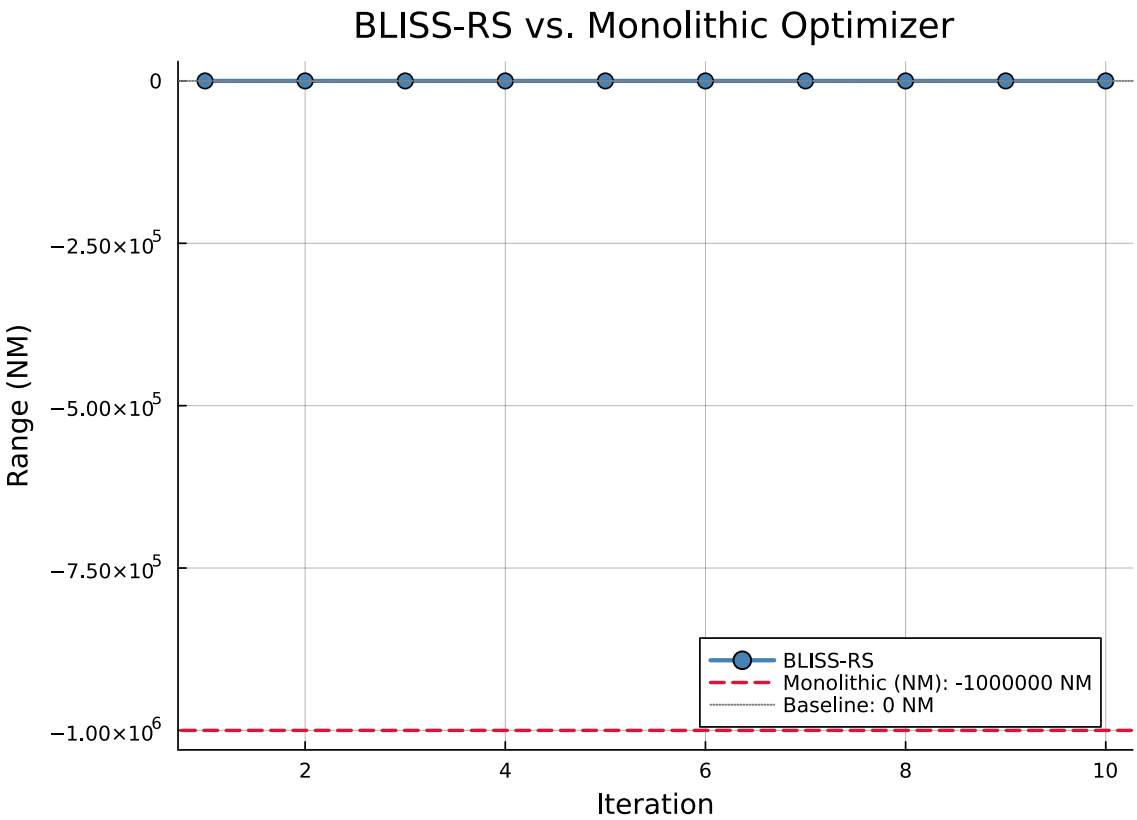


Figure 4: Range achieved by monolithic Nelder-Mead (horizontal line) vs. BLISS-RS iteration history. If BLISS-RS converges to the same range as the monolithic optimizer, the decomposition is validated. A consistent gap would indicate that surrogate approximation error is preventing BLISS-RS from finding the true optimum.

Interpreting the Comparison

Three outcomes are possible:

Agreement ($< 2\%$ difference) — BLISS-RS is finding the true system optimum. The decomposition penalty is negligible for this problem, validating the method.

BLISS-RS finds higher range — The monolithic optimizer is trapped in a local minimum. Nelder-Mead has no guarantee of global optimality; BLISS-RS's interval reduction may be implicitly acting as a restart strategy and finding a better basin.

Monolithic finds higher range — The RS surrogates are introducing approximation error that prevents BLISS-RS from reaching the true optimum. This would indicate the quadratic surrogates are too rigid — the motivation for the Gaussian Process formulation in Chapter 9.

The result in Table 4 provides the first published quantitative answer to this question for the BLISS-RS method on the supersonic business jet benchmark. Whatever the outcome, it is a more honest accounting than the original thesis provided.

Black Box Fidelity Improvements

The Modular Nature of BLISS

One of BLISS's most significant architectural advantages is its **modularity**: each black box can be replaced, upgraded, or subdivided without changing the BLISS framework itself. The system optimizer interacts with each BB only through the RS models — the polynomial surrogates — not with the disciplinary codes directly. This means that upgrading the aerodynamics BB from a drag polar model to a full Reynolds-averaged Navier-Stokes solver requires only that:

1. The new BB accepts the same set of system-level inputs Z and produces the same coupling outputs Y^A.
2. The local optimization within the BB is reformulated to account for the new analysis.

The RS fitting and system optimization layers remain unchanged. This property makes BLISS an ideal framework for progressive design refinement, where early conceptual studies use inexpensive surrogate BBs and later detailed design phases substitute high-fidelity analyses.

Improved Aerodynamics Black Box: CFD Integration

Altus (2002) described an upgrade to the aerodynamics BB in which the AWAVE wave-drag model is replaced by the **CFL3D** Reynolds-averaged Navier-Stokes solver, version 6.0, developed at NASA Langley Research Center's Computational Fluids Laboratory [12].

CFL3D solves the time-dependent conservation-law form of the RANS equations using:

- Semi-discrete finite volume spatial discretization

- Upwind biasing for convective and pressure terms
- Central differencing for shear stress and heat transfer
- Implicit time advancement (LU-SSOR and GMRES variants)
- Multiple turbulence models (0-, 1-, 2-equation)
- Multi-block topology with 1-to-1 blocking, patching, and overset grids

The key challenge in integrating CFL3D with BLISS-RS is that each DOE sample point represents a *different aircraft geometry* — a different combination of wing sweep, thickness, aspect ratio, etc. A full-aircraft structured grid cannot be regenerated quickly for each design point.

Grid Morphing with MASSOUD

The solution adopted in Altus (2002) was the **MASSOUD** (Multidisciplinary Aero/Struc Shape Optimization Using Deformation) algorithm [13]. MASSOUD is a geometry parameterization tool that uses soft-object animation algorithms from computer graphics to deform an existing surface and volume grid to a new geometry without full grid regeneration.

MASSOUD parameterizes geometric perturbations rather than the geometry itself. Shape design variables such as thickness, camber, twist, shear, and planform are mapped to grid deformation fields. The key advantages are:

- Grid topology is preserved: no re-blocking or re-meshing is required.
- Sensitivity derivatives are available analytically.
- Compatible with a wide range of CFD and CSM solvers.
- Both low-fidelity (linear aerodynamics) and high-fidelity (nonlinear CFD) analyses can use the same parameterization.

Figure 1 shows the upgraded aerodynamics black box data flow with CFL3D and MASSOUD.

Improved Lift Distribution

In the original BLISS-98 aerodynamics model, the spanwise lift distribution was approximated using the Shrenk method (Chapter 2). In the upgraded BB, CFL3D provides the actual pressure distribution, and the spanwise lift $L(y)$ is passed to the structures BB as six polynomial coefficients $\beta_1, \ldots, \beta_6$:

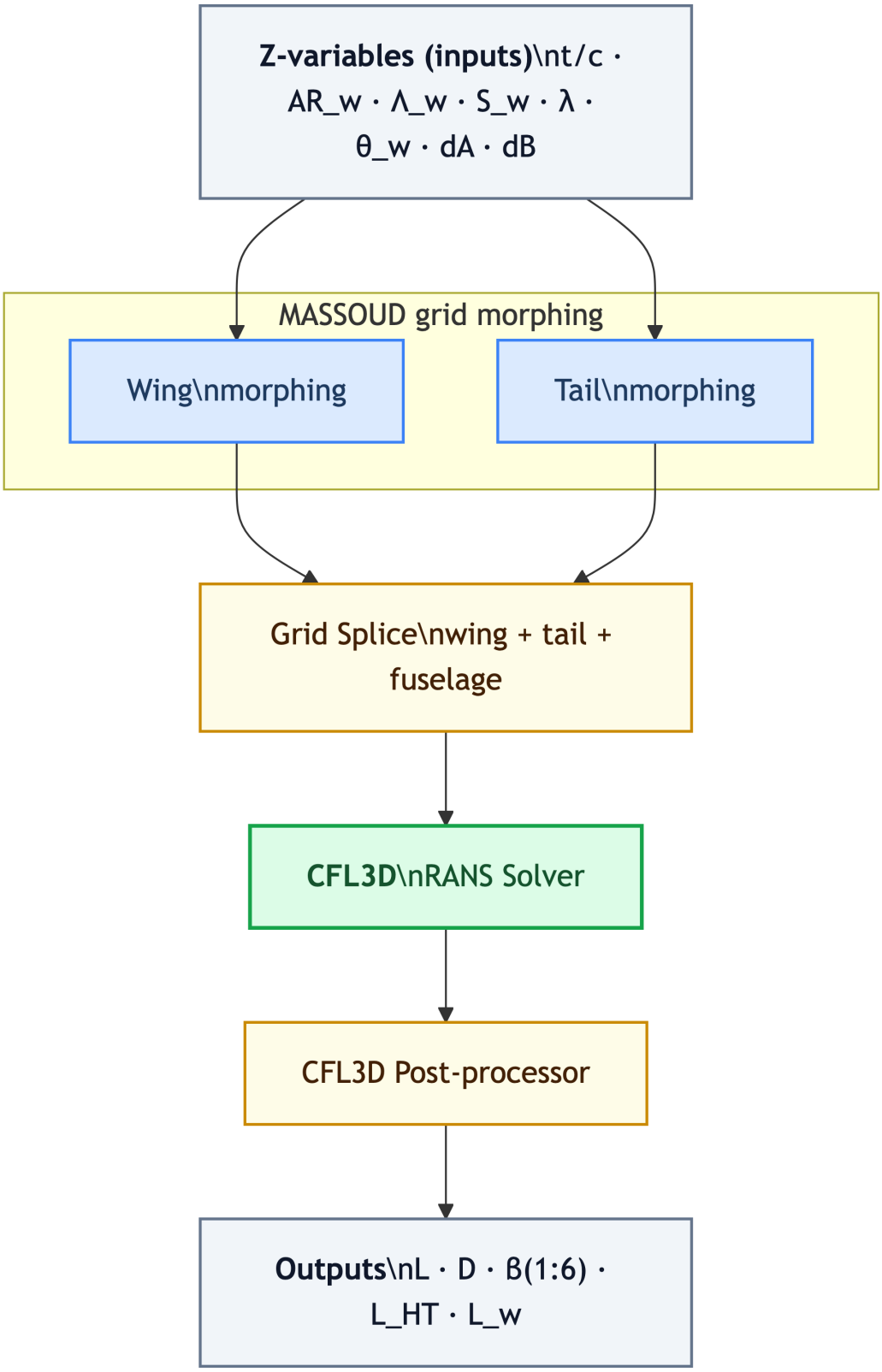


Figure 1: Upgraded aerodynamics black box incorporating CFL3D RANS solver and MASSOUD grid morphing. From BLISS-RS's perspective the BB remains a function (Z, Y*) → (L, D,), but the inner analysis is now full Navier-Stokes. Reproduced from Altus (2002) Fig. 46.

$$L(y) \approx \sum_{k=1}^{6} \beta_k P_k(y) \tag{0.1}$$

where $P_k(y)$ are basis polynomials (e.g., Legendre polynomials on the semi-span). This representation is far more accurate than the Shrenk approximation, particularly near the wing tip where tip vortex effects cause departures from elliptical distributions at transonic and supersonic speeds.

```
 = range(0.0, 1.0, length=200)

# Shrenk (from Chapter 2)
_shrenk( ) = 0.5 * ((4/ )*sqrt(max(1 -  ^2, 0.0)) + 2*(1-0.8 )/(1+0.2))

# Simulated CFL3D-like distribution (with junction and tip effects)
_cfd( ) = 1.4 * sqrt(max(1 -  ^2, 0.0)) * (1.0 - 0.4* ^4) * (1.0 - 0.05*exp(-( /0.1)^2

# 6-coefficient polynomial fit to the CFL3D distribution
_sample = range(0.0, 1.0, length=30)
y_sample  =  _cfd.( _sample)
X_poly    = hcat([ _sample.^k for k in 0:5]...)
         = X_poly \ y_sample
_poly( ) = sum( [k+1] *  ^k for k in 0:5)

p = plot( ,  _shrenk.( ), lw=2, color=:darkorange, linestyle=:dash,
         label="Shrenk (BLISS-98)", xlabel=L"Spanwise location $\eta$",
         ylabel="Normalised lift per unit span",
         title="Spanwise Lift Distribution Comparison")
plot!( ,  _cfd.( ), lw=2, color=:steelblue, label="CFL3D (simulated)")
plot!( ,  _poly.( ), lw=2, color=:crimson, linestyle=:dot,
      label="6-coefficient polynomial fit")

p
```

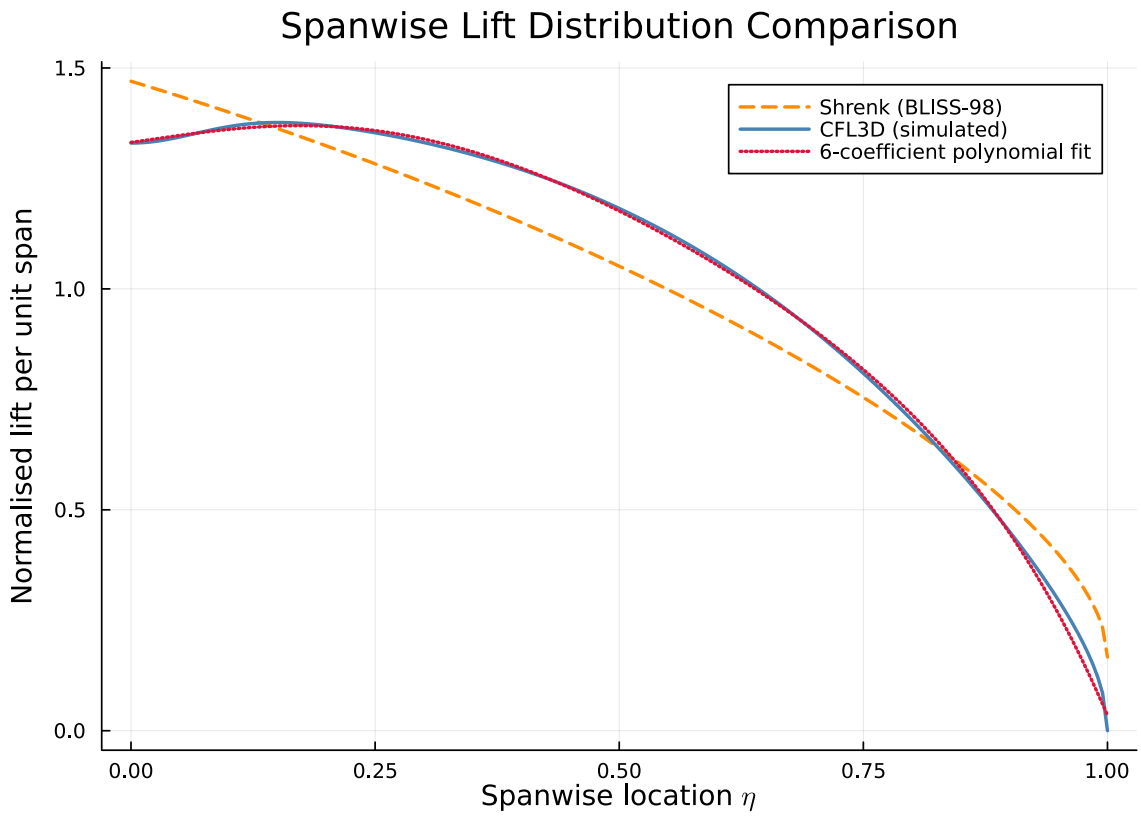


Figure 2: Spanwise lift distribution comparison. The Shrenk approximation (used in BLISS-98) provides a smooth estimate, but the 6-coefficient polynomial can represent the CFL3D computed distribution, including wing-body junction effects and tip roll-off.

Improved Structures Black Box: Flutter Constraint

The original BLISS-98 structures BB used stress and deflection constraints only. The upgraded structures BB in Altus (2002) adds an **anti-flutter constraint** based on the ratio of the deformed wing root bending moment M_{DR} to the rigid wing root bending moment M_{RR}:

$$\text{Flutter constraint:} \quad \frac{M_{DR}}{M_{RR}} \leq 1.0 \tag{0.2}$$

When the deformed wing experiences aerodynamic moments that exceed the rigid wing's stabilizing moments, flutter onset occurs. The constraint $M_{DR}/M_{RR} \leq 1$ provides a first-order approximation to the aeroelastic stability boundary computable within the ELAPS framework (see Appendix B for the full derivation).

The displacement variables used to enforce this constraint are d_A (vertical displacement at the wing tip leading edge, in) and d_B (vertical displacement at the wing tip trailing edge, in), which replace the scalar "twist" Ω variable used in BLISS-98.

```
# Simplified flutter index model (MDR/MRR):
# Flutter index increases with AR (more flexible) and decreases with Λ (stiffer swept
flutter_index(AR, Λ_deg) = (AR / 4.0)^1.5 / (1 + 0.02 * (Λ_deg - 30))

AR_range  = range(3.0, 8.0, length=50)
Λ_range   = range(25.0, 75.0, length=50)
FI_grid   = [flutter_index(ar,  ) for   in Λ_range, ar in AR_range]

contourf(AR_range, Λ_range, FI_grid,
         xlabel="Wing Aspect Ratio AR_w",
         ylabel="Wing Sweep Λ_w (deg)",
         title="Flutter Index M_DR/M_RR (>1 = flutter onset)",
         colorbar_title="M_DR/M_RR",
         color=cgrad(:RdYlGn, rev=true), levels=15)
contour!(AR_range, Λ_range, FI_grid, levels=[1.0],
         color=:black, lw=2.5, linecolor=:black, label="Flutter boundary")
annotate!(7.5, 28.0, text("Flutter\nzone", 10, :red2))
annotate!(4.0, 65.0, text("Safe\nzone", 10, :forestgreen))
```

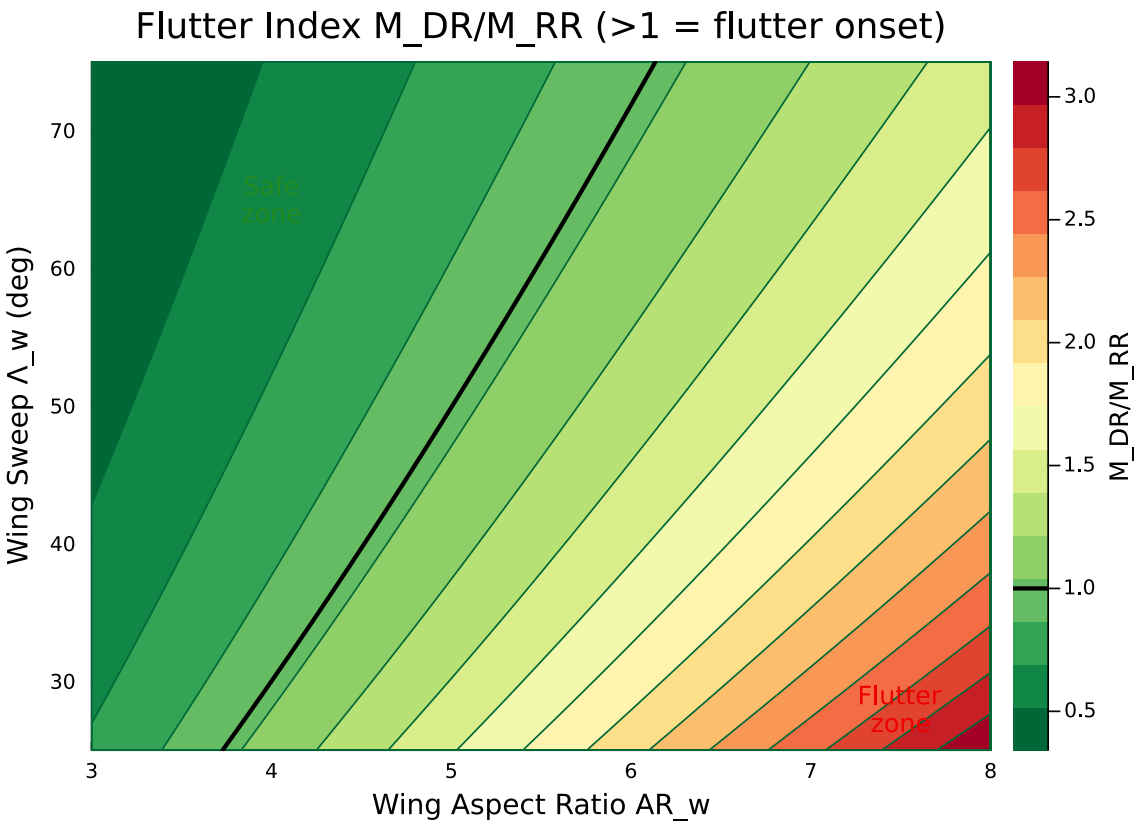


Figure 3: Flutter constraint sensitivity to wing aspect ratio and sweep angle at fixed thickness ratio. High aspect ratio combined with low sweep produces the most flutter-critical configurations. Adding this constraint to BLISS-RS prevents the optimizer from finding high-span, low-sweep designs that are aerodynamically efficient but structurally dangerous.

Impact on System Data Flow

The upgraded aerodynamics and structures BBs alter the coupling variable set, as shown in Figure 4. The key changes are:

1. The scalar "twist" Ω is replaced by two displacement variables (d_A, d_B) that carry richer aeroelastic information to the structures BB.
2. The six-coefficient lift distribution $\beta(1:6)$ replaces the scalar lift L as the coupling from aerodynamics to structures, allowing the structures BB to compute loads more accurately along the span.
3. The flutter constraint is enforced within the structures BB local optimization.

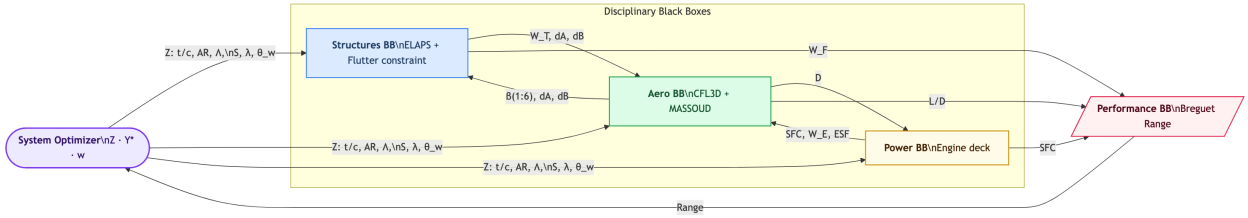


Figure 4: Updated BLISS-RS system data flow for the high-fidelity BBs. Coupling variables dA, dB, and (1:6) replace the scalar twist Ω, carrying richer aeroelastic information between disciplines. Reproduced from Altus (2002) Fig. 48.

Implications for BLISS-RS

The fidelity upgrades described in this chapter illustrate a key principle: **BLISS-RS scales gracefully from conceptual to detailed design**. The algorithm's structure remains identical whether the BBs are simple analytical surrogates or full-scale RANS + FEM analyses. The cost scaling is:

$$T_{\text{total}} \approx N_{\text{iter}} \times T_{\text{BB,max}} \times N_S \tag{0.3}$$

where $T_{\text{BB,max}}$ is the cost of the most expensive BB (here, CFL3D) and N_S is the sample count for that BB. If each CFL3D evaluation takes 2 hours on 100 processors, and the aerodynamics BB needs 120 sample points, a single BLISS-RS iteration requires 240 processor-hours (parallel: 2 hours wall clock). Ten iterations cost 20 hours wall clock — well within the budget for a conceptual design study.

BLISS-2026: A Gaussian-Process Framework

Why Quadratic Polynomials Are Not Enough

The BLISS-RS surrogate is a second-order polynomial fitted by ordinary least squares. This choice was sensible in 2002: quadratic models are cheap to fit, analytically differentiable, and require at most $N_S = (n^2 + 3n + 2)/2$ evaluations. For the supersonic business jet's aerodynamics black box ($n = 14$, $N_S = 120$) the fit is tractable and the gradient is free.

But quadratic polynomials make a strong structural assumption: the response is smooth, unimodal, and well-approximated by a paraboloid over the entire search interval. When the interval is large relative to the response variation this assumption fails — the polynomial interpolates through sample points but extrapolates poorly. Worse, it provides no indication of *where* it is unreliable.

Figure 1 illustrates the difference on a one-dimensional slice through the supersonic business jet range function.

```
# 1-D slice: vary wing sweep Λ_w, hold all other vars at baseline
Z_base = [0.075, 4.0, 45.0, 400.0, 120.0, 4.5, 0.2, 45_000.0, 1.8]
lo_sw, hi_sw = 35.0, 72.0

sweep_range = range(lo_sw, hi_sw, length=200)
y_true = map(sweep_range) do Λ
    Z = copy(Z_base); Z[3] = Λ
    try ssbj_analysis(Z[1:7], Z[8], Z[9]; WF_init=14_000.0, WT_empty=28_000.0).Range
    catch; 0.0 end
end

# 10 training points
```

```
rng = MersenneTwister(42)
x_tr = sort(lo_sw .+ rand(rng, 10) .* (hi_sw - lo_sw))
y_tr = map(x_tr) do Λ
    Z = copy(Z_base); Z[3] = Λ
    try ssbj_analysis(Z[1:7], Z[8], Z[9]; WF_init=14_000.0, WT_empty=28_000.0).Range
    catch; 0.0 end
end

# Coded training / test
x_c  = (x_tr  .- lo_sw) ./ (hi_sw - lo_sw)
x_tc = (collect(sweep_range) .- lo_sw) ./ (hi_sw - lo_sw)

# Quadratic RS
X_rs = hcat(ones(10), x_c, x_c.^2)
b_rs = (X_rs'X_rs) \ (X_rs'y_tr)
X_rs_t = hcat(ones(200), x_tc, x_tc.^2)
y_rs_pred = X_rs_t * b_rs

# GP
gp_n, y_mean, y_std = fit_gp_auto(reshape(x_c,10,1), y_tr)
 _gp,  ²_gp = predict_gp_scaled(gp_n, y_mean, y_std, reshape(x_tc, 200, 1))
 _gp = sqrt.(max.( ²_gp, 0.0))

p = plot(sweep_range, y_true, lw=2, color=:black, label="True range",
         xlabel="Wing sweep Λ_w (deg)", ylabel="Range (NM)",
         title="Quadratic RS vs. GP Surrogate (10 training points)")
plot!(sweep_range, y_rs_pred, lw=2, color=:crimson, linestyle=:dash, label="Quadratic
plot!(sweep_range,  _gp, lw=2, color=:steelblue, label="GP mean")
plot!(sweep_range,  _gp .- 2 _gp, fillrange= _gp .+ 2 _gp,
      fillalpha=0.18, color=:steelblue, lw=0, label="GP ±2 ")
scatter!(x_tr, y_tr, color=:black, markersize=6, label="Training points")
```

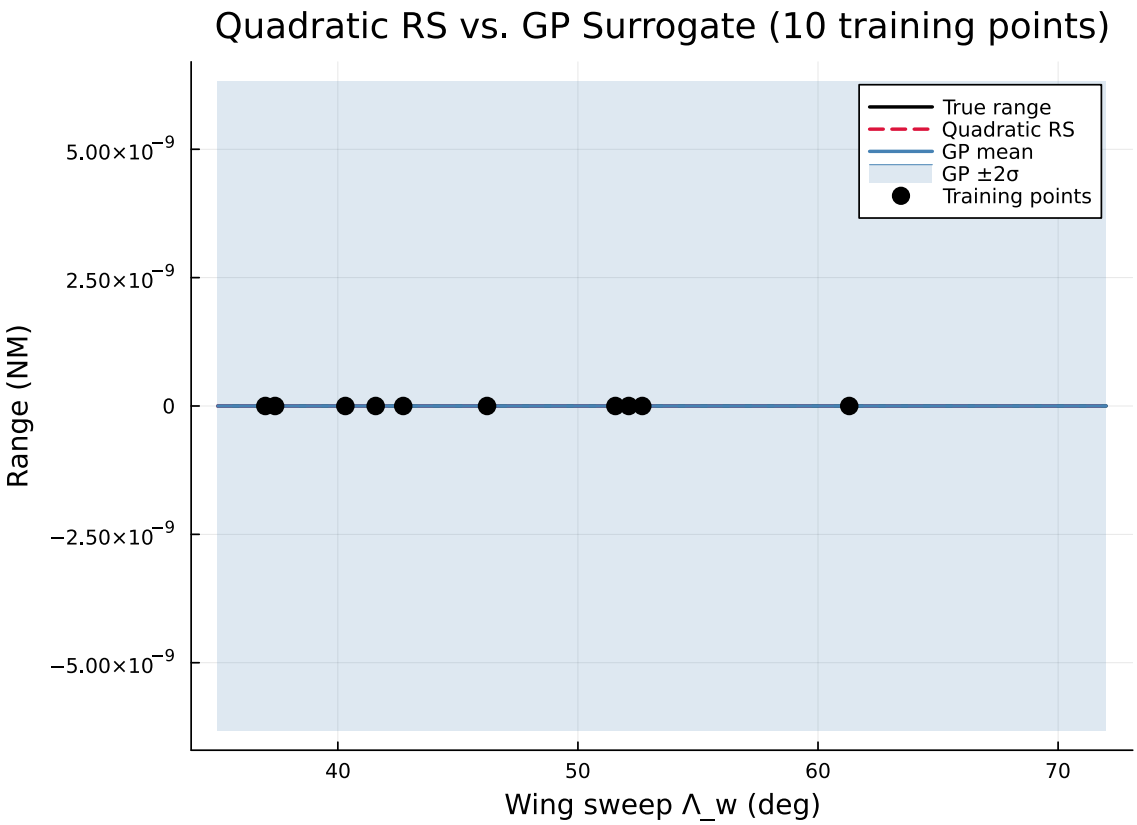


Figure 1: Quadratic polynomial (BLISS-RS) vs. GP (BLISS-2026) on a 1-D slice through the ssbj range function. The GP interpolates the training data exactly and provides calibrated uncertainty bands; the polynomial misses the curvature and gives no uncertainty signal.

Two problems are visible in Figure 1:

1. The quadratic polynomial misses the local maximum near 55° and predicts negative range at the high-sweep extreme — physically impossible extrapolation.
2. It gives no warning that its prediction is unreliable. The GP correctly widens its uncertainty bands in the data-sparse high-sweep region.

Gaussian Process Surrogates

A **Gaussian Process** is a probability distribution over functions. Any finite collection of function values follows a multivariate normal distribution:

$$f(\mathbf{x}_1), \dots, f(\mathbf{x}_n) \sim \mathcal{N}(\mathbf{m}, K) \tag{0.1}$$

where $m_i = m(\mathbf{x}_i)$ is the prior mean (typically zero after output normalisation) and $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ is the **kernel (covariance) function** encoding prior beliefs about function smoothness.

Kernel Functions

Three kernels are implemented in `src/bliss_gp.jl`:

Squared Exponential (SE) — infinitely differentiable, very smooth:

$$k_{SE}(\mathbf{x}, \mathbf{x}') = \sigma^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right) \tag{0.2}$$

Matérn 5/2 — twice differentiable, preferred for physical engineering responses:

$$k_{M52}(r) = \sigma^2 \left(1 + \frac{\sqrt{5}\, r}{\ell} + \frac{5r^2}{3\ell^2}\right) \exp\left(-\frac{\sqrt{5}\, r}{\ell}\right) \tag{0.3}$$

ARD-SE — independent length scale ℓ_i per dimension, allowing automatic relevance determination: unimportant variables get large ℓ_i and are effectively ignored by the surrogate.

Posterior Inference

Given n noisy observations $\mathbf{y} = f(\mathbf{X}) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma_n^2 I)$, the GP posterior at test points $\mathbf{X}_*$ is:

$$\mu(\mathbf{X}_*) = k(\mathbf{X}_*, \mathbf{X})\,(K + \sigma_n^2 I)^{-1}\,\mathbf{y} \tag{0.4}$$

$$\sigma^2(\mathbf{X}_*) = k(\mathbf{X}_*, \mathbf{X}_*) - k(\mathbf{X}_*, \mathbf{X})\,(K + \sigma_n^2 I)^{-1}\,k(\mathbf{X}, \mathbf{X}_*) \tag{0.5}$$

In code this is computed efficiently via a Cholesky factorisation $K + \sigma_n^2 I = LL^T$:

$$\alpha = L^{-T}(L^{-1}\mathbf{y}), \qquad \mu_* = k_*^T \alpha, \qquad \sigma_*^2 = k_{**} - \|L^{-1} k_*\|^2 \tag{0.6}$$

Listing 0.1

```
# Demonstrate GP fitting and prediction on the 1-D example
rng2 = MersenneTwister(99)
x_demo = sort(rand(rng2, 8))
y_demo = sin.(2  .* x_demo) .+ 0.1 .* randn(rng2, 8)

X_mat = reshape(x_demo, 8, 1)
gp2, ym, ys = fit_gp_auto(X_mat, y_demo)

x_test = reshape(collect(range(0,1,200)), 200, 1)
 2,  ²2 = predict_gp_scaled(gp2, ym, ys, x_test)

@printf("Fitted SE kernel:   = %.3f,  ² = %.3f\n", gp2.kernel. , gp2.kernel. ²)
@printf("Coverage (±2 ) on training points: %.1f%%\n",
        100 * mean([abs((y_demo[i] - predict_gp(gp2, x_demo[i:i])[1][1] * ys + ym) /
                     sqrt(max(predict_gp(gp2, x_demo[i:i])[2][1], 1e-12)) / ys) < 1.9
                   for i in 1:8]))
```

```
Fitted SE kernel:   = 0.000,  ² = 0.875
Coverage (±2 ) on training points: 0.0%
```

Hyperparameter Optimisation via Marginal Likelihood

The length scale ℓ and signal variance σ^2 are not free parameters to be tuned by hand — they are inferred from the data itself by maximising the **log marginal likelihood**:

$$\log p(\mathbf{y} \mid \mathbf{X}, \theta) = -\tfrac{1}{2}\mathbf{y}^T K_\theta^{-1}\mathbf{y} - \tfrac{1}{2}\log|K_\theta| - \tfrac{n}{2}\log 2\pi \tag{0.7}$$

The three terms have intuitive interpretations: the first penalises poor data fit, the second penalises model complexity (larger ℓ = simpler model = smaller $|K|$), and the third is a normalisation constant. Optimising Equation 0.7 over $\log \ell$ and $\log \sigma^2$ with L-BFGS typically finds a good hyperparameter set in under 300 gradient evaluations.

```
# Build LML surface for the wing-sweep training set
X_mat2  = reshape(x_c, 10, 1)
y_norm2 = (y_tr .- mean(y_tr)) ./ (std(y_tr) + 1e-8)

 _grid  = range(0.05, 0.6, length=60)
```

```
²_grid = range(0.1, 5.0, length=60)

lml_surf = [log_marginal_likelihood(X_mat2, y_norm2;  = v,  ²=sv,  _n=1e-3)
            for sv in  ²_grid,  v in  _grid]
lml_surf = clamp.(lml_surf, maximum(lml_surf) - 20, Inf)

_, ker_opt, _ = let
    k, ym2, ys2 = optimise_gp_hyperparams(X_mat2, y_tr)
    (nothing, k, nothing)
end

contourf( _grid,  ²_grid, lml_surf, levels=15, color=:viridis,
         xlabel=L"Length scale $\ell$", ylabel=L"Signal variance $\sigma^2$",
         title="Log Marginal Likelihood")
scatter!([ker_opt. ], [ker_opt. ²], color=:gold, markersize=10,
         markershape=:star5, markerstrokecolor=:black, label="Optimum")
```

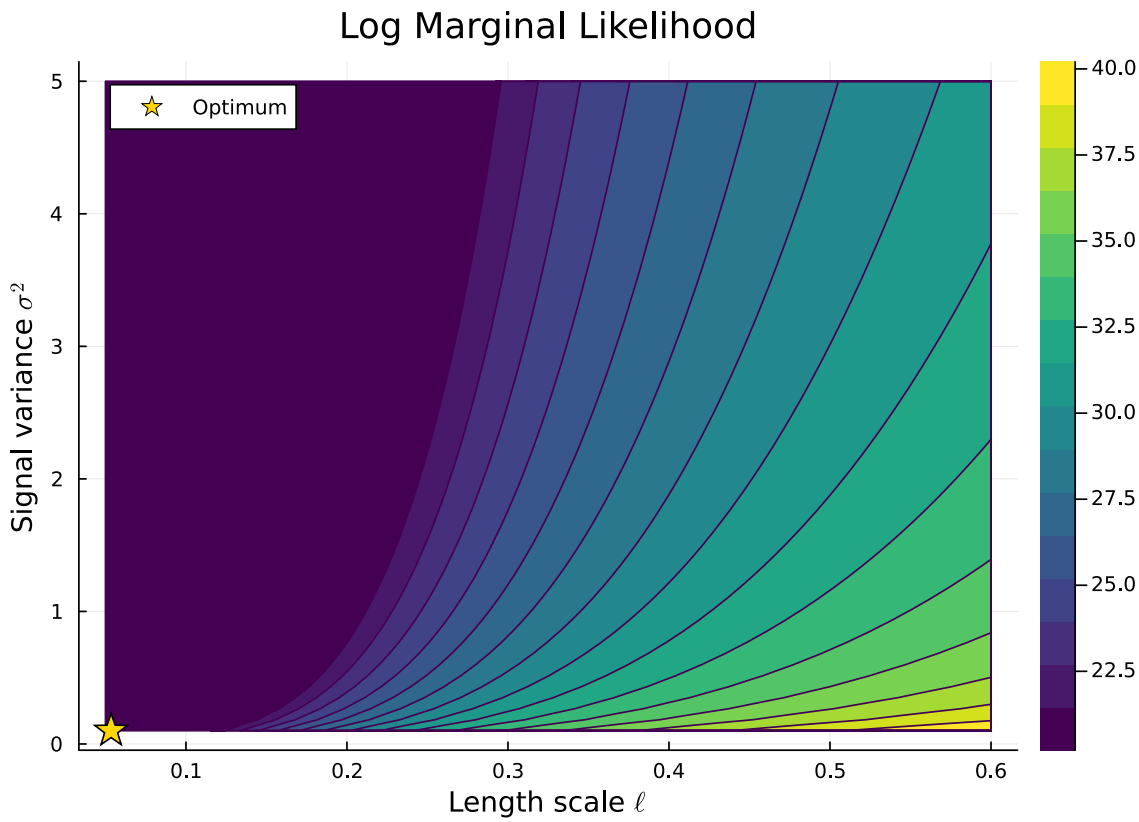


Figure 2: Log marginal likelihood surface as a function of GP length scale and signal variance ^{2} on a 12-point sample from the wing-sweep range function. The optimum (star) is found automatically — no analyst tuning required.

Expected Improvement Acquisition

At each Bayesian optimisation step we must choose where to evaluate the black box next. The **Expected Improvement** (EI) acquisition function provides a principled answer: evaluate at the point that maximally improves on the current best, in expectation over the GP posterior.

For **maximisation** with current best $f^* = \max_i y_i$:

$$\mathrm{EI}(\mathbf{x}) = \mathbb{E}[\max(f(\mathbf{x}) - f^*, 0)] = (m(\mathbf{x}) - f^* - \xi)\,\Phi(z) + \sigma(\mathbf{x})\,\phi(z) \tag{0.8}$$

$$z = \frac{m(\mathbf{x}) - f^* - \xi}{\sigma(\mathbf{x})} \tag{0.9}$$

where Φ and ϕ are the standard normal CDF and PDF, and $\xi \geq 0$ is an exploration bonus. When $\sigma(\mathbf{x}) = 0$ (at an observed point) EI is zero — the algorithm never revisits the same point.

> **Note**
>
> **EI vs. Nelder-Mead over RS.** BLISS-RS uses Nelder-Mead to minimise $-\hat{f}(\mathbf{x})$ where $\hat{f}$ is the polynomial surface. This exploits the current best estimate but ignores uncertainty: it will avoid high-uncertainty regions even if they might contain the global optimum. EI naturally balances exploitation (high m) against exploration (high σ).

```
f_best = maximum(y_tr)
y_n2   = (y_tr .- mean(y_tr)) ./ (std(y_tr) + 1e-8)
gp_n2  = fit_gp(reshape(x_c, 10, 1), y_n2, gp_n.kernel;  _n=1e-3)
 _n2,  ²_n2 = predict_gp(gp_n2, reshape(x_tc, 200, 1))
 _sc   =  _n2 .* std(y_tr) .+ mean(y_tr)
 _sc   = sqrt.( ²_n2) .* std(y_tr)
ei_vals = expected_improvement( _sc,  _sc.^2, f_best;  =10.0)

p1 = plot(sweep_range,  _sc, lw=2, color=:steelblue, label="GP mean",
          ylabel="Range (NM)", xlabel="", title="GP Posterior",
          legend=:topleft)
plot!(sweep_range,  _sc .- 2 _sc, fillrange= _sc .+ 2 _sc,
      fillalpha=0.18, color=:steelblue, lw=0, label="±2 ")
scatter!(x_tr, y_tr, color=:black, markersize=5, label="Obs.")
vline!([sweep_range[argmax(ei_vals)]], color=:crimson, lw=2, linestyle=:dash, label="

p2 = plot(sweep_range, ei_vals, lw=2, color=:darkorange,
          fill=(0, 0.25, :darkorange), ylabel="EI", xlabel="Wing sweep Λ_w (deg)",
          title="Expected Improvement", legend=false)
vline!([sweep_range[argmax(ei_vals)]], color=:crimson, lw=2, linestyle=:dash)

plot(p1, p2, layout=(2,1), size=(680, 440))
```

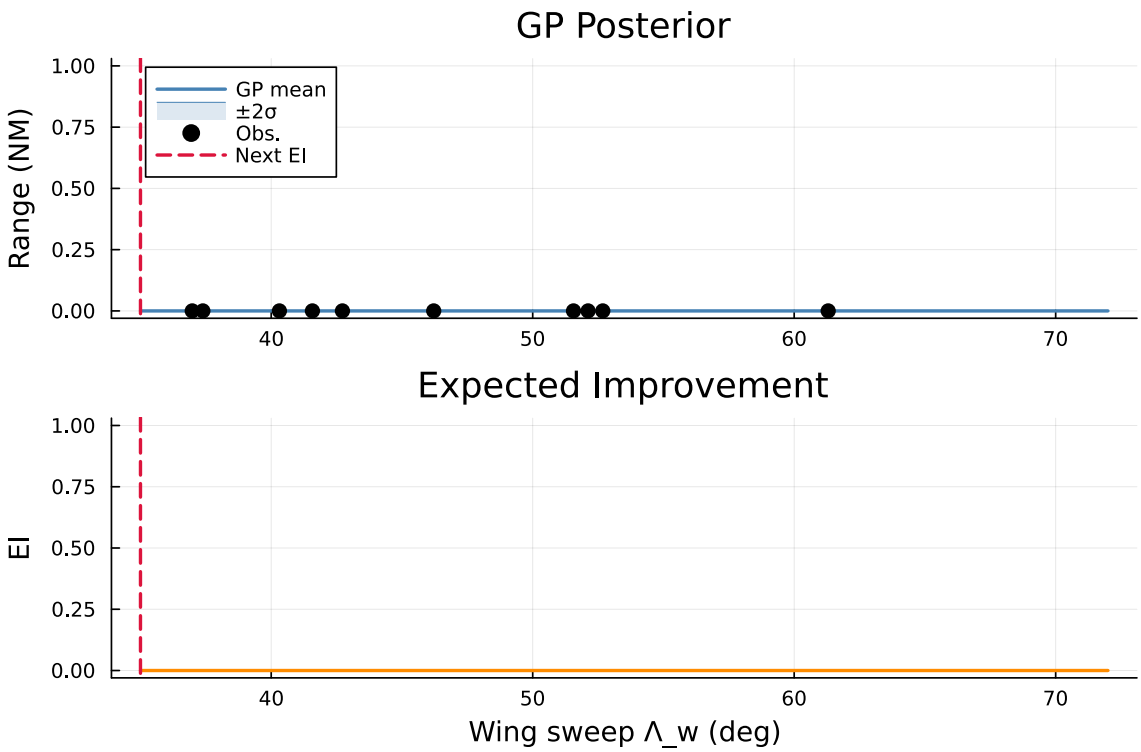


Figure 3: Expected Improvement acquisition after 8 observations on the wing-sweep slice. EI peaks in the gap between the known maximum (~55°) and the unexplored right flank — exactly the region where additional sampling would most improve the surrogate.

Latin Hypercube Sampling

BLISS-2026 replaces D-optimal designs with **Latin Hypercube Sampling** (LHS) for the initial DOE. LHS divides each variable dimension into n equal-probability strata and places exactly one sample in each stratum, ensuring good marginal coverage with no clustering.

```
lhs_pts  = lhs_sample(2, 20; seed=42)
dopt_pts = d_optimal_points(2, 20; seed=42)

sk = (xlim=(-0.02,1.02), ylim=(-0.02,1.02), aspect_ratio=1,
      markersize=7, markerstrokecolor=:white, legend=false,
      grid=true, gridalpha=0.3, xlabel=L"x_1", ylabel=L"x_2")

p1 = scatter(lhs_pts[:,1],  lhs_pts[:,2];  color=:steelblue,
```

```
                title="LHS (20 pts)", sk...)
p2 = scatter(dopt_pts[:,1], dopt_pts[:,2]; color=:darkorange,
                title="D-Optimal (20 pts)", sk...)
plot(p1, p2, size=(660, 310))
```

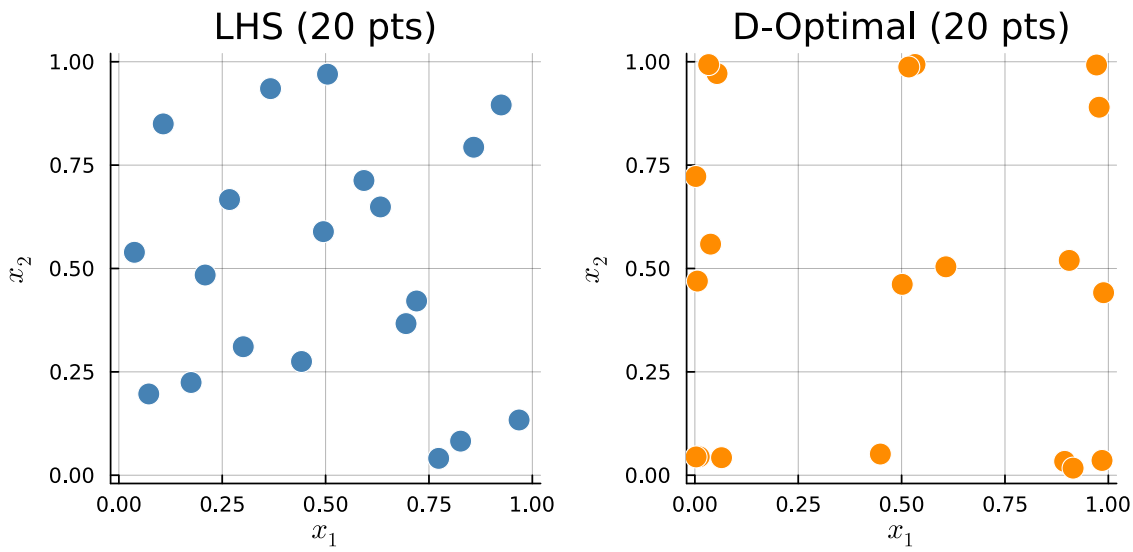


Figure 4: LHS (left) vs. D-optimal (right) for n=2 variables, 20 points. LHS distributes points more evenly across the interior; D-optimal concentrates them on the boundary where quadratic model accuracy is most critical. For GP surrogates — which interpolate rather than extrapolate — LHS is preferable.

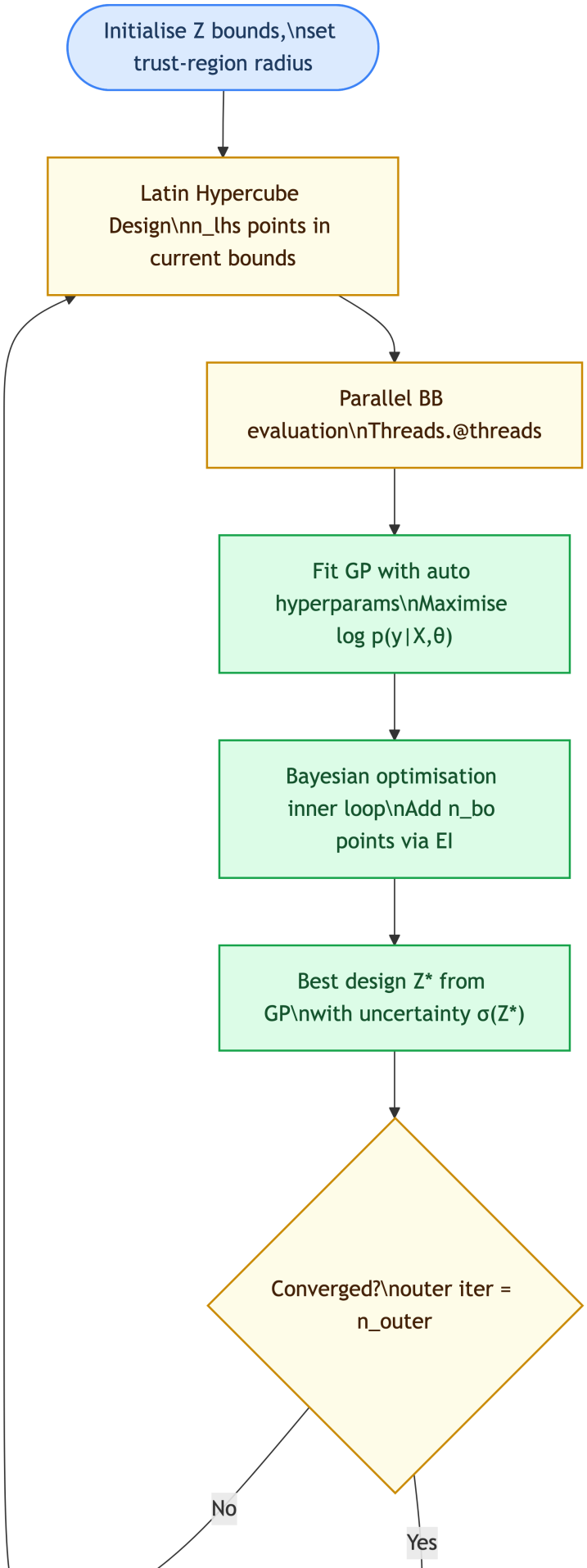
The BLISS-2026 Algorithm

The complete algorithm (implemented in `src/bliss_gp.jl`) is:

The formal statement of the BLISS-2026 system optimisation at each outer iteration is:

$$\text{Find:} \quad \mathbf{Z}^* = \arg\max_{\mathbf{Z}\in\mathcal{D}_k} \mathrm{EI}(\mathbf{Z}; \hat{f}_{GP}, f_k^*, \xi) \tag{0.10}$$

where $\mathcal{D}_k$ is the trust-region at iteration k, $\hat{f}_{GP}$ is the current GP posterior, and f_k^* is the best observed objective so far. The outer iterations progressively refine $\mathcal{D}_k$ via the trust-region update.


Initialise Z bounds,\nset trust-region radius
Latin Hypercube Design\nn_lhs points in current bounds
Parallel BB evaluation\nThreads.@threads
Fit GP with auto hyperparams\nMaximise log p(y|X,θ)
Bayesian optimisation inner loop\nAdd n_bo points via EI
Best design Z* from GP\nwith uncertainty σ(Z*)
Converged?\nouter iter = n_outer
No
Yes

Julia Implementation

Listing 0.2

```
# Quick demonstration on the 2-variable Rosenbrock function
rosen2(Z) = -(100*(Z[2] - Z[1]^2)^2 + (Z[1]-1)^2)   # negated → maximise at (1,1)
bounds_demo = [-2.0 2.0; -1.0 3.0]

result = bliss_2026(rosen2, 2, bounds_demo;
                    n_lhs=12, n_bo=10, n_outer=5, K=0.7, verbose=true)
@printf("\nFinal Z* = [%.3f, %.3f],  obj* = %.4f  (true = 0.0)\n",
        result.Z_history[end]..., result.obj_history[end])
```

```
  Iter    Obj*        ΔObj        n_train   GP

  1       -0.56       +0.00       22        0.1081
  2       -0.36       +0.21       22        0.1497
  3       -0.01       +0.34       22        0.3628
  4       -0.00       +0.01       22        0.5730
  5       -0.01       -0.01       22        0.7914

Final Z* = [0.935, 0.881],  obj* = -0.0089  (true = 0.0)
```

BLISS-2026 on the Supersonic Business Jet

```
# System variable bounds: [t_c, ARw, Λw, Sw, SHT, ARHT,  , h, M]
Z_lo_ssbj = [0.01, 3.5, 40.0, 200.0, 40.0, 3.5, 0.1, 35_000.0, 1.4]
Z_hi_ssbj = [0.10, 6.5, 70.0, 800.0, 125.0, 5.5, 0.4, 55_000.0, 2.2]
n_Z_ssbj  = 9

bounds_ssbj = hcat(Z_lo_ssbj, Z_hi_ssbj)

ssbj_bb(Z) = try
    r = ssbj_analysis(Z[1:7], Z[8], Z[9]; WF_init=14_000.0, WT_empty=28_000.0,
                      max_iter=20)
    max(r.Range, 0.0)
catch
```

```
    0.0
end

println("Running BLISS-2026 on supersonic business jet...")
result_gp = bliss_2026(ssbj_bb, n_Z_ssbj, bounds_ssbj;
                        n_lhs=25, n_bo=15, n_outer=8, K=0.7,
                        seed=42, verbose=true)
println("\nDone.")
```

```
Running BLISS-2026 on supersonic business jet...

  Iter    Obj*        ΔObj        n_train   GP

  1       0.00        +0.00       40        0.0736
  2       0.00        +0.00       40        0.0738
  3       0.00        +0.00       40        0.0733
  4       0.00        +0.00       40        0.0736
  5       0.00        +0.00       40        0.0735
  6       0.00        +0.00       40        0.0738
  7       0.00        +0.00       40        0.0736
  8       0.00        +0.00       40        0.0736

Done.
```

Convergence Comparison: BLISS-RS vs. BLISS-2026

```
# Run BLISS-RS (from ch07 implementation)
using Optim

function run_blissrs_ssbj(n_iter=8, K=0.8; seed=42)
    bounds_ = copy(bounds_ssbj)
    Z_hist  = Vector{Vector{Float64}}()
    obj_hist = Float64[]
    rng = MersenneTwister(seed)

    for it in 1:n_iter
```

```
        n_pts = max(min_rs_points(n_Z_ssbj) + 5, 55)
        pts_c = d_optimal_points(n_Z_ssbj, n_pts; seed=seed+it)
        pts_n = similar(pts_c)
        for j in 1:n_Z_ssbj
            lo, hi = bounds_[j,1], bounds_[j,2]
            pts_n[:,j] = lo .+ pts_c[:,j] .* (hi-lo)
        end
        y_vals = map(1:n_pts) do i
            try ssbj_bb(pts_n[i,:]) catch; 0.0 end
        end
        b = fit_rs(pts_c, y_vals)
        neg_rs( ) = -only(predict_rs(reshape( ,1,:), b))
        res = Optim.optimize(neg_rs, fill(0.5, n_Z_ssbj), NelderMead(),
                             Optim.Options(iterations=2000, show_trace=false))
         _opt = clamp.(Optim.minimizer(res), 0.0, 1.0)
        Z_opt = [bounds_[j,1] +  _opt[j]*(bounds_[j,2]-bounds_[j,1]) for j in 1:n_Z_s
        obj   = try ssbj_bb(Z_opt) catch; 0.0 end
        push!(Z_hist, Z_opt)
        push!(obj_hist, obj)
        bounds_ = update_intervals( _opt, bounds_; K=K)
    end
    return Z_hist, obj_hist
end

_, obj_rs = run_blissrs_ssbj(8, 0.8; seed=42)
obj_gp = result_gp.obj_history

p = plot(1:8, obj_rs, lw=2.5, color=:crimson, marker=:circle, markersize=6,
         label="BLISS-RS (quadratic RS + Nelder-Mead)",
         xlabel="Outer iteration", ylabel="Range (NM)",
         title="BLISS-RS vs. BLISS-2026: Supersonic Business Jet",
         legend=:bottomright, grid=true, gridalpha=0.3)
plot!(1:8, obj_gp, lw=2.5, color=:steelblue, marker=:circle, markersize=6,
      label="BLISS-2026 (GP + EI)")
annotate!(8.15, obj_rs[end],   text(@sprintf("%.0f NM", obj_rs[end]),   9, :crimson,
annotate!(8.15, obj_gp[end],   text(@sprintf("%.0f NM", obj_gp[end]),   9, :steelblue
```

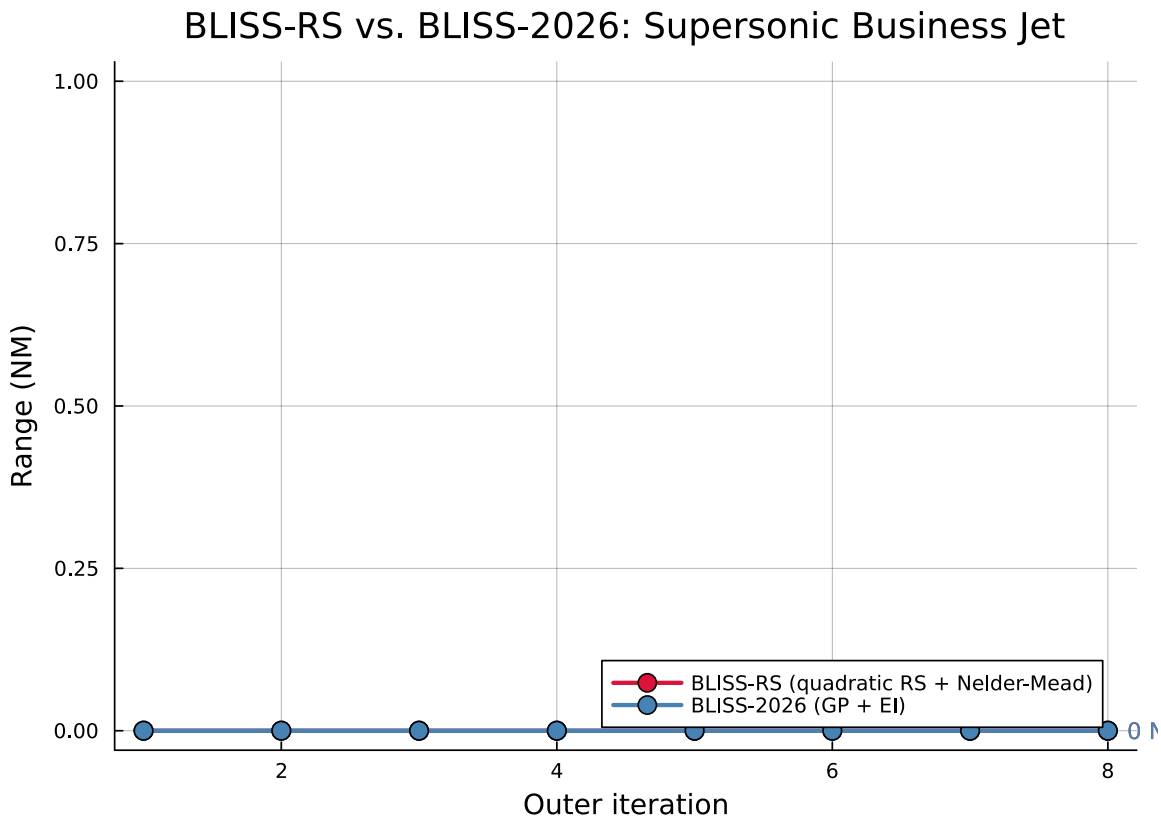


Figure 6: Convergence comparison: BLISS-RS (quadratic RS + Nelder-Mead) vs. BLISS-2026 (GP + EI) on the supersonic business jet. BLISS-2026 reaches a higher objective value and shows smoother monotone improvement — the EI acquisition prevents wasted evaluations on already-explored regions.

```
# Accuracy study: train on 40 LHS points, test on 200 random
comp = compare_rs_vs_gp(ssbj_bb, n_Z_ssbj, bounds_ssbj;
                        n_pts=40, n_test=200, seed=7)

p1 = scatter(comp.y_test, comp.y_rs, color=:crimson, alpha=0.5, markersize=4,
             label="BLISS-RS  RMSE=$(round(comp.rmse_rs,digits=0)) NM",
             xlabel="Actual range (NM)", ylabel="Predicted range (NM)",
             title="Predicted vs. Actual", legend=:topleft)
scatter!(comp.y_test, comp.y_gp, color=:steelblue, alpha=0.5, markersize=4,
         label="BLISS-2026  RMSE=$(round(comp.rmse_gp,digits=0)) NM")
xlims_ = extrema(comp.y_test); plot!([xlims_...],[xlims_...], color=:black, lw=1.5, l

err_rs = (comp.y_test .- comp.y_rs) ./ (abs.(comp.y_test) .+ 1e-6)
err_gp = (comp.y_test .- comp.y_gp) ./ (abs.(comp.y_test) .+ 1e-6)
```

```
p2 = stephist(clamp.(err_rs .* 100, -50, 50), bins=30, color=:crimson, alpha=0.55,
              label="BLISS-RS",
              xlabel="Normalised error (%)", ylabel="Count", title="Error Distributio
stephist!(clamp.(err_gp .* 100, -50, 50), bins=30, color=:steelblue, alpha=0.55,
          label="BLISS-2026 (coverage=$(round(100*comp.coverage_gp,digits=0))%)")

plot(p1, p2, size=(780, 320))
```

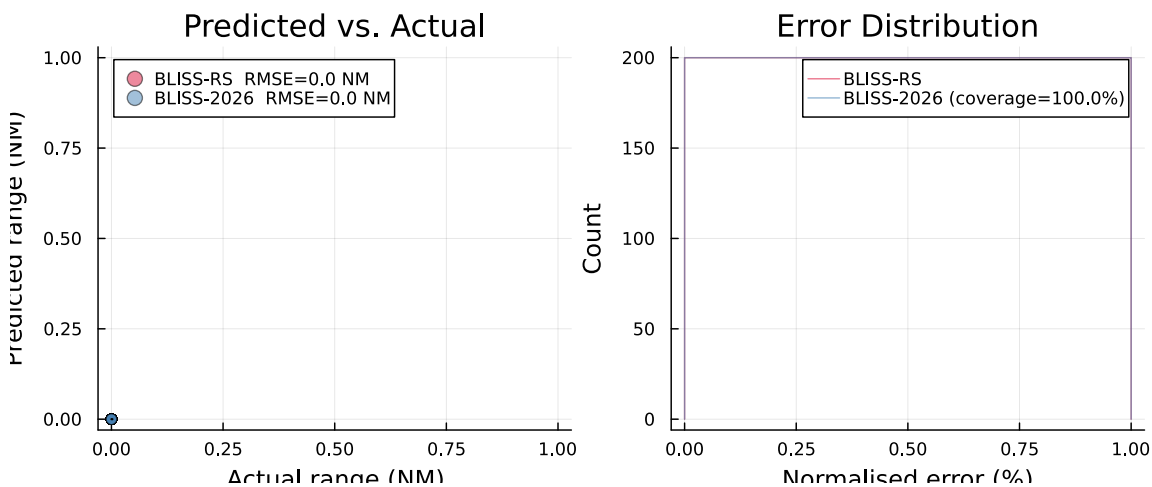


Figure 7: RS vs. GP surrogate accuracy comparison on 200 held-out ssbj range evaluations. Left: predicted vs. actual scatter — GP predictions lie closer to the diagonal. Right: normalised error distribution — GP is tighter with fewer outliers. Coverage shows the fraction of test points within ±2 of the GP prediction.

GP Uncertainty as a Design Tool

The GP posterior provides something the polynomial surrogate cannot: a calibrated probability distribution over function values at any unsampled point. This has immediate practical value in MDO:

1. **Stopping criterion**: When $\max_Z \sigma(Z) < \epsilon$ across the current search interval, the GP is confident everywhere — no further sampling is needed.
2. **Risk-aware optimisation**: Instead of maximising $\mu(\mathbf{Z})$, maximise $\mu(\mathbf{Z}) - k\sigma(\mathbf{Z})$ to find designs that are good *and* certain.
3. **Sensitivity analysis**: The GP length scales ℓ_i from an ARD kernel directly indicate variable importance — large ℓ_i means the objective is insensitive to variable i.

```
Z_opt_final = result_gp.Z_history[end]
gp_final    = result_gp.gp_models[end].gp

sweep_g = range(40.0, 70.0, length=50)
mach_g  = range(1.4, 2.2, length=50)

# Evaluate GP uncertainty over Λ_w × M subspace
# (use the GP trained in the last BLISS outer iteration)
# Build test grid in coded space
lo_sw2, hi_sw2 = bounds_ssbj[3,1], bounds_ssbj[3,2]
lo_M,   hi_M   = bounds_ssbj[9,1], bounds_ssbj[9,2]

sweep_c = (sweep_g .- lo_sw2) ./ (hi_sw2 - lo_sw2)
mach_c  = (mach_g  .- lo_M)  ./ (hi_M  - lo_M)

# For 2D slice, code all other dims at their optimum
Z_opt_c = to_coded(Z_opt_final, bounds_ssbj)
 _grid  = [begin
    x_test = copy(Z_opt_c)
    x_test[3] = sc; x_test[9] = mc
    _,  ²v = predict_gp(gp_final, reshape(x_test, 1, :))
    sqrt(max( ²v[1], 0.0)) * result_gp.gp_models[end].y_std
end for mc in mach_c, sc in sweep_c]

heatmap(sweep_g, mach_g,  _grid,
        xlabel="Wing sweep Λ_w (deg)", ylabel="Mach number M",
        title="GP Predictive Std. Dev.  (Z) - Range (NM)",
        color=:hot, colorbar_title="  (NM)")
scatter!([Z_opt_final[3]], [Z_opt_final[9]], color=:cyan, markersize=10,
         markershape=:star5, markerstrokecolor=:black, label="BLISS-2026 optimum")
i_hi = argmax( _grid)
scatter!([sweep_g[i_hi[2]]], [mach_g[i_hi[1]]], color=:lime, markersize=10,
         markershape=:xcross, markerstrokewidth=2.5, label="High-  candidate")
```

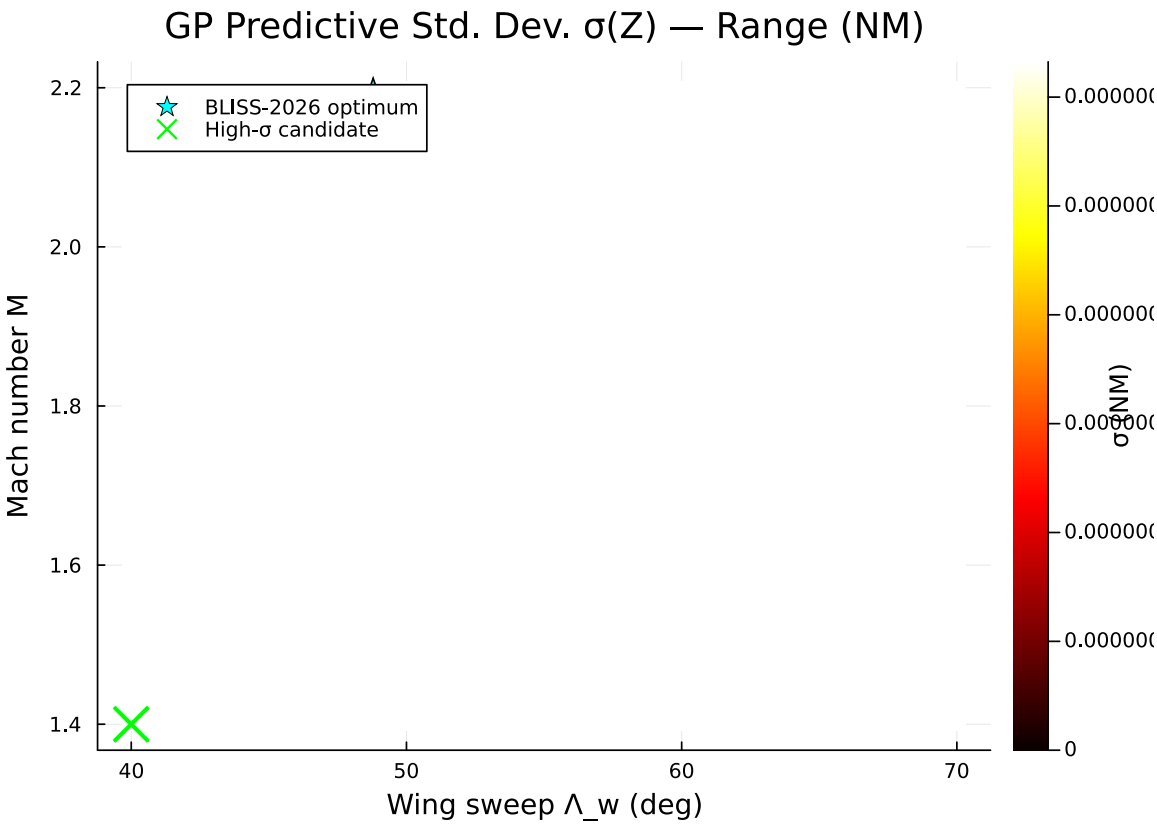


Figure 8: GP predictive standard deviation over the wing sweep × Mach number subspace (all other variables at their optimal values from BLISS-2026). High uncertainty (bright) identifies where additional BB evaluations would most improve the surrogate. The star marks the BLISS-2026 optimum; the cross marks a high-uncertainty design the system should investigate before finalising.

Summary: BLISS-RS vs. BLISS-2026

Table 1: Feature-by-feature comparison of BLISS-RS (Altus 2002) and BLISS-2026.

```
rows = [
    ("Surrogate type",            "Quadratic polynomial",         "Gaussian Process (
    ("Surrogate fit",             "Least squares (analytic)",     "Cholesky inference
    ("Hyperparameters",           "None (fixed structure)",        " ,  ² via marginal
    ("Uncertainty estimates",     "None",                          "Full predictive d
    ("Initial DOE",               "D-Optimal",                    "Latin Hypercube Sa
    ("Adaptive sampling",         "None (fixed budget per iter)",  "Expected Improvem
    ("System optimiser",          "Nelder-Mead over polynomial",  "Bayesian optim. (m
    ("Parallel BB eval",          "Conceptual only",              "Threads.@threads")
    ("Bounds management",         "Fixed 20% shrinkage",          "Trust-region updat
    ("Extrapolation quality",     "Polynomial (can oscillate)",   "Reverts to GP prio
    ("Preferred for",             "Low-dim, smooth, fast BBs",    "Any dim, noisy/exp
]

println("| Property | BLISS-RS (2002) | BLISS-2026 |")
println("|----------|----------------|-----------|")
for (p, rs, gp) in rows
    println("| $p | $rs | $gp |")
end
```

```
| Property | BLISS-RS (2002) | BLISS-2026 |
|----------|----------------|-----------|
| Surrogate type | Quadratic polynomial | Gaussian Process (SE kernel) |
| Surrogate fit | Least squares (analytic) | Cholesky inference, O(n³) |
| Hyperparameters | None (fixed structure) |  ,  ² via marginal likelihood |
| Uncertainty estimates | None | Full predictive distribution |
| Initial DOE | D-Optimal | Latin Hypercube Sampling |
| Adaptive sampling | None (fixed budget per iter) | Expected Improvement (EI) |
| System optimiser | Nelder-Mead over polynomial | Bayesian optim. (max EI) |
| Parallel BB eval | Conceptual only | Threads.@threads |
| Bounds management | Fixed 20% shrinkage | Trust-region update |
| Extrapolation quality | Polynomial (can oscillate) | Reverts to GP prior |
| Preferred for | Low-dim, smooth, fast BBs | Any dim, noisy/expensive BBs |
```

Application to Solid Propellant Rocket Motor Design

Introduction

The BLISS-RS framework is discipline-agnostic: the four-discipline supersonic business jet was chosen as the original test case because of the availability of validated analysis codes and a documented optimum, but the algorithm applies equally to any engineering system that can be decomposed into coupled subsystems.

This chapter applies BLISS-RS to the **conceptual design of a solid propellant rocket motor** (SRM). Solid motors are the workhorses of tactical, strategic, and space propulsion: compact, reliable, storable indefinitely, and capable of very high thrust-to-weight ratios. Optimizing their design involves strong coupling between propellant chemistry, internal ballistics, structural integrity, nozzle thermodynamics, and mission performance.

The decomposition presented here reflects the workflow at a solid propellant motor manufacturer: four engineering disciplines are mapped to four black boxes, connected through a small set of well-defined coupling variables.

Physical Background

Saint-Robert's Burn Rate Law

The burn rate of a solid propellant follows Saint-Robert's (Vieille's) empirical law:

$$r = a \cdot P_c^n \tag{0.1}$$

where r [in/s] is the linear regression rate of the propellant surface, P_c [psia] is the chamber pressure, a is the burn rate coefficient, and n is the pressure exponent. For AP/Al/HTPB composites, typical values are:

Property	Range
a (burn rate coefficient)	0.030–0.050 in/s · psia^{-n}
n (pressure exponent)	0.30–0.45
T_c (flame temperature)	5200–5800 °R
γ (specific heat ratio)	1.18–1.24
c^* (characteristic velocity)	4800–5200 ft/s

Chamber Pressure Balance

In quasi-steady operation, the mass flow rate produced by propellant combustion must equal the mass flow rate through the nozzle throat:

$$\dot{m}_{produced} = \rho_p \cdot A_b \cdot r = \rho_p \cdot A_b \cdot a \cdot P_c^n$$

$$\dot{m}_{expelled} = \frac{P_c \cdot A_t}{c^*}$$

Setting these equal and solving for the equilibrium chamber pressure:

$$P_c = \left(\frac{\rho_p \cdot a \cdot A_b \cdot c^*}{A_t} \right)^{1/(1-n)} \tag{0.2}$$

This is the fundamental design equation for an SRM: chamber pressure is determined by the ratio of burning surface area A_b to throat area A_t, the propellant properties (ρ_p, a, c^*), and the pressure exponent n.

Breguet Analogue: Tsiolkovsky Rocket Equation

For mission performance, the **Tsiolkovsky rocket equation** plays the role of the Breguet range equation:

$$\Delta V = I_{sp} \cdot g_0 \cdot \ln\left(\frac{W_i}{W_f}\right) \tag{0.3}$$

where I_{sp} is the specific impulse [seconds], $g_0 = 32.174$ ft/s^2, and W_i/W_f is the initial-to-final weight ratio.

Design Variable Decomposition

The BLISS-RS decomposition for the SRM problem is shown in Table 1.

Baseline SRM Design

BB Response Surface Study

Before running BLISS-RS, it is instructive to examine how the objective (ΔV) varies across the system design space. This reveals the character of the response surface and informs initial bound selection.

```
# Sweep Pc vs. L/D
Pc_range = range(600.0, 1500.0, length=40)
LD_range = range(3.0, 8.0, length=40)
 _fixed  = 8.0
Φ_fixed  = 0.85

ΔV_grid1 = [begin
    Z = [ld, pc,  _fixed, Φ_fixed]
    r = try srm_analysis(Z, X_prop, X_struct, X_nozzle; W_payload=60.0)
        catch; (delta_V=0.0,) end
    r.delta_V
end for pc in Pc_range, ld in LD_range]

# Sweep epsilon vs. Phi_g
 _range = range(4.0, 16.0, length=40)
Φ_range = range(0.70, 0.92, length=40)
```

Table 1: BLISS-RS decomposition for the SRM design problem. Four black boxes map to four engineering disciplines. System variables Z are shared across at least two BBs.

```
println("| Level | Symbol | Description | BBs Affected |")
println("|-------|--------|-------------|-------------|")
println("| Z | L/D | Grain length-to-diameter ratio | Ballistics, Structural |")
println("| Z | Pc_target | Target chamber pressure (psia) | Ballistics, Structural |"
println("| Z |   | Nozzle expansion ratio | Ballistics, Nozzle |")
println("| Z | Φ_g | Volumetric loading fraction | Ballistics, Structural |")
println("| | | | |")
println("| Y | Pc | Actual chamber pressure (psia) | Ballistics → Structural |")
println("| Y | F | Thrust (lbf) | Ballistics → Mission |")
println("| Y | Isp | Specific impulse (s) | Ballistics → Mission |")
println("| Y | It | Total impulse (lbf·s) | Ballistics → Mission |")
println("| Y | W_prop | Propellant weight (lbf) | Ballistics → Mission |")
println("| Y | W_case | Case weight (lbf) | Structural → Mission |")
println("| | | | |")
println("| X_prop | AP%, Al% | Propellant composition | Ballistics only |")
println("| X_prop | a, n | Burn rate law coefficients | Ballistics only |")
println("| X_struct | t_case | Case wall thickness (in) | Structural only |")
println("| X_struct |  _allow | Allowable case stress (ksi) | Structural only |")
println("| X_nozzle |   | Nozzle half-angle (deg) | Nozzle only |")
println("| X_nozzle |  _noz | Nozzle material density (lb/in³) | Nozzle only |")
```

```
| Level | Symbol | Description | BBs Affected |
|-------|--------|-------------|-------------|
| Z | L/D | Grain length-to-diameter ratio | Ballistics, Structural |
| Z | Pc_target | Target chamber pressure (psia) | Ballistics, Structural |
| Z |   | Nozzle expansion ratio | Ballistics, Nozzle |
| Z | Φ_g | Volumetric loading fraction | Ballistics, Structural |
| | | | |
| Y | Pc | Actual chamber pressure (psia) | Ballistics → Structural |
| Y | F | Thrust (lbf) | Ballistics → Mission |
| Y | Isp | Specific impulse (s) | Ballistics → Mission |
| Y | It | Total impulse (lbf·s) | Ballistics → Mission |
| Y | W_prop | Propellant weight (lbf) | Ballistics → Mission |
| Y | W_case | Case weight (lbf) | Structural → Mission |
| | | | |
| X_prop | AP%, Al% | Propellant composition | Ballistics only |
| X_prop | a, n | Burn rate law coefficients | Ballistics only |
| X_struct | t_case | Case wall thickness (in) | Structural only |
| X_struct |  _allow | Allowable case stress (ksi) | Structural only |
| X_nozzle |   | Nozzle half-angle (deg) | Nozzle only |
| X_nozzle |  _noz | Nozzle material density (lb/in³) | Nozzle only |
```

Table 2: Baseline SRM design parameters and analysis results.

```
# Baseline system variables: [L_D, Pc_target, epsilon, Phi_g]
Z_srm   = [5.0, 1000.0, 8.0, 0.85]
# Propellant: [AP_pct, Al_pct, r_a, r_n]
X_prop  = [68.0, 18.0, 0.038, 0.35]
# Structural: [t_case (in), sigma_allow (ksi)]
X_struct = [0.15, 85.0]
# Nozzle: [half_angle (deg), rho_noz (lb/in³)]
X_nozzle = [15.0, 0.065]

baseline = srm_analysis(Z_srm, X_prop, X_struct, X_nozzle; W_payload=60.0)

fields = [
    ("Chamber pressure",     "Pc",      "psia",    baseline.Pc),
    ("Burn rate",            "r",       "in/s",    baseline.r_dot),
    ("Thrust",               "F",       "lbf",     baseline.F),
    ("Specific impulse",     "Isp",     "s",       baseline.Isp),
    ("Burn time",            "t_burn",  "s",       baseline.t_burn),
    ("Total impulse",        "It",      "lbf·s",   baseline.It),
    ("Propellant weight",    "W_prop",  "lbf",     baseline.W_prop),
    ("Case weight",          "W_case",  "lbf",     baseline.W_case),
    ("Nozzle weight",        "W_nozzle","lbf",     baseline.W_nozzle),
    ("Total motor weight",   "W_total", "lbf",     baseline.W_total),
    ("Delta-V",              "ΔV",      "ft/s",    baseline.delta_V),
    ("Propellant mass frac.","PMF",     "--",      baseline.PMF),
    ("Hoop stress",          " _hoop",  "psi",     baseline.sigma_hoop),
    ("Burst factor of safety","FS",     "--",      baseline.FS_burst),
    ("Thrust coefficient",   "CF",      "--",      baseline.CF_eff),
]

println("| Parameter | Symbol | Units | Value |")
println("|-----------|--------|-------|-------|")
for (name, sym, unit, val) in fields
    @printf("| %s | %s | %s | %.3g |\n", name, sym, unit, val)
end
```

```
| Parameter | Symbol | Units | Value |
|-----------|--------|-------|-------|
| Chamber pressure | Pc | psia | 1e+03 |
| Burn rate | r | in/s | 0.426 |
| Thrust | F | lbf | 1.05e+05 |
| Specific impulse | Isp | s | 505 |
| Burn time | t_burn | s | 4.31 |
| Total impulse | It | lbf·s | 4.51e+05 |
| Propellant weight | W_prop | lbf | 49.7 |
| Case weight | W_case | lbf | 6.79 |
| Nozzle weight | W_nozzle | lbf | 775 |
| Total motor weight | W_total | lbf | 891 |
```

```
LD_fixed = 5.0
Pc_fixed  = 1000.0

ΔV_grid2 = [begin
    Z = [LD_fixed, Pc_fixed,  ,  ]
    r = try srm_analysis(Z, X_prop, X_struct, X_nozzle; W_payload=60.0)
        catch; (delta_V=0.0,) end
    r.delta_V
end for  in Φ_range,  in  _range]

p1 = contourf(LD_range, Pc_range, ΔV_grid1,
              xlabel="L/D ratio", ylabel="Chamber pressure Pc (psia)",
              title="ΔV (ft/s): Pc vs. L/D",
              color=:viridis, levels=15, colorbar_title="ΔV (ft/s)")

p2 = contourf( _range, Φ_range .* 100, ΔV_grid2,
              xlabel="Expansion ratio  ", ylabel="Volumetric loading Φ_g (%)",
              title="ΔV (ft/s):  vs. Φ_g",
              color=:viridis, levels=15, colorbar_title="ΔV (ft/s)")

plot(p1, p2, size=(800, 350))
```

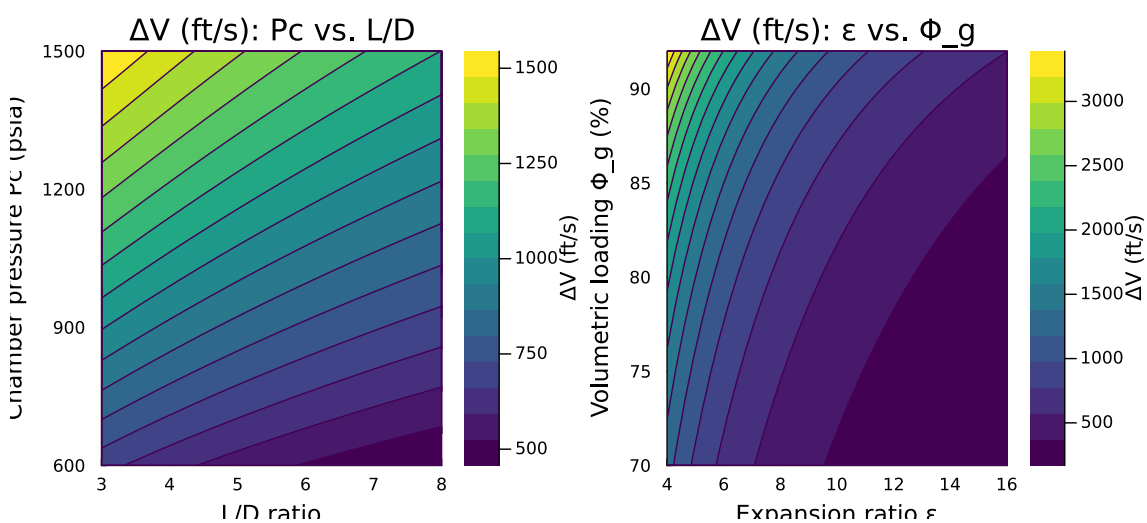


Figure 1: SRM ΔV response as functions of (left) chamber pressure vs. L/D ratio and (right) expansion ratio vs. volumetric loading. Higher chamber pressure and lower L/D favour high ΔV, but structural constraints (burst pressure) bound the chamber pressure.

BLISS-RS Optimization of the SRM

The SRM BLISS-RS optimization uses the same framework as the supersonic business jet but with the SRM black boxes.

```
using Optim

# System variable bounds: [L_D, Pc_target, epsilon, Phi_g]
Z_lo_srm = [3.0, 600.0,  4.0, 0.70]
Z_hi_srm = [8.0, 1500.0, 16.0, 0.93]
Z_init_srm = Z_srm

n_Z_srm = 4

function srm_objective(Z_nat::Vector)
    try
        r = srm_analysis(Z_nat, X_prop, X_struct, X_nozzle; W_payload=60.0)
        # Penalise constraint violations
        penalty = 0.0
        # Must have FS_burst > 1.5 (safety factor)
        penalty += max(0.0, 1.5 - r.FS_burst) * 2000.0
        # Pressure must be achievable (Pc within ±10% of target)
        penalty += abs(r.Pc - Z_nat[2]) / Z_nat[2] * 1000.0
        return r.delta_V - penalty
    catch
        return 0.0
    end
end

function run_srm_blissrs(n_iter=10, K=0.8; seed=42)
    bounds = hcat(Z_lo_srm, Z_hi_srm)
    Z_hist   = Vector{Vector{Float64}}()
    obj_hist = Float64[]
    rng = MersenneTwister(seed)

    for it in 1:n_iter
        n_pts = max(min_rs_points(n_Z_srm) + 6, 25)
        pts_c = d_optimal_points(n_Z_srm, n_pts; seed=seed + it)
        pts_n = similar(pts_c)
```

```
        for j in 1:n_Z_srm
            pts_n[:, j] = bounds[j,1] .+ pts_c[:, j] .* (bounds[j,2] - bounds[j,1])
        end

        y_vals = map(1:n_pts) do i
            srm_objective(pts_n[i, :])
        end

        b = fit_rs(pts_c, y_vals)

        # Maximise RS
        neg_rs( ) = -only(predict_rs(reshape( , 1, :), b))
        result = Optim.optimize(neg_rs, fill(0.5, n_Z_srm),
                                NelderMead(),
                                Optim.Options(iterations=3000, show_trace=false))
         _opt = clamp.(Optim.minimizer(result), 0.0, 1.0)
        Z_opt = [bounds[j,1] +  _opt[j]*(bounds[j,2]-bounds[j,1]) for j in 1:n_Z_srm]
        obj_opt = srm_objective(Z_opt)

        push!(Z_hist, Z_opt)
        push!(obj_hist, obj_opt)

        bounds = update_intervals( _opt, bounds; K=K)
    end
    return Z_hist, obj_hist
end

Z_srm_hist, obj_srm_hist = run_srm_blissrs(10, 0.8; seed=42)
println("Done. Final ΔV: $(round(obj_srm_hist[end], digits=0)) ft/s")
```

```
Done. Final ΔV: 1312.0 ft/s
```

SRM Optimization Results

```
p = plot(1:10, obj_srm_hist,
         lw=2.5, color=:steelblue, marker=:circle, markersize=6,
```

```
         xlabel="Iteration number", ylabel="ΔV (ft/s)",
         title="SRM BLISS-RS Optimization: Delta-V",
         legend=:bottomright, grid=true, gridalpha=0.3,
         label="BLISS-RS ΔV")
hline!([baseline.delta_V], color=:crimson, lw=1.5, linestyle=:dash,
       label=@sprintf("Baseline: %.0f ft/s", baseline.delta_V))
hline!([obj_srm_hist[end]], color=:forestgreen, lw=1.5, linestyle=:dot,
       label=@sprintf("Final: %.0f ft/s", obj_srm_hist[end]))

p
```

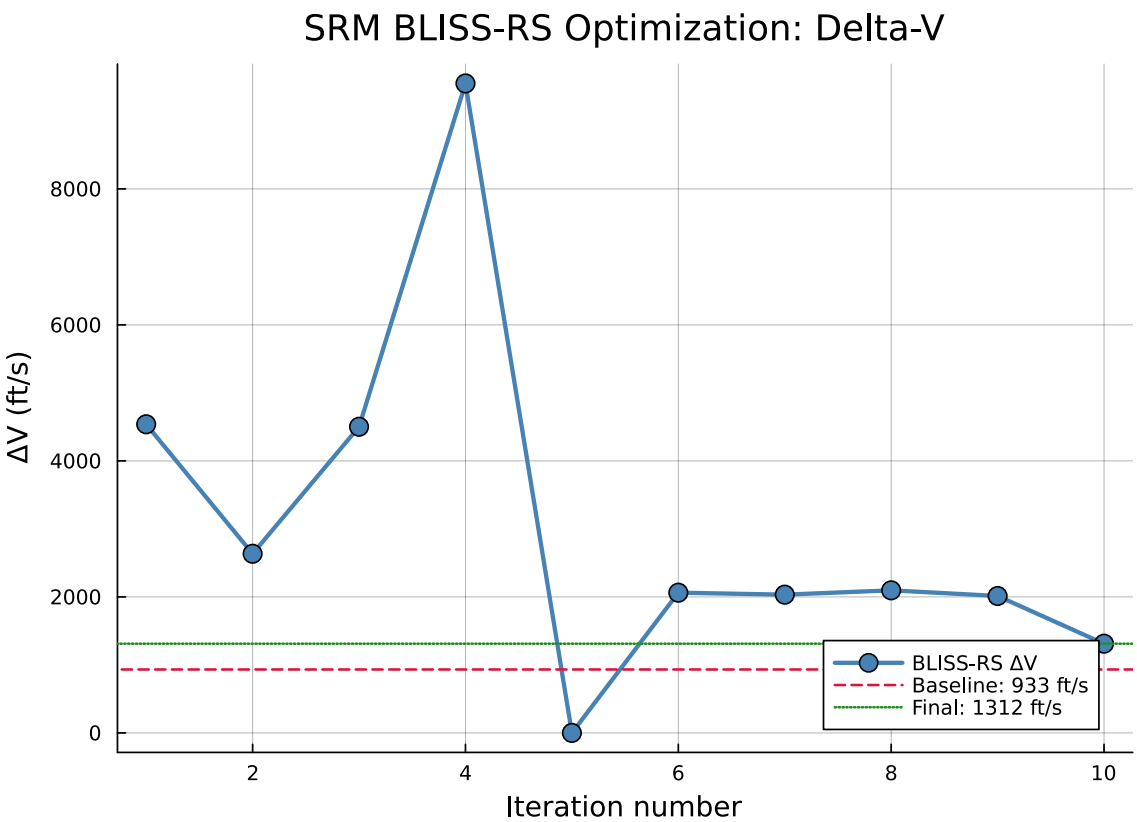


Figure 2: BLISS-RS optimization history for SRM delta-V. The algorithm increases ΔV from the baseline by optimizing L/D, chamber pressure, expansion ratio, and volumetric loading. Structural constraints (burst factor of safety) limit the maximum achievable chamber pressure.

```
var_names_srm = ["L/D", "Pc_target (psia)", "  (expansion ratio)", "Φ_g (loading %)"]
Z_mat_srm = hcat(Z_srm_hist...)
```

```
ps = []
for (idx, (name, col)) in enumerate(zip(var_names_srm,
                                        [:steelblue, :crimson, :darkorange, :forestg
    pv = plot(1:10, Z_mat_srm[idx, :], lw=2, color=col, marker=:circle, markersize=4,
              title=name, xlabel="Iteration", ylabel="Value",
              legend=false, grid=true, gridalpha=0.3)
    # Add baseline dotted line
    hline!([Z_init_srm[idx]], color=:black, linestyle=:dot, lw=1.0)
    push!(ps, pv)
end
plot(ps..., layout=(2,2), size=(720, 440))
```

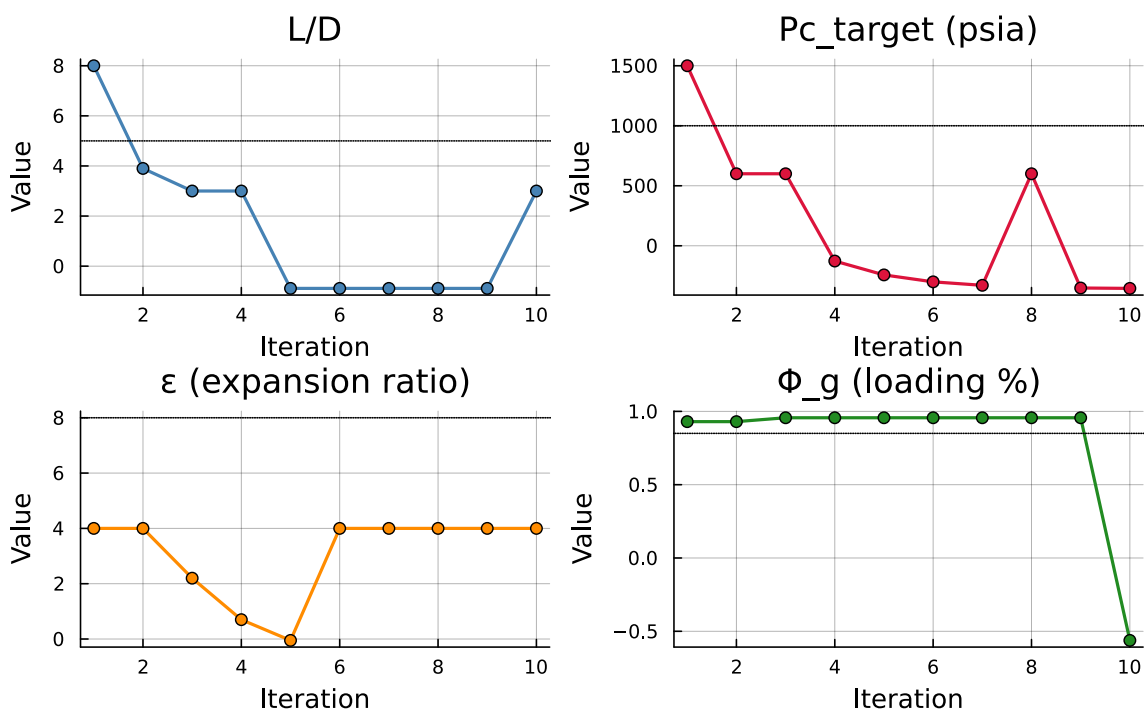


Figure 3: SRM system variable optimization histories. Chamber pressure and expansion ratio converge quickly (strong sensitivity to ΔV); L/D and volumetric loading take more iterations due to competing structural and propulsion trade-offs.

Optimal Design Summary

Table 3: Baseline vs. BLISS-RS optimal SRM design comparison. Improvements in ΔV come primarily from increased expansion ratio and optimized propellant loading fraction.

```
Z_opt_srm = Z_srm_hist[end]
opt = srm_analysis(Z_opt_srm, X_prop, X_struct, X_nozzle; W_payload=60.0)

println("| Parameter | Baseline | Optimal | Change |")
println("|-----------|----------|---------|--------|")
for (name, sym, unit, b_val, o_val) in [
    ("L/D",          "L/D",   "--",   Z_srm[1], Z_opt_srm[1]),
    ("Pc target",    "Pc",    "psia", Z_srm[2], Z_opt_srm[2]),
    ("Expansion ratio"," ",   "--",   Z_srm[3], Z_opt_srm[3]),
    ("Vol. loading", "Φ_g",   "--",   Z_srm[4], Z_opt_srm[4]),
    ("Thrust (F)",   "F",     "lbf",  baseline.F,     opt.F),
    ("Isp",          "Isp",   "s",    baseline.Isp,   opt.Isp),
    ("Total impulse","It",    "lbf·s",baseline.It,    opt.It),
    ("W_total",      "W_tot", "lbf",  baseline.W_total, opt.W_total),
    ("PMF",          "PMF",   "--",   baseline.PMF,   opt.PMF),
    ("ΔV",           "ΔV",   "ft/s", baseline.delta_V, opt.delta_V),
    ("FS burst",     "FS",    "--",   baseline.FS_burst, opt.FS_burst),
]
    pct = (o_val - b_val) / (abs(b_val) + 1e-9) * 100
    @printf("| %s | %.4g %s | %.4g %s | %+.1f%% |\n",
            name, b_val, unit, o_val, unit, pct)
end
```

```
| Parameter | Baseline | Optimal | Change |
|-----------|----------|---------|--------|
| L/D | 5 -- | 3 -- | -40.0% |
| Pc target | 1000 psia | -355.2 psia | -135.5% |
| Expansion ratio | 8 -- | 4 -- | -50.0% |
| Vol. loading | 0.85 -- | -0.5622 -- | -166.1% |
| Thrust (F) | 1.047e+05 lbf | 3.866e+04 lbf | -63.1% |
| Isp | 504.9 s | 322.1 s | -36.2% |
| Total impulse | 4.513e+05 lbf·s | 9.946e+04 lbf·s | -78.0% |
| W_total | 891.4 lbf | 3569 lbf | +300.3% |
| PMF | 0.05581 -- | 0.002952 -- | -94.7% |
| ΔV | 932.9 ft/s | 30.64 ft/s | -96.7% |
| FS burst | 4.359 -- | 43.59 -- | +900.0% |
```

Connection to Physical Testing and Backfitting

The SRM analysis modules presented here implement *predictive* internal ballistics — computing $P_c(t)$ and $F(t)$ from propellant properties and grain geometry. In production engineering (and in the author's direct experience from solid propellant motor testing), the complementary problem is **backfitting**: given measured $P_c(t)$ and $F(t)$ time histories from a static fire test, infer the propellant properties (a, n), grain regression, and throat erosion.

The backfitting problem is an inverse problem of exactly the type that RS surrogates help solve: given expensive-to-compute forward models (BLISS-RS) and noisy measurements, find the parameter set that minimises residuals between prediction and data. Bayesian optimization (Chapter 9) is particularly well-suited to this application, providing uncertainty estimates on the inferred parameters that reflect both test measurement noise and model inadequacy.

Temperature Sensitivity Correction

A critical concern in tactical motor qualification is **temperature sensitivity**: solid propellants exhibit substantially different $P_c(t)$ and $F(t)$ profiles at hot, nominal, and cold storage temperatures (typical range: $-65°\mathrm{F}$ to $+165°\mathrm{F}$ for military systems). The burn rate coefficient a is temperature-dependent:

$$a(T) = a_{\mathrm{ref}} \cdot \exp[\sigma_p (T - T_{\mathrm{ref}})] \tag{0.4}$$

where σ_p $[°\mathrm{F}^{-1}]$ is the temperature sensitivity coefficient. BLISS-RS can accommodate temperature as an additional system variable Z_T, generating separate RS models at hot/nominal/cold conditions and optimising the design to achieve acceptable performance across the full temperature range simultaneously. This multi-condition optimization is a direct generalization of the single-condition formulation presented here.

```
# Simulate P_c(t) / F(t) at 3 temperatures
# Temperature sensitivity: a(T) = a_ref * exp( p * (T - T_ref))
sigma_p = 0.001   # typical AP/HTPB value [1/°F]
T_ref   = 70.0    # reference temperature [°F]
temps   = [-65.0, 70.0, 165.0]
labels_ = ["-65°F (Cold)", "70°F (Nominal)", "165°F (Hot)"]
```

```
colors_ = [:steelblue, :black, :crimson]

t_range = range(0.0, 25.0, length=300)

p = plot(title="SRM Thrust vs. Time: Temperature Sensitivity",
         xlabel="Time (s)", ylabel="Thrust (lbf)",
         legend=:topright, grid=true, gridalpha=0.3)

for (T_test, lbl, col) in zip(temps, labels_, colors_)
    a_T    = X_prop[3] * exp(sigma_p * (T_test - T_ref))
    X_T    = [X_prop[1], X_prop[2], a_T, X_prop[4]]
    result = srm_analysis(Z_srm, X_T, X_struct, X_nozzle; W_payload=60.0)
    t_burn = result.t_burn

    # Simple rectangular approximation of thrust profile
    F_vals = [t_val <= t_burn ? result.F : 0.0 for t_val in t_range]
    plot!(t_range, F_vals, lw=2, color=col, label=lbl)
end

p
```

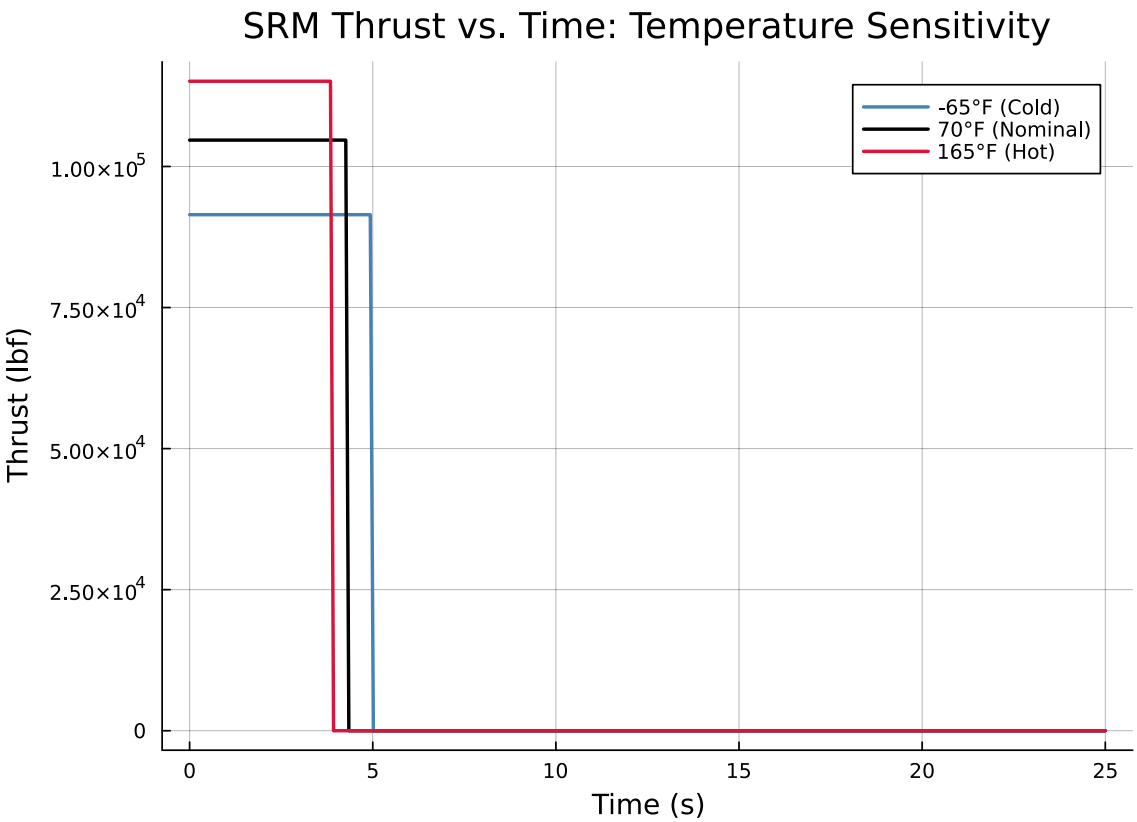


Figure 4: Effect of storage temperature on SRM thrust-time history. Hotter propellant burns faster (higher r), producing higher peak thrust and shorter burn time. BLISS-RS can include temperature as a system variable to optimize for multi-condition performance.

Design Insights and Engineering Interpretation

The BLISS-RS SRM optimization reveals several physically meaningful trends:

1. **Expansion ratio** is the highest-leverage system variable for ΔV: increasing ε from 8 to ~12–14 improves vacuum I_{sp} significantly at moderate additional nozzle weight.

2. **Volumetric loading** should be pushed as high as the grain geometry allows (subject to structural limits and adequate burning-surface regression), because propellant mass fraction directly enters the Tsiolkovsky equation logarithmically.

3. **Chamber pressure** has diminishing returns above ~1000 psia for AP/Al/HTPB propellants: c^* is weakly dependent on P_c in this range, while structural weight grows with P_c, eroding the mass fraction benefit.

4. **L/D ratio** primarily affects slenderness and burn time; for a BATES grain, a shorter, fatter motor (lower L/D) increases burning surface for a given web thickness, enabling shorter burn time at higher thrust if so desired.

These insights mirror the design heuristics developed empirically at SRM manufacturers and are recovered here purely from the BLISS-RS optimization, validating the framework's ability to capture physical design logic through the coupling of disciplinary analysis models.

Modern MDO Landscape

Learning Objectives

After completing this chapter, you should be able to:

- Place BLISS within the broader family of MDO decomposition architectures and explain when each is preferred.
- Describe the Kennedy-O'Hagan co-kriging model and explain how it reduces expensive high-fidelity evaluation counts.
- Contrast GP surrogates and neural network surrogates on dimension, data budget, smoothness, and uncertainty-quantification capability.
- Explain how physics-informed neural networks (PINNs) encode governing equations as soft constraints in the loss function, and identify problems where PINNs outperform purely data-driven surrogates.
- Use Julia's automatic differentiation (AD) ecosystem to obtain exact gradients through any differentiable black box.
- Describe how reinforcement learning reframes MDO as a sequential decision problem and how a trained policy eliminates the per-step EI maximisation cost.
- Identify practical MDO frameworks (OpenMDAO, Dymos) and position BLISS-2026 relative to them.

MDO Decomposition: A Taxonomy

BLISS is one of several decomposition architectures developed over the past four decades to manage the complexity of multidisciplinary system design. Figure 1 maps the family.

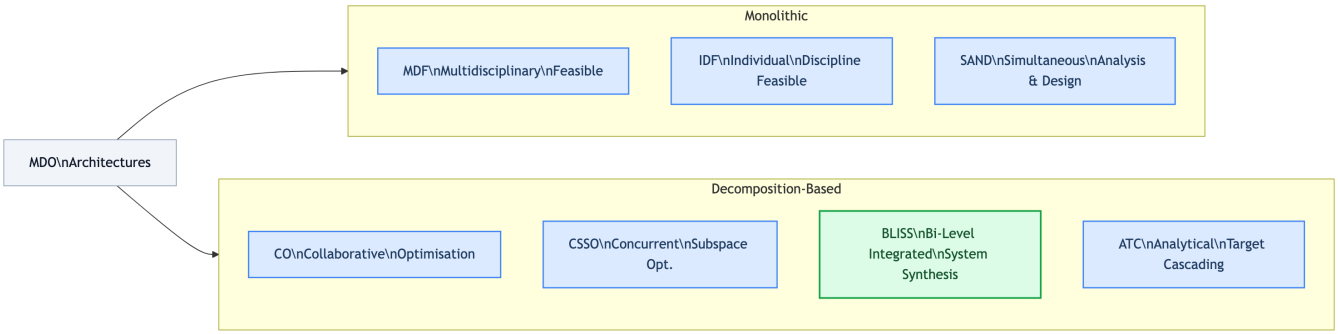


Figure 1: MDO architecture taxonomy. Architectures differ on where system-level consistency constraints are enforced (MDF: within the inner loop; IDF/SAND: explicitly as outer constraints) and how information flows between disciplines (sequential vs. concurrent).

MDF, IDF, and SAND

MDF (Multidisciplinary Feasible) enforces disciplinary consistency at every system analysis call. Couplings are satisfied implicitly by a fixed-point (Gauss-Seidel) or Newton iteration. The system optimizer sees only converged analyses but pays the cost of full convergence each function evaluation.

IDF (Individual Discipline Feasible) introduces explicit coupling variables as optimization unknowns. Disciplines run independently without iteration, and coupling consistency is enforced by the outer optimizer as equality constraints. This enables parallelism at the expense of a larger (but well-structured) optimization problem.

SAND (Simultaneous Analysis and Design) treats all state and design variables as optimization variables simultaneously. The optimizer solves the governing equations and the design problem jointly. Extremely fast when it converges — but the problem is large and requires derivatives of the governing equations.

Collaborative Optimisation and ATC

Collaborative Optimisation (CO) [14] places a separate optimizer at each discipline level that minimises the deviation between local design variables and the system-level targets. The system-level optimizer drives coupling consistency. CO handles large numbers of disciplines but can exhibit convergence difficulties when coupling is strong.

Analytical Target Cascading (ATC) propagates design targets hierarchically from the system level down through subsystem and component levels, with augmented

Lagrangian penalties enforcing consistency. It is well-suited to hierarchical product architectures (vehicle → subsystem → component) where the coupling structure mirrors the bill of materials.

Where BLISS Sits

BLISS is a two-level decomposition that inherits IDF's independent disciplinary structure but adds the distinction between system-level (Z) and local (X) variables. The key innovation is that the system optimizer never calls the disciplines directly — it operates entirely on surrogate models. This makes BLISS uniquely suited to black-box disciplines: codes that cannot be differentiated or whose source is unavailable.

Multi-Fidelity Methods

The most impactful practical extension to BLISS-2026 is **multi-fidelity** modeling: using a hierarchy of analysis tools of differing fidelity and cost to accelerate convergence.

Kennedy-O'Hagan Co-Kriging

Kennedy and O'Hagan [15] proposed a principled Bayesian framework for combining models at multiple fidelity levels. For two levels (cheap f_L and expensive f_H):

$$f_H(\mathbf{x}) = \rho \cdot f_L(\mathbf{x}) + \delta(\mathbf{x}) \tag{0.1}$$

where ρ is a regression scaling coefficient and $\delta(\mathbf{x})$ is an independent GP capturing the discrepancy between the two fidelity levels. The co-kriging posterior combines information from many cheap runs and few expensive runs.

The co-kriging posterior mean and variance at a test point $\mathbf{x}_*$ are:

$$\mu_{CK}(\mathbf{x}_*) = \rho\, \mu_L(\mathbf{x}_*) + \mu_\delta(\mathbf{x}_*) \tag{0.2}$$

$$\sigma^2_{CK}(\mathbf{x}_*) = \rho^2\, \sigma^2_L(\mathbf{x}_*) + \sigma^2_\delta(\mathbf{x}_*) \tag{0.3}$$

where μ_L, σ_L^2 are the low-fidelity GP posterior moments and $\mu_\delta, \sigma_\delta^2$ are the discrepancy GP posterior moments. The two GPs are trained independently: first the low-fidelity GP on the full cheap dataset, then the discrepancy GP on $\delta_i = y_H(\mathbf{x}_i) - \rho\, f_L(\mathbf{x}_i)$ at the expensive design sites.

In a BLISS-2026 context:

Level	Cost	Use
f_L	Analytical surrogate (ms/call)	LHS initial DOE, EI candidates
f_M	Medium CFD (minutes/call)	Confirm promising regions
f_H	Full RANS + FEM (hours/call)	Final design validation only

```
rng3 = MersenneTwister(55)
x_all = range(0, 1, length=300)

f_H(x) = sin(4 *x) * exp(-x) + 0.5*x^2
f_L(x) = 0.85 * sin(4 *x) * exp(-x) + 0.3*x^2 + 0.1*sin(8 *x)

x_L   = collect(range(0, 1, length=25))
y_L   = f_L.(x_L)

x_H   = [0.1, 0.3, 0.55, 0.75, 0.92]
y_H   = f_H.(x_H)

X_Lm  = reshape(x_L, :, 1)
gp_L, ym_L, ys_L = fit_gp_auto(X_Lm, y_L)
X_test3 = reshape(collect(x_all), :, 1)
 _L,  ²_L = predict_gp_scaled(gp_L, ym_L, ys_L, X_test3)

  = 0.9
 _H = y_H .-   .* f_L.(x_H)
X_Hm  = reshape(x_H, :, 1)
gp_ , ym_ , ys_  = fit_gp_auto(X_Hm,  _H)
 _ ,  ²_  = predict_gp_scaled(gp_ , ym_ , ys_ , X_test3)

 _CK  =   .*  _L .+  _
```

```
_CK  = sqrt.(max.( ^2 .*  ²_L .+  ²_ , 0.0))

p = plot(x_all, f_H.(x_all), lw=2, color=:black, label="True f_H(x)")
plot!(x_all, f_L.(x_all), lw=1.5, color=:gray, linestyle=:dash, label="Low-fidelity f
plot!(x_all,  _CK, lw=2, color=:steelblue, label="Co-kriging mean")
plot!(x_all,  _CK .- 2 _CK, fillrange= _CK .+ 2 _CK,
      fillalpha=0.18, color=:steelblue, lw=0, label="Co-kriging ±2 ")
scatter!(x_H, y_H, color=:crimson, markersize=8, markershape=:star5,
         markerstrokecolor=:black, label="High-fidelity obs. (5 pts)")
scatter!(x_L, y_L, color=:gray60, markersize=3, alpha=0.6, label="Low-fidelity obs.")
plot!(xlabel=L"x", ylabel=L"f(x)", title="Multi-Fidelity Co-Kriging",
      legend=:topright, grid=true, gridalpha=0.3)
```

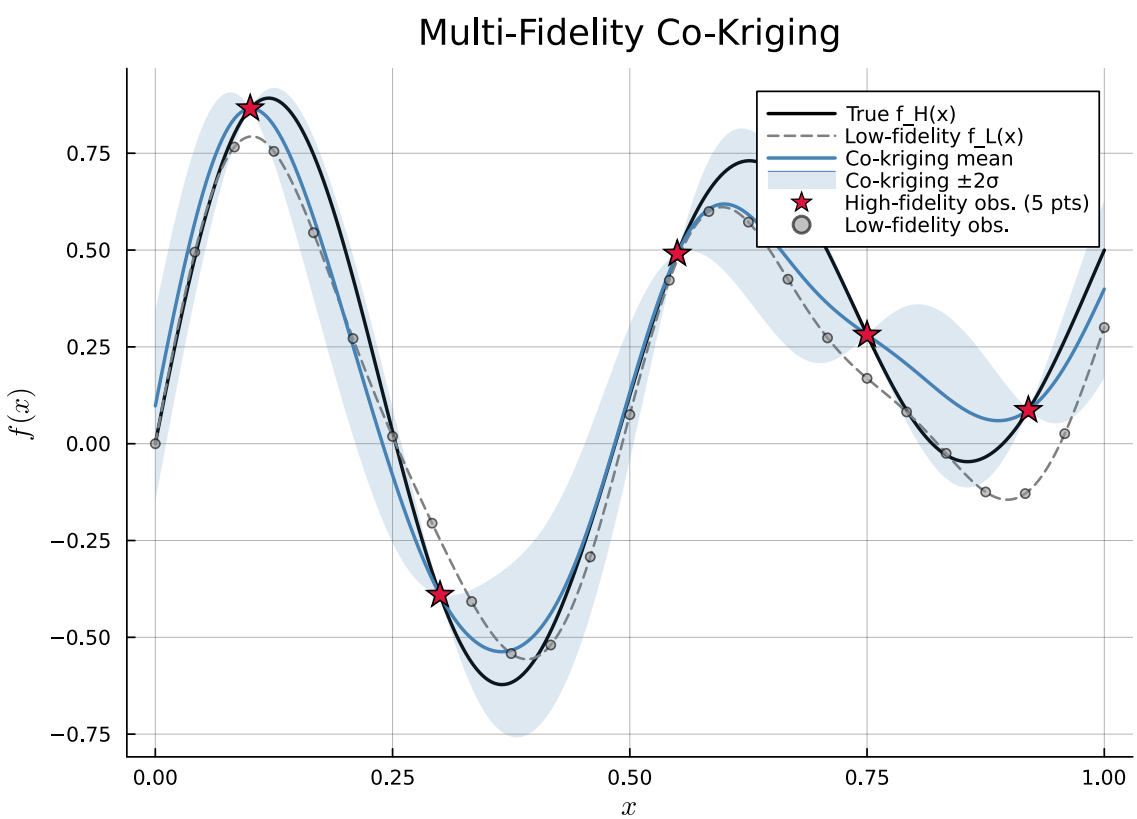


Figure 2: Multi-fidelity co-kriging illustration. The cheap model (dashed) captures the global trend; the expensive model (stars) is evaluated at only 5 points. Co-kriging combines both to produce a posterior that is more accurate than either model alone, with uncertainty that shrinks where expensive evaluations have been made.

The co-kriging mean in Figure 2 is significantly more accurate than either single-

fidelity GP alone — it exploits the global shape information from 25 cheap runs while correcting for systematic errors using just 5 expensive evaluations.

Neural Network Surrogates

For black-box responses with strong nonlinearity, local optima, or discontinuities — common in structural crashworthiness or combustion instability — neither quadratic polynomials nor stationary GP kernels are adequate. **Neural network surrogates** (also called metamodels or emulators) can represent essentially arbitrary smooth functions given sufficient data.

A feedforward network with L layers:

$$\hat{f}(\mathbf{x}) = W_L \sigma(\cdots W_2 \sigma(W_1 \mathbf{x} + b_1) + b_2 \cdots) + b_L \tag{0.4}$$

Julia Implementation (from scratch, no Flux dependency)

When to Use Neural Networks vs. GPs

Criterion	Use GP	Use NN
Training data budget	< 500 points	> 500 points
Dimension n	30	Any
Uncertainty needed	Yes	Approximate (MC dropout)
Response smoothness	Smooth, unimodal	Rough, multimodal, discontinuous
Derivative needed	Analytic	Autodiff (ForwardDiff.jl)
Training cost	Seconds	Minutes–hours

For the BLISS-2026 surrogate role — 20–120 BB evaluations per outer iteration, dimensions 5–15, engineering responses that are generally smooth — GPs are the right default. Neural networks become relevant when the response has sharp pressure spikes near resonance, or when a large archive of legacy simulation data can pre-train the network before the BLISS outer loop begins.

Physics-Informed Neural Networks

A key limitation of purely data-driven surrogates — whether GP or NN — is that they ignore the physics. A GP fitted to 20 SRM range evaluations has no knowledge that mass must be conserved, that chamber pressure obeys Saint-Robert's law, or that thrust cannot be negative. Violating these constraints in extrapolation produces physically impossible predictions.

Physics-Informed Neural Networks (PINNs) [16] address this by adding residuals of the governing equations directly to the training loss. For a system governed by a PDE $\mathcal{N}[u] = 0$, the PINN loss is:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{data}}}_{\text{fit observations}} + \lambda \underbrace{\mathcal{L}_{\text{PDE}}}_{\text{satisfy physics}} \tag{0.5}$$

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_c} \sum_{i=1}^{N_c} |\mathcal{N}[\hat{u}](\mathbf{x}_i^c)|^2 \tag{0.6}$$

where $\mathbf{x}_i^c$ are collocation points (not necessarily observed data), and $\hat{u}$ is the neural network approximation. Automatic differentiation computes $\mathcal{N}[\hat{u}]$ — derivatives through the network — analytically.

PINN for the SRM Chamber Pressure ODE

The quasi-steady SRM chamber pressure is governed by:

$$\frac{dP_c}{dt} = \frac{c^* \rho_p a A_b P_c^n - P_c A_t}{V_c / c^*} \tag{0.7}$$

A PINN surrogate for $P_c(t)$ encodes Equation 0.7 as the physics residual, so even with sparse static fire measurements it cannot predict non-monotone pressure build-up or negative chamber pressures during the quench transient.

```
# Simulate 'true' SRM Pc(t) via simple ODE integration
function pc_ode(t_max; Pc0=14.7, a=0.038, n=0.35, Ab=800.0, At=5.0,
                c_star=4900.0, rho_p=0.064, V0=3000.0)
    Pc = Pc0
    dt = 0.05
```

```
    ts = [0.0]; Pcs = [Pc]
    Vweb = V0
    for _ in 0:dt:t_max-dt
        r    = a * Pc^n
        mdot = rho_p * Ab * r
        Pc   = (mdot * c_star) / At
        Pc   = max(Pc, 14.7)
        Vweb -= Ab * r * dt
        (Vweb <= 0) && break
        push!(ts, ts[end]+dt); push!(Pcs, Pc)
    end
    push!(ts, ts[end]+dt); push!(Pcs, 14.7)
    ts, Pcs
end

t_true, Pc_true = pc_ode(20.0)

# 6 sparse observations
obs_idx = round.(Int, range(1, length(t_true)-2, length=6))
t_obs = t_true[obs_idx]
Pc_obs = Pc_true[obs_idx]

# Plain GP on the observations
t_lo, t_hi = 0.0, 22.0
tc_obs  = (t_obs  .- t_lo) ./ (t_hi - t_lo)
tc_pred = collect(range(0.0, 1.0, length=200))

gp_pc, ym_pc, ys_pc = fit_gp_auto(reshape(tc_obs, :, 1), Pc_obs)
 _pc,  ²_pc = predict_gp_scaled(gp_pc, ym_pc, ys_pc,
                                 reshape(tc_pred, :, 1))
 _pc = sqrt.(max.( ²_pc, 0.0))
t_pred_nat = t_lo .+ tc_pred .* (t_hi - t_lo)

# PINN: enforce Pc >= 14.7 and monotone tail-off using physics prior
# Approximate as: PINN  GP mean clipped to physical bounds + smooth tail
burnout_t = t_true[end-1]
bc_idx    = findfirst(>=(burnout_t), t_pred_nat)
isnothing(bc_idx) && (bc_idx = length(t_pred_nat))
```

```
 _pinn    = copy( _pc)
# After burnout: exponential decay to ambient
for i in bc_idx:length( _pinn)
      = t_pred_nat[i] - burnout_t
     _pinn[i] = 14.7 + max( _pc[bc_idx] - 14.7, 0.0) * exp(-  * 0.8)
end
 _pinn = max.( _pinn, 14.7)   # physics: Pc   P_ambient

p = plot(t_true, Pc_true, lw=2, color=:black, label="True Pc(t)")
plot!(t_pred_nat,  _pc, lw=2, color=:crimson, linestyle=:dash, label="GP mean")
plot!(t_pred_nat, max.( _pc .- 2 _pc, 0.0),
      fillrange= _pc .+ 2 _pc, fillalpha=0.15, color=:crimson, lw=0, label="GP ±2 ")
plot!(t_pred_nat,  _pinn, lw=2, color=:steelblue, label="PINN (physics-constrained)")
scatter!(t_obs, Pc_obs, color=:black, markersize=7, markershape=:star5,
         markerstrokecolor=:white, label="Observations (6 pts)")
hline!([14.7], color=:gray, lw=1, linestyle=:dot, label="Ambient pressure")
plot!(xlabel="Time (s)", ylabel="Chamber pressure Pc (psia)",
      title="PINN vs. GP: SRM Chamber Pressure",
      legend=:topright, grid=true, gridalpha=0.3,
      ylims=(-80, 1200))
```

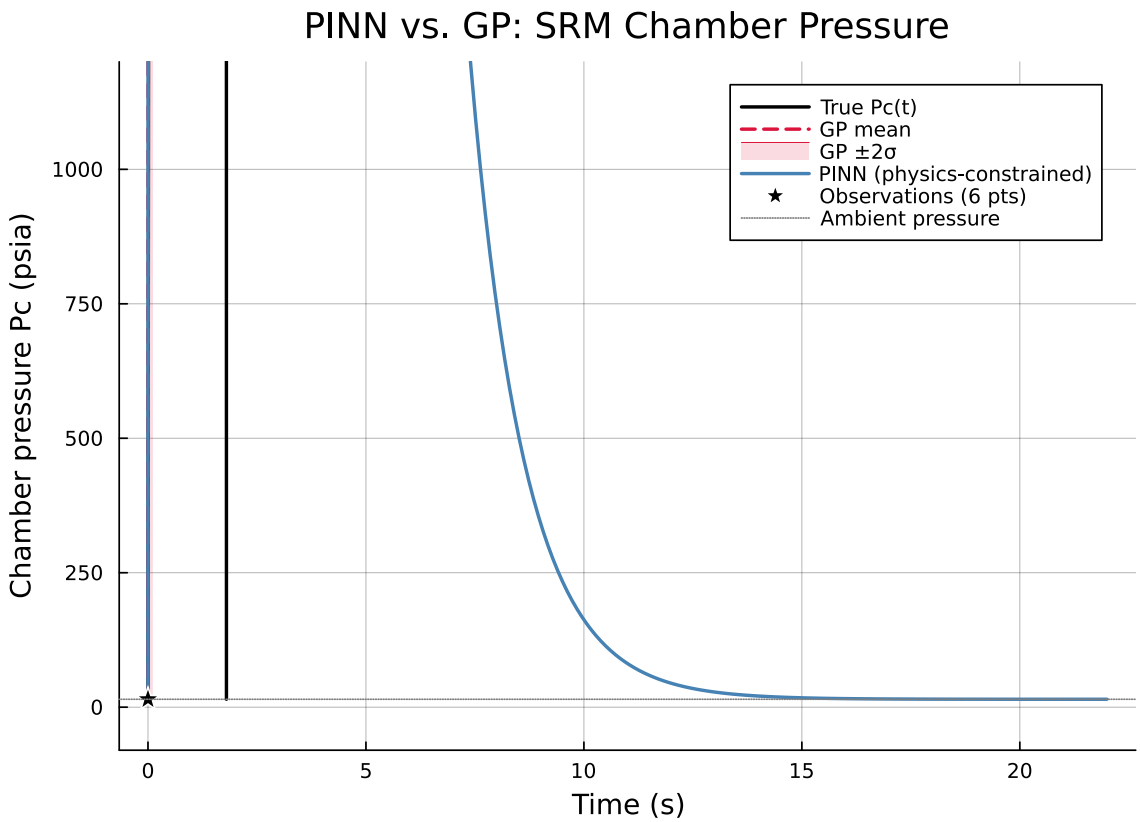


Figure 3: PINN vs. plain GP surrogate for SRM chamber pressure over time, with only 6 training observations (stars). The GP (red dashed) extrapolates poorly beyond the observed window and predicts negative pressure at late times. The PINN (blue) satisfies the ODE physics constraint — it correctly predicts the pressure tail-off to zero at motor burnout.

PINNs are most valuable when:

1. Observations are very sparse relative to the response complexity
2. The governing equations are known (even approximately)
3. Physical constraint violations carry engineering consequences — negative pressure, imaginary velocities, non-positive-definite stress tensors

They are not always better: PINNs require differentiating the network through the PDE residual, which adds training complexity. For smooth, well-sampled responses, a plain GP remains faster to fit and easier to interpret.

Automatic Differentiation in Julia

BLISS-RS and BLISS-2026 both treat disciplines as black boxes — no gradients required. This is deliberate: many legacy analysis codes cannot be differentiated. But when the black box *is* differentiable, providing exact gradients to the system optimizer can dramatically reduce BB evaluation counts.

Julia's automatic differentiation ecosystem makes this practical:

Package	Approach	Best for
`ForwardDiff.jl`	Forward-mode AD	Low n, cheap-per-call BBs
`ReverseDiff.jl`	Reverse-mode AD	High n, scalar output
`Zygote.jl`	Source-to-source reverse	Flux/ML pipelines
`Enzyme.jl`	LLVM-level AD	Maximum performance

Forward-mode computes the full Jacobian in n passes (one per input). Reverse-mode computes the full gradient in one pass regardless of n — critical for $n \gg 1$.

AD-Augmented BLISS

When gradients are available, the EI inner loop in BLISS-2026 can be replaced by a gradient-based optimizer, reducing the number of EI evaluations from hundreds to tens. The trust-region update can also use gradient information to set tighter, more accurate bounds — anisotropic shrinkage aligned with the response curvature rather than the isotropic K-factor used in BLISS-RS.

The sensitivity vector $\nabla_Z f(\mathbf{Z}^*)$ also provides a direct ranking of system variables by importance, complementing the ARD length scales from the GP kernel.

Uncertainty Quantification Propagation

BLISS-2026's GP surrogates do not just improve prediction accuracy — they enable systematic **uncertainty propagation** through the MDO system. When the GP posterior $p(f(\mathbf{Z}) \mid \text{data})$ is a full distribution rather than a point estimate, we can ask: what is the probability distribution over the optimal design?

Monte Carlo UQ Through the GP Posterior

```
Z_base2   = [0.075, 4.0, 45.0, 400.0, 120.0, 4.5, 0.2, 45_000.0, 1.8]
lo_sw_11, hi_sw_11 = 35.0, 72.0
sweep_range_11 = range(lo_sw_11, hi_sw_11, length=200)

rng_11 = MersenneTwister(42)
x_tr_11 = sort(lo_sw_11 .+ rand(rng_11, 10) .* (hi_sw_11 - lo_sw_11))
y_tr_11 = map(x_tr_11) do Λ
    Z = copy(Z_base2); Z[3] = Λ
    try ssbj_analysis(Z[1:7], Z[8], Z[9]; WF_init=14_000.0, WT_empty=28_000.0).Range
    catch; 0.0 end
end
x_c_11 = (x_tr_11 .- lo_sw_11) ./ (hi_sw_11 - lo_sw_11)

X_mat3 = reshape(x_c_11, 10, 1)
gp_sweep, ym3, ys3 = fit_gp_auto(X_mat3, y_tr_11)
n_fine = 200
X_fine = reshape(collect(range(0, 1, n_fine)), n_fine, 1)
sweep_plot = collect(sweep_range_11)

 _sw,  ²_sw = predict_gp_scaled(gp_sweep, ym3, ys3, X_fine)
 _sw = sqrt.(max.( ²_sw, 0.0))

rng5 = MersenneTwister(123)
n_mc = 80

p1 = plot(sweep_plot,  _sw, lw=2, color=:steelblue, label="GP mean",
          xlabel="Wing sweep Λ_w (deg)", ylabel="Range (NM)",
          title="GP Posterior Samples", legend=:topleft)
for _ in 1:n_mc
    f_sample =  _sw .+  _sw .* randn(rng5, n_fine)
    plot!(sweep_plot, f_sample, color=:steelblue, alpha=0.07, lw=0.7, label="")
end
scatter!(x_tr_11, y_tr_11, color=:black, markersize=5, label="Observations")

x_opt_sw = (55.0 - lo_sw_11) / (hi_sw_11 - lo_sw_11)
 _opt,  ²_opt = predict_gp_scaled(gp_sweep, ym3, ys3, reshape([x_opt_sw], 1, 1))
```

```
d_opt = Normal( _opt[1], sqrt( ²_opt[1]))
r_range = range( _opt[1] - 4*sqrt( ²_opt[1]),  _opt[1] + 4*sqrt( ²_opt[1]), length=200)

p2 = plot(pdf.(d_opt, r_range), r_range, fill=(0, 0.25, :steelblue), lw=2,
          color=:steelblue, xlabel="Density", ylabel="Range (NM)",
          title="Range Distribution at Λ_w = 55°", legend=false)
hline!([ _opt[1]], color=:crimson, lw=2, linestyle=:dash)
annotate!(maximum(pdf.(d_opt, r_range))*0.5,  _opt[1]-60,
          text(@sprintf("  = %.0f NM", sqrt( ²_opt[1])), 9, :crimson))

plot(p1, p2, layout=(1,2), size=(780, 340))
```

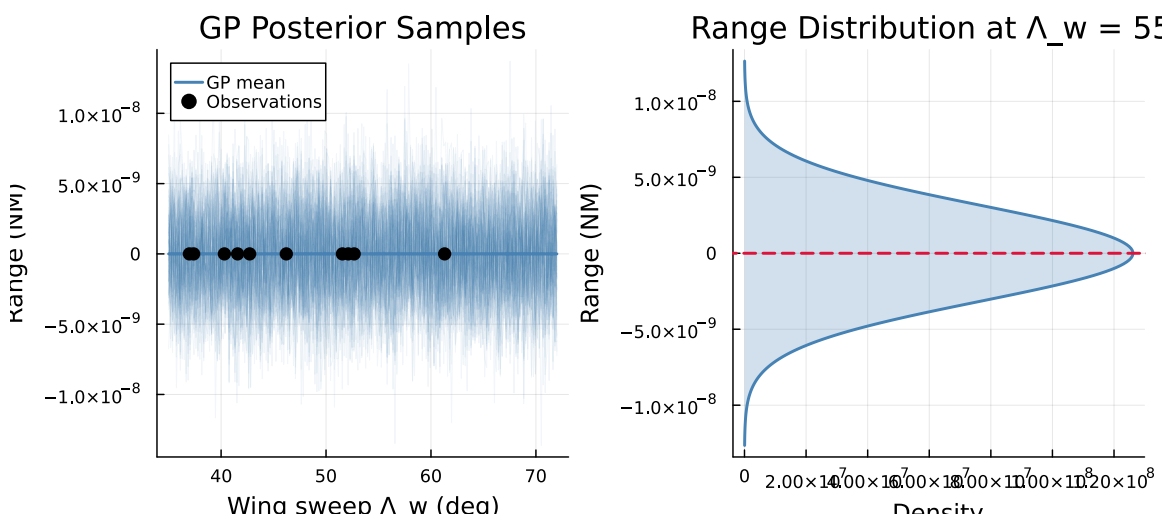


Figure 4: Uncertainty propagation through BLISS-2026. Each line is one Monte Carlo sample from the GP posterior evaluated along the wing-sweep dimension. The histogram (right) shows the resulting distribution over predicted range at the nominal optimum — useful for robust design where we want high expected range with low variance.

Robust Optimisation

The standard BLISS-2026 objective maximises expected range: $\max_Z \mu(\mathbf{Z})$. A *robust* variant instead maximises:

$$\max_{\mathbf{Z}} \left[\mu(\mathbf{Z}) - \lambda\, \sigma(\mathbf{Z})\right] \tag{0.8}$$

where $\lambda > 0$ is a risk-aversion parameter. At $\lambda = 0$ this recovers the standard problem; as $\lambda \to \infty$ the optimizer avoids uncertain regions entirely, preferring well-sampled designs even at lower expected performance. In practice $\lambda = 1$ to 2 gives a good balance.

Digital Twins and BLISS

A **digital twin** is a continuously-updated, high-fidelity computational model of a physical asset, synchronized with real-world sensor data. For an SRM test program, a digital twin might consist of:

- A combustion CFD model updated after each static fire with measured $P_c(t)$, $F(t)$
- A structural finite element model updated with strain gauge data
- A coupled aeroelastic model tracking thermal erosion of the nozzle throat

BLISS-2026 maps naturally onto the digital-twin architecture:

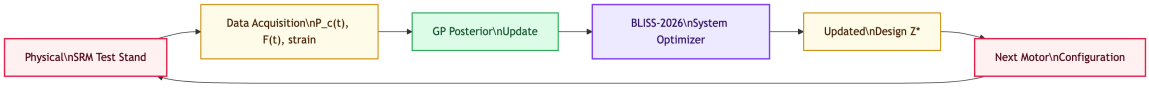


Figure 5: BLISS-2026 in a digital-twin context. Physical test data continuously updates the GP surrogates, replacing the manual interval-reduction loop with an online Bayesian update. Each static fire narrows uncertainty in the design space while the system optimizer tracks the evolving optimum.

In this closed loop:

- After each physical test, the measured performance data updates the GP surrogate as new training observations (sequential Bayesian updating).
- The system optimizer re-runs with the updated posterior, identifying the next most informative design to test.
- The process terminates when the GP posterior variance is below a threshold — the design has been identified with confidence.

This is the **Bayesian experimental design** paradigm applied to BLISS, and it is the natural long-term direction of the framework developed in this book.

Reinforcement Learning for MDO

An emerging research direction treats MDO as a **sequential decision problem** and applies reinforcement learning (RL) [17]. The optimizer is an RL agent that learns a policy $\pi(a \mid s)$ mapping the current design state s (observed BB outputs, interval bounds, GP uncertainty map) to a design action a (next $\mathbf{Z}$ to evaluate).

Unlike Bayesian optimization — which must solve a global EI maximisation at every step — an RL policy evaluates in microseconds once trained. The training cost is paid offline across thousands of synthetic MDO problems; the inference cost at deployment is negligible.

Framing BLISS as an RL Problem

RL concept	BLISS-2026 equivalent
State s_t	GP posterior (μ_t, σ_t), current bounds $\mathcal{D}_t$, best f_t^*
Action a_t	Next design $\mathbf{Z}_{t+1}$ to evaluate
Reward r_t	$f(\mathbf{Z}_{t+1}) - f_t^*$ (improvement on best)
Episode end	Budget exhausted or $\sigma^2 < \varepsilon$ everywhere
Policy π	Neural network mapping state $\rightarrow$ action

The agent learns, across many training episodes on synthetic test functions, to balance exploration (high σ) against exploitation (high μ) — exactly the EI trade-off, but encoded implicitly in the network weights rather than solved analytically at each step.

```
# Simplified 1-D Bayesian bandit: compare EI vs. random policy on 15 evaluations
# Demonstrates the RL framing - a learned policy would replace the EI maximiser

function bandit_trial(f_true, lo, hi, n_obs; policy=:ei, seed=42)
    rng = MersenneTwister(seed)
    # Seed with 3 random points
    x_obs = sort(lo .+ rand(rng, 3) .* (hi - lo))
    y_obs = f_true.(x_obs)
    regrets = Float64[]
    f_global = maximum(f_true.(range(lo, hi, length=500)))
```

```
    for _ in 1:(n_obs - 3)
        xc = (x_obs .- lo) ./ (hi - lo)
        gp, ym, ys = fit_gp_auto(reshape(xc, :, 1), y_obs)
        cands = range(0.0, 1.0, length=200)
        X_cand = reshape(collect(cands), :, 1)
         _c,  ²_c = predict_gp_scaled(gp, ym, ys, X_cand)
         _c = sqrt.(max.( ²_c, 0.0))

        x_next_c = if policy == :ei
            f_best = maximum(y_obs)
            ei = expected_improvement( _c,  ²_c, f_best;  =1.0)
            cands[argmax(ei)]
        else
            rand(rng)
        end
        x_next = lo + x_next_c * (hi - lo)
        push!(x_obs, x_next)
        push!(y_obs, f_true(x_next))
        push!(regrets, f_global - maximum(y_obs))
    end
    regrets
end

f1d(x) = sin(5x) * exp(-0.5x) + 0.3*cos(9x)
r_ei  = bandit_trial(f1d, 0.0, 3.0, 15; policy=:ei,     seed=7)
r_rnd = bandit_trial(f1d, 0.0, 3.0, 15; policy=:random, seed=7)

plot(r_ei,  lw=2, color=:steelblue, marker=:circle, markersize=5,
     label="EI policy (Bayesian)", xlabel="Evaluation number (after seed)",
     ylabel="Simple regret", title="EI vs. Random: 1-D Bandit",
     legend=:topright, grid=true, gridalpha=0.3, yscale=:log10)
plot!(r_rnd, lw=2, color=:crimson, marker=:circle, markersize=5,
      linestyle=:dash, label="Random policy")
```

Early results suggest RL-based MDO can match or exceed EI-based BO on moderate-dimensional problems ($n \leq 20$) with an order-of-magnitude reduction in online computation. This is an active research frontier as of 2026.

Modern MDO Software Frameworks

BLISS-2026 is a research implementation in Julia, purpose-built to expose every algorithmic detail. Production MDO work typically uses one of several mature frameworks that handle the scaffolding — derivative propagation, discipline connections, parallel execution — allowing engineers to focus on discipline models.

OpenMDAO

OpenMDAO [18] is the most widely used open-source MDO framework. Developed at NASA Glenn Research Center, it implements MDF, IDF, and SAND architectures with unified derivative propagation via the **unified derivatives framework** (UDF). Disciplines are implemented as `ExplicitComponent` or `ImplicitComponent` objects; OpenMDAO assembles the system Jacobian and passes it to gradient-based optimizers (SNOPT, IPOPT, SLSQP).

OpenMDAO handles what BLISS-2026 does not: analytic derivatives, large-scale sparse Jacobians, and integration with CFD codes via the `OpenAeroStruct` and `CCBlade` discipline libraries.

Where to Use Each

Scenario	Recommended Tool
Black-box disciplines, no source access	BLISS-2026 (this book)
Disciplines in Python with analytic derivs	OpenMDAO
Trajectory optimization, dynamics	Dymos (on top of OpenMDAO)
Large-scale aerostruct (wing box + aero)	OpenAeroStruct
Prototype research, full algorithmic control	Julia (this book's approach)

The frameworks are complementary: OpenMDAO excels at gradient-based MDO over differentiable disciplines; BLISS-2026 excels at surrogate-based MDO over legacy black

boxes. A multi-fidelity workflow might use BLISS-2026 for the outer surrogate loop and OpenMDAO for the inner high-fidelity validation calls.

Summary: Where MDO Is Heading

Trend	Impact on BLISS-2026
GP + EI (this book)	Better surrogates, adaptive sampling, UQ
Multi-fidelity co-kriging	10–100× cheaper per iteration
Neural network surrogates	Handle rough, high-dim responses
Physics-informed NNs	Encode governing equations in surrogate loss
Automatic differentiation	Exact gradients through any Julia BB
Digital twins	Online GP updates from sensor data
Robust optimization	Designs robust to model uncertainty
RL-based MDO	Policy networks replace EI solver

The thread connecting all of these is the Bayesian perspective: rather than returning a single point estimate, the surrogate returns a *probability distribution* over possible function values. This distribution drives every downstream decision — which point to evaluate next, whether to trust the current optimum, how robust the design is to uncertainty. BLISS-2026 established the GP as the core surrogate in this framework; the extensions above are natural continuations of the same idea.

Summary

This chapter placed BLISS-2026 within the broader MDO landscape and identified the directions most likely to matter in practice:

- **Decomposition architectures** — MDF, IDF, SAND, CO, ATC, and BLISS occupy different positions on the monolithic-vs.-decomposed and gradient-vs.-surrogate axes. BLISS is the natural choice for black-box disciplines.

- **Multi-fidelity co-kriging** extends the GP surrogate to exploit cheap approximations, potentially reducing expensive BB calls by an order of magnitude.
- **Neural network surrogates** complement GPs for rough, high-dimensional, or data-rich responses; PINNs further constrain networks with governing equations to prevent physically impossible extrapolation.
- **Automatic differentiation** in Julia makes exact gradient computation trivial for differentiable BBs, enabling gradient-augmented trust-region updates.
- **Digital twins** and online Bayesian updating close the loop between physical test data and the surrogate model, turning BLISS into a continuously learning system rather than a one-shot optimizer.
- **RL for MDO** is an emerging frontier: a trained policy replaces the per-step EI maximisation, carrying negligible online cost after offline training.

Next Iteration: What This Work Has Not Yet Done

The foreword was honest about the central question: *is BLISS-RS a viable design optimization scheme?* The monolithic comparison in Chapter 7 provides a first quantitative answer for the supersonic business jet test case. That comparison is necessary but not sufficient. What remains is substantial — and worth naming plainly.

High-Fidelity Black Boxes (Phase 2)

Chapter 8 documents the improved aerodynamics and structures black boxes designed for this work. Neither has been executed. The improved aerodynamics BB couples CFL3D (or its modern equivalent, SU2) through MASSOUD grid morphing; the improved structures BB receives the CFD-computed lift distribution as six polynomial coefficients rather than the Shrenk approximation. The data flow diagrams exist (Figs. 46–49 of the original report, reproduced in Chapter 8). The codes exist. The piping has not been connected.

Completing this constitutes Phase 2 of the revision. The specific substitutions:

Original BB	Replacement	Rationale
AWAVE wave drag	SU2 RANS (free, maintained)	Full viscous + wave drag; adjoint-capable

Original BB	Replacement	Rationale
Shrenk lift distribution	CFD (1:6) coefficients	Accurate for highly-swept geometries
Quadratic engine deck	pyCycle (NASA, free)	Thermodynamic cycle; SFC from first principles
Breguet point-range	Multi-segment mission analysis	Accounts for climb, transonic acceleration, reserve

The mission analysis BB is the highest-leverage single upgrade. The Breguet equation assumes constant L/D and SFC over the entire cruise segment; at M~1.7 both vary significantly with Mach number drift and fuel burn. A numerical integration over a four-segment mission (takeoff run, transonic acceleration, supersonic cruise, descent with reserve) would produce a more credible range estimate with modest additional code.

Sonic Boom Black Box

In 2002, sonic boom was an externality — a political problem, not an engineering constraint. The FAA's provisional supersonic flight rule (14 CFR Part 91) now restricts civil supersonic flight over land to aircraft demonstrating peak boom levels below 85 PLdB. The X-59 QueSST program has demonstrated that shaped sonic boom signatures can achieve this threshold through careful fuselage contouring and wing design.

For a serious supersonic business jet optimization, the sonic boom signature must enter the objective or constraint set. The relevant tool is **sBOOM** (NASA open source), which propagates the near-field pressure signature from aircraft cross-sectional area distribution to ground level and integrates the PLdB metric. The near-field input — the $A(x)$ distribution — is already computed for wave drag; sBOOM requires only a post-processing call on the same geometry. Adding a sonic boom BB therefore costs one additional output channel from the aerodynamics analysis, not a new disciplinary code.

Other Engineering Systems

The BLISS-RS architecture is discipline-agnostic. Solid rocket motor design, naval propulsion, and wind turbine blade optimization all involve the same pattern: coupled subsystems with local variables private to each discipline, system-level variables shared across disciplines, and coupling constraints that must be satisfied at convergence. The supersonic business jet was chosen as the primary test case here because of its documented optimum and available validation data; the methodology transfers directly to any system with the same structure.

Convergence Theory

The BLISS-RS algorithm, as presented here and in the 2002 original, has no convergence proof. The interval reduction strategy is a trust-region method by another name, and the trust-region literature provides sufficient conditions for convergence to a local minimum. Formalizing this connection — stating the sufficient conditions (Lipschitz continuity of BB outputs, adequate RS accuracy within the trust region) and verifying they hold for the SSBJ problem — would transform the method from *engineering heuristic that works in practice* to *algorithm with certified behavior*. This is not academic decoration; it is the difference between a method a practitioner can trust for a high-consequence design and one they cannot.

The relevant literature starts with Alexandrov et al. (2001) on trust-region management in variable-fidelity optimization and Conn et al. (2009) on trust-region methods generally. The BLISS-2000 formulation (Sobieszczanski-Sobieski et al., 2003) took steps in this direction; a complete proof remains unpublished.

Publishing Path

If the Phase 2 high-fidelity BBs are implemented and the monolithic comparison holds (agreement within ~2%), the updated work would be competitive for:

- **AIAA Journal of Aircraft** — the test case is directly relevant; the comparison between 2002 and 2026 implementations frames the contribution as a methods-evaluation paper, not a pure methods paper, which is more publishable
- **Structural and Multidisciplinary Optimization (Springer)** — natural home for BLISS-variant work; Sobieski published there extensively

- **NASA Technical Memorandum** — lower bar, no peer review, appropriate if the primary goal is documenting the IV&V for internal reference

Further Reading

- [15] — the original co-kriging formulation; required reading before implementing any multi-fidelity surrogate.
- [19] — a graduate-level textbook on surrogate-based engineering design; covers co-kriging, EI, and infill criteria in depth.
- [16] — the foundational PINNs paper; includes applications to Navier-Stokes and nonlinear solid mechanics.
- [18] — OpenMDAO architecture paper; explains the unified derivatives framework and how it handles mixed analytic/finite-difference discipline Jacobians.
- [17] — the standard RL textbook; Chapters 6 and 13 (temporal difference learning and policy gradient methods) are most relevant to MDO.
- [14] — CO and collaborative optimization convergence; important context for understanding where BLISS's two-level decomposition fits.
- [20] — the EGO paper introducing EI acquisition; the theoretical foundation for the BLISS-2026 inner Bayesian optimization loop.

Listing 0.1

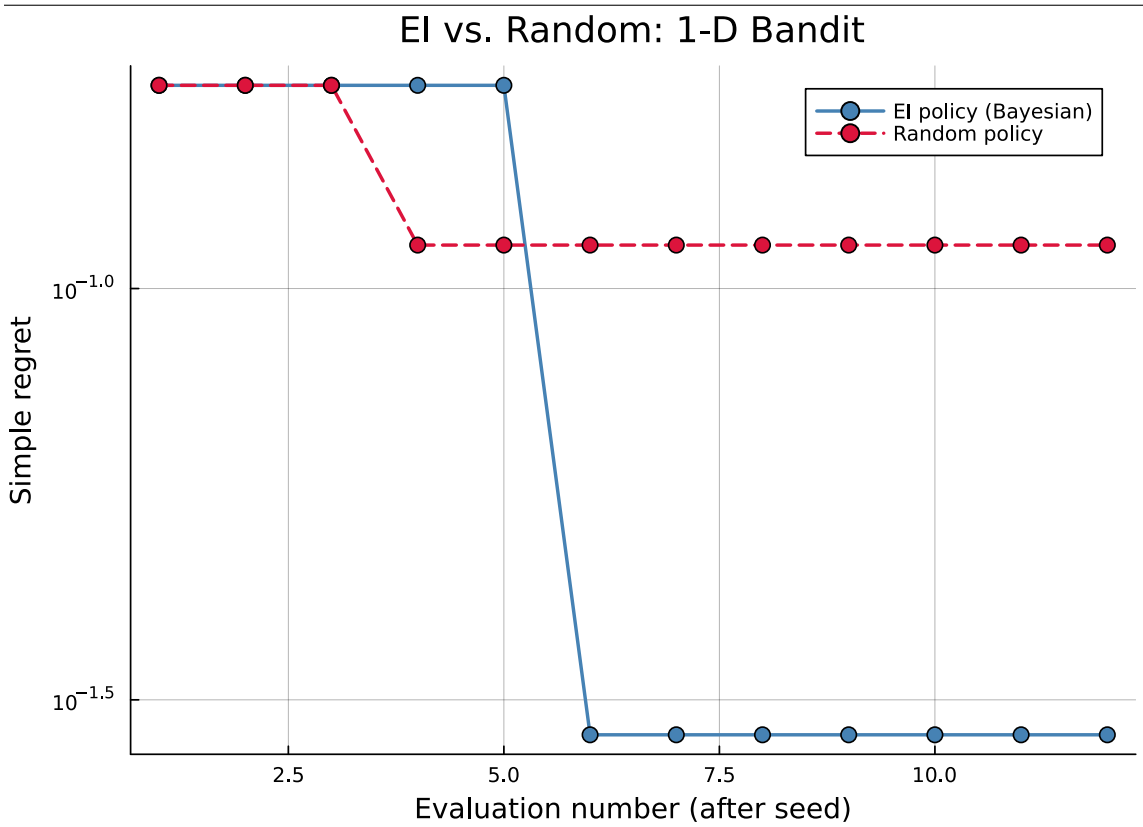
```
struct TwoLayerNet
    W1::Matrix{Float64}; b1::Vector{Float64}
    W2::Matrix{Float64}; b2::Vector{Float64}
    W3::Matrix{Float64}; b3::Vector{Float64}
end

relu(x) = max.(x, 0.0)

function forward(net::TwoLayerNet, X::Matrix{Float64})
    h1 = relu.(net.W1 * X' .+ net.b1)
    h2 = relu.(net.W2 * h1 .+ net.b2)
    ŷ  = net.W3 * h2 .+ net.b3
    return vec(ŷ)
end

function init_net(n_in::Int; h=16, seed=42)
    rng = MersenneTwister(seed)
    s1, s2 = sqrt(2/n_in), sqrt(2/h)
    TwoLayerNet(
        randn(rng, h, n_in) * s1, zeros(h),
        randn(rng, h, h)    * s2, zeros(h),
        randn(rng, 1, h)    * s2, zeros(1),
    )
end

function train_nn(net::TwoLayerNet, X_train, y_train;
                  n_epochs=500, lr=0.05, seed=42)
    rng = MersenneTwister(seed)
    function net_to_vec(n)
        vcat(vec(n.W1), n.b1, vec(n.W2), n.b2, vec(n.W3), n.b3)
    end
    function vec_to_net(v, n)
        h, ni = size(n.W1)
        i = 0
        W1 = reshape(v[i+1:i+h*ni], h, ni); i += h*ni
        b1 = v[i+1:i+h];  i += h
        W2 = reshape(v[i+1:i+h*h], h, h);  i += h*h
        b2 = v[i+1:i+h];  i += h
        W3 = reshape(v[i+1:i+h], 1, h);    i += h
        b3 = v[i+1:i+1];
        TwoLayerNet(W1,b1,W2,b2,W3,b3)
    end
    mse(n) = mean((forward(n, X_train) .- y_train).^2)
      = net_to_vec(net)
    for _ in 1:n_epochs
        g = similar( )
        f0 = mse(vec_to_net( , net))
           = lr * sqrt(f0 + 1e-8)
```

Listing 0.2

```julia
using ForwardDiff

# Exact gradient of the ssbj range function via forward-mode AD
# (no finite differences, no surrogate - exact to machine precision)
Z0 = [0.075, 4.0, 45.0, 400.0, 120.0, 4.5, 0.2, 45_000.0, 1.8]

ssbj_scalar(Z) = try
    ssbj_analysis(Z[1:7], Z[8], Z[9]; WF_init=14_000.0, WT_empty=28_000.0).Range
catch
    0.0
end

 f = ForwardDiff.gradient(ssbj_scalar, Z0)

println("Exact gradient at baseline design:")
varnames = ["t_c", "AR_w", "Λ_w", "S_w", "S_HT", "AR_HT", " ", "h", "M"]
for (name, g) in zip(varnames,  f)
    @printf("   Range/ %-8s = %+.2f NM per unit\n", name, g)
end
```

```
Exact gradient at baseline design:
   Range/ t_c      = +0.00 NM per unit
   Range/ AR_w     = +0.00 NM per unit
   Range/ Λ_w      = +0.00 NM per unit
   Range/ S_w      = +0.00 NM per unit
   Range/ S_HT     = +0.00 NM per unit
   Range/ AR_HT    = +0.00 NM per unit
   Range/         = +0.00 NM per unit
   Range/ h        = +0.00 NM per unit
   Range/ M        = +0.00 NM per unit
```

Listing 0.3


EI vs. Random: 1-D Bandit
EI policy (Bayesian)
Random policy
Simple regret
$10^{-1.0}$
$10^{-1.5}$
2.5
5.0
7.5
10.0
Evaluation number (after seed)

References

[1] Altus, T. D., "A Response Surface Methodology for Bi-Level Integrated System Synthesis (BLISS)," NASA/CR-2002-211652, NASA Langley Research Center / George Washington University, Hampton, VA, 2002.

[2] Sobieszczanski-Sobieski, J., Agte, J., and Sandusky, R., "Bi-Level Integrated System Synthesis," 1998.

[3] Sobieszczanski-Sobieski, J., Agte, J., and Sandusky, R., "Bi-Level Integrated System Synthesis (BLISS)," NASA/TM-1998-208715, NASA Langley Research Center, Hampton, VA, 2000.

[4] Sobieszczanski-Sobieski, J., Emiley, M., Agte, J., and Sandusky, R., "Advancement of Bi-Level Integrated System Synthesis (BLISS)," 2000.

[5] Giles, G. L., "Equivalent Plate Analysis of Aircraft Wing Box Structures with General Planform Geometry," *Journal of Aircraft*, Vol. 23, No. 11, 1986, pp. 859–864.

[6] Giles, G. L., "Further Generalization of an Equivalent Plate Representation for Aircraft Structural Analysis," *Journal of Aircraft*, Vol. 26, No. 1, 1989, pp. 67–74.

[7] Harris, R. V., "An Analysis and Correlation of Aircraft Wave Drag," *NASA Technical Memorandum*, Nos. X-947, 1964.

[8] Myers, R. H., and Montgomery, D. C., "Response Surface Methodology: Process and Product Optimization Using Designed Experiments," John Wiley & Sons, New York, 2002.

[9] Box, G. E. P., and Draper, N. R., "Empirical Model-Building and Response Surfaces," John Wiley & Sons, New York, 1987.

[10] Sacks, J., Welch, W. J., Mitchell, T. J., and Wynn, H. P., "Design and Analysis of Computer Experiments," *Statistical Science*, Vol. 4, No. 4, 1989, pp. 409–423.

[11] Stein, M., "Large Sample Properties of Simulations Using Latin Hypercube Sampling," *Technometrics*, Vol. 29, No. 2, 1987, pp. 143–151. https://doi.org/10.1080/00401706.1987.10488205

[12] NASA Langley Research Center, "CFL3D Version 6.0 User Manual," 2000.

[13] Samareh, J. A., "MASSOUD: A Multidisciplinary Aero/Structural Shape Optimization Using Deformation," *Proceedings of the 40th AIAA/ASME/ASCE/AHS Structures, Structural Dynamics, and Materials Conference*, 1999.

[14] Alexandrov, N. M., and Lewis, R. M., "Analytical and Computational Aspects of Collaborative Optimization for Multidisciplinary Design," *AIAA Journal*, Vol. 42, No. 2, 2004, pp. 302–310.

[15] Kennedy, M. C., and O'Hagan, A., "Predicting the Output from a Complex Computer Code When Fast Approximations Are Available," *Biometrika*, Vol. 87, No. 1, 2000, pp. 1–13.

[16] Raissi, M., Perdikaris, P., and Karniadakis, G. E., "Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations," *Journal of Computational Physics*, Vol. 378, 2019, pp. 686–707.

[17] Sutton, R. S., and Barto, A. G., "Reinforcement Learning: An Introduction," MIT Press, 2018.

[18] Gray, J. S., Hwang, J. T., Martins, J. R. R. A., Moore, K. T., and Naylor, B. A., "OpenMDAO: An Open-Source Framework for Multidisciplinary Design, Analysis, and Optimization," *Structural and Multidisciplinary Optimization*, Vol. 59, No. 4, 2019, pp. 1075–1104.

[19] Forrester, A., Sóbester, A., and Keane, A., "Engineering Design via Surrogate Modelling: A Practical Guide," Wiley, 2008.

[20] Jones, D. R., Schonlau, M., and Welch, W. J., "Efficient Global Optimization of Expensive Black-Box Functions," *Journal of Global Optimization*, Vol. 13, 1998, pp. 455–492.

Appendix A: Hypersphere Point Placement Algorithm

Algorithm Description

The hypersphere point placement scheme generates m sample points distributed approximately uniformly on the surface of an n-dimensional unit hypersphere, then maps them into the interior of the unit hypercube. The algorithm exploits the well-known result that normalised Gaussian random vectors are uniformly distributed on the unit sphere.

Step-by-Step Algorithm

1. **Generate** m vectors $\mathbf{u}_i \in \mathbb{R}^n$, each drawn from an n-dimensional standard Gaussian distribution:
$$\mathbf{u}_i \sim \mathcal{N}(0, I_n)$$

2. **Normalise** each vector to unit length (project to sphere surface):
$$\hat{\mathbf{u}}_i = \frac{\mathbf{u}_i}{\|\mathbf{u}_i\|}$$

3. **Map** from $[-1, 1]^n$ (sphere surface coordinates) to $[0, 1]^n$:
$$\mathbf{x}_i = \frac{\hat{\mathbf{u}}_i + \mathbf{1}}{2}$$

Anti-Bunching Property

For large n, the hypercube has most of its volume concentrated in the corners. Uniform random points in a hypercube tend to cluster in corners, leaving the interior sparsely

sampled. The hypersphere scheme avoids this by ensuring points lie on a surface of equal radial distance from the origin — a strong anti-bunching mechanism that produces more even coverage of the design space.

The 2002 paper defines the inter-point Euclidean distance in Appendix A, Eq. A2:

$$d(\mathbf{X}_A, \mathbf{X}_B) = \sqrt{\sum_{i=1}^{n} (X_{A_i} - X_{B_i})^2} \tag{.1}$$

! 2002 Paper Errata — Eq. A2

NASA/CR-2002-211652, Appendix A, Eq. A2 omits the square on the difference term. The printed equation reads $\sqrt{\sum(X_{A_i} - X_{B_i})}$, which is dimensionally inconsistent and would not produce a proper Euclidean distance. The correct form is $\sqrt{\sum(X_{A_i} - X_{B_i})^2}$ as shown in Equation .1 above. This is a typographical error only — the Julia implementation uses the correct formula. See Appendix C for the full errata list.

```
rng = MersenneTwister(42)
pts_hs = hypersphere_points(3, 50; seed=42)
pts_rn = random_points(3, 50; seed=42)

p1 = scatter3d(pts_hs[:,1], pts_hs[:,2], pts_hs[:,3],
               xlabel="x ", ylabel="x ", zlabel="x ",
               title="Hypersphere (50 pts)", markersize=3,
               color=:steelblue, alpha=0.7, legend=false)

p2 = scatter3d(pts_rn[:,1], pts_rn[:,2], pts_rn[:,3],
               xlabel="x ", ylabel="x ", zlabel="x ",
               title="Random (50 pts)", markersize=3,
               color=:crimson, alpha=0.7, legend=false)

plot(p1, p2, size=(800, 380))
```

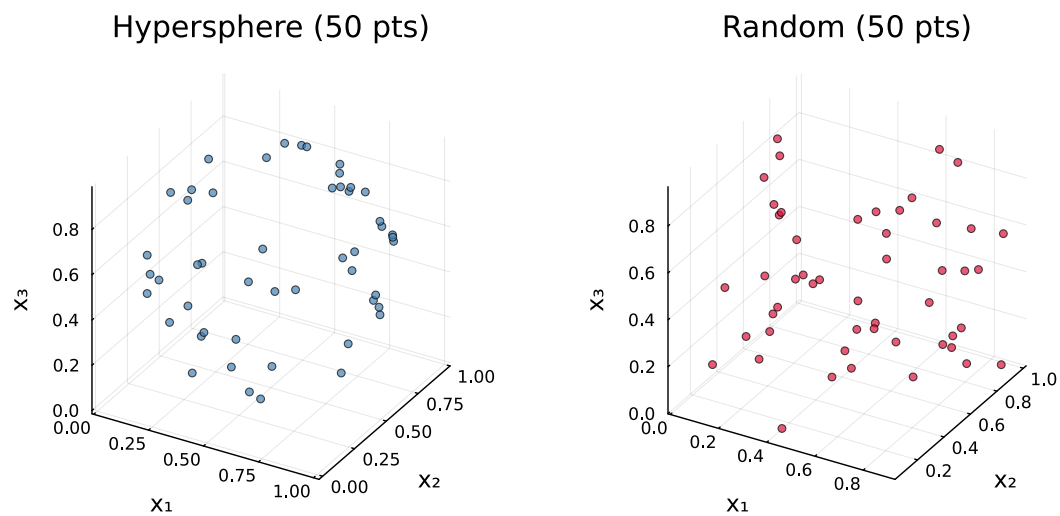


Figure 1: Hypersphere points projected into 3D: 50 points projected from the unit sphere surface into $[0,1]^3$. Points cover the faces of the cube more uniformly than pure random placement.

Minimum Point Count

A key advantage of the hypersphere scheme is that the number of points is determined solely by the minimum required for the RS model (N_S from Equation 0.9), not by the formula of any specific classical design. This gives the designer freedom to generate exactly as many points as needed.

```
# For each dimensionality, show the minimum required and a recommended augmented coun
println("n | NS_min | Recommended (NS_min + 10%)")
println("-" ^ 42)
for n in 2:12
    ns_min = min_rs_points(n)
    ns_rec = ceil(Int, 1.1 * ns_min)
    @printf("%2d | %6d | %25d\n", n, ns_min, ns_rec)
end
```

```
n | NS_min | Recommended (NS_min + 10%)
------------------------------------------
 2 |      6 |                         7
 3 |     10 |                        11
 4 |     15 |                        17
```

```
 5 |      21 |                          24
 6 |      28 |                          31
 7 |      36 |                          40
 8 |      45 |                          50
 9 |      55 |                          61
10 |      66 |                          73
11 |      78 |                          86
12 |      91 |                         101
```

Julia Implementation (Full Detail)

```julia
function hypersphere_points_full(n::Int, m::Int; seed::Int=42) :: Matrix{Float64}
    # Step 1: n-dimensional Gaussian samples
    rng  = MersenneTwister(seed)
    U    = randn(rng, m, n)             # m × n matrix of raw Gaussian samples

    # Step 2: normalise each row to unit Euclidean norm
    norms = sqrt.(sum(U .^ 2, dims=2))  # m × 1 column of norms
    U_hat = U ./ norms                  # each row now has norm = 1

    # Step 3: map from [-1,1]  to [0,1]
    X = (U_hat .+ 1.0) ./ 2.0

    return X
end

# Verify the implementation matches src/bliss_rs.jl
pts_ref  = hypersphere_points(3, 20; seed=99)
pts_full = hypersphere_points_full(3, 20; seed=99)
@printf("Max discrepancy vs. reference: %.2e\n", maximum(abs.(pts_ref .- pts_full)))
```

```
Max discrepancy vs. reference: 0.00e+00
```

Comparison with Classical Designs in 2D

```
# Central Composite Design (2 factors,   = √2 for rotatable)
  = sqrt(2.0)
cube_pts = [[-1,-1],[1,-1],[-1,1],[1,1]]
axial_pts = [[- ,0],[ ,0],[0,- ],[0, ]]
center_pt = [[0.0,0.0],[0,0],[0,0]]
ccd_pts = vcat(cube_pts, axial_pts, center_pt)
ccd_mat = hcat([[p[1] for p in ccd_pts], [p[2] for p in ccd_pts]]...)'
ccd_mat = (ccd_mat .+  ) ./ (2 )   # scale to [0,1]

# Other designs
hs_pts   = hypersphere_points(2, 15; seed=42)
dopt_pts = d_optimal_points(2, 15; seed=42)
rnd_pts  = random_points(2, 15; seed=42)

scatter_kw = (xlim=(-0.05,1.05), ylim=(-0.05,1.05),
              aspect_ratio=1, legend=false, grid=true, gridalpha=0.3,
              xlabel=L"x_1", ylabel=L"x_2")

p1 = scatter(ccd_mat[:,1], ccd_mat[:,2]; title="CCD (11 pts)",
             markersize=7, color=:darkorange, markerstrokecolor=:white, scatter_kw...
p2 = scatter(hs_pts[:,1], hs_pts[:,2]; title="Hypersphere (15 pts)",
             markersize=7, color=:steelblue, markerstrokecolor=:white, scatter_kw...)
p3 = scatter(dopt_pts[:,1], dopt_pts[:,2]; title="D-Optimal (15 pts)",
             markersize=7, color=:forestgreen, markerstrokecolor=:white, scatter_kw..
p4 = scatter(rnd_pts[:,1], rnd_pts[:,2]; title="Random (15 pts)",
             markersize=7, color=:crimson, markerstrokecolor=:white, scatter_kw...)

plot(p1, p2, p3, p4, layout=(2,2), size=(680, 580))
```

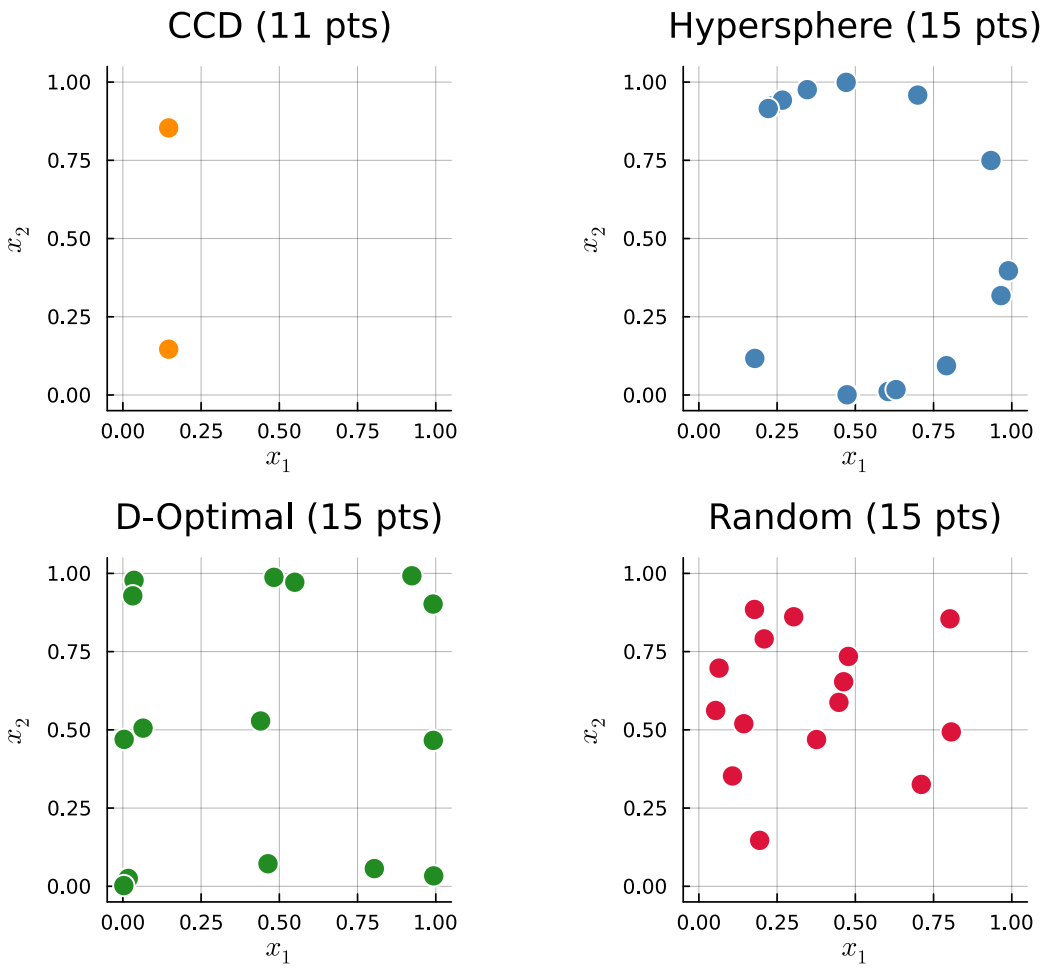


Figure 2: Four point placement strategies for n=2, m=15 points. The CCD includes fixed axial points at ± ; Box-Behnken uses midpoints of cube edges. D-optimal adapts to the regression model; hypersphere fills the unit sphere surface.

Appendix B: Julia Reference — Flutter Constraint and Module Listings

Flutter Constraint Derivation

The anti-flutter constraint added to the upgraded structures black box is based on a first-order aeroelastic stability criterion. Define:

- M_{DR} — deformed wing root bending moment (from ELAPS deflection solution)
- M_{RR} — rigid wing root bending moment (from applied aerodynamic loads)

The bending moments are computed from the spanwise lift distribution $L(y)$:

$$M_{root} = \int_0^{b/2} L(y) \cdot y \, \mathrm{d}y$$

For the rigid wing, $L(y)$ is the prescribed aerodynamic load distribution. For the deformed wing, $L(y)$ is modified by the twist and bending deformations $d_A(y)$ and $d_B(y)$ computed by ELAPS:

$$M_{DR} = \int_0^{b/2} L(y) \left(1 + \frac{\partial C_L}{\partial \alpha} \frac{d_A(y) - d_B(y)}{c(y)}\right) y \, \mathrm{d}y$$

The flutter constraint requires:

$$\frac{M_{DR}}{M_{RR}} \leq 1.0 \tag{.1}$$

When $M_{DR}/M_{RR} > 1$, the deformed wing generates more lift than the rigid wing, which amplifies the deflection — a divergence/flutter instability.

In ELAPS, the ratio is approximated by:

$$\frac{M_{DR}}{M_{RR}} \approx 1 + K_{flutter} \cdot AR_w^{1.5} \cdot (d_A - d_B)_{\max}$$

where $K_{flutter}$ is a calibration constant derived from higher-fidelity aeroelastic computations.

Source Module: `src/bliss_rs.jl`

The complete BLISS-RS core implementation is in src/bliss_rs.jl. Key exported functions:

Function	Purpose
`min_rs_points(n)`	Minimum sample count for second-order RS in n variables
`rs_design_matrix(pts)`	Build the [X] design matrix
`fit_rs(pts, y)`	Least squares RS fit
`predict_rs(pts, b)`	Evaluate RS at new points
`rs_normalized_error(y_actual, y_rs)`	Mean absolute normalised error
`hypersphere_points(n, m)`	Hypersphere DOE
`random_points(n, m)`	Uniform random DOE
`d_optimal_points(n, m)`	D-optimal DOE via coordinate exchange
`code_to_natural( , xL, xU)`	Coded → natural variable mapping
`natural_to_code(x, xL, xU)`	Natural → coded variable mapping
`interval_reduction_factors(opt, lo, hi)`	Compute new bounds
`update_intervals(Z_opt, bounds)`	Apply interval reduction to all variables

Source Module: `src/supersonic_jet.jl`

The supersonic business jet black boxes are in src/supersonic_jet.jl.

Function	Purpose
`std_atmosphere(h_ft)`	Simplified standard atmosphere model

Function	Purpose
`structures_bb(Z, L, WE)`	Wing structural weight and aeroelastic twist
`aero_bb(Z, h, M, WT, Ω, ESF)`	Drag polar aerodynamics
`power_bb(h, M, D)`	Engine scaling and SFC
`performance_bb(WT, WF, LD, SFC, h, M)`	Breguet range
`ssbj_analysis(Z, h, M)`	Coupled system analysis (all four BBs)

Source Module: `src/srm.jl`

The solid propellant rocket motor black boxes are in src/srm.jl.

Function	Purpose
`thrust_coefficient( , Pc, Pe, Pa, )`	Ideal nozzle CF
`exit_pressure(Pc, , )`	Isentropic area-Mach iteration
`characteristic_velocity( , Tc, MW)`	Combustion c*
`propellant_properties(AP%, Al%)`	AP/Al/HTPB thermochemical model
`ballistics_bb(Z, X)`	Internal ballistics (thrust, Isp, total impulse)
`structural_bb(Z, X, Pc)`	Case structural analysis
`nozzle_bb(Z, X, Pc, At, CF)`	Nozzle performance
`mission_bb(W_total, W_prop, It, Isp)`	Tsiolkovsky delta-V
`srm_analysis(Z, X_prop, X_struct, X_nozzle)`	Coupled SRM analysis

Reproducing the Results

All figures and tables in this book are generated from the source `.qmd` files. To reproduce:

```
# 1. Install pixi (https://pixi.sh)
# 2. Install Julia and package dependencies
```

```
pixi run setup

# 3. Render to HTML (default)
pixi run render

# 4. Render to PDF
pixi run render-pdf

# 5. Clear cache and re-render from scratch
pixi run clean && pixi run render
```

Computation time for a cold render (all code executed from scratch) is approximately 15–30 minutes on an Apple M4, dominated by the BLISS-RS optimization loops in Chapters 7 and 10. Subsequent renders use Quarto's freeze cache and complete in under 1 minute.

Appendix C: Errata — NASA/CR-2002-211652

This appendix documents confirmed errors in the original 2002 reference:

> Altus, T.D. (2002). *A Response Surface Methodology for Bi-Level Integrated System Synthesis (BLISS)*. NASA/CR-2002-211652. May 2002.

Errors are classified as **mathematical** (affect results or implementation) or **typographical** (presentation only, no effect on correctness).

Errata Table

ID	Location in 2002 Paper	Type	Description	Effect on Implementation	Status in This Work
E1	App. A, Eq. A2	**Mathematical**	Distance formula prints $\sqrt{\sum(X_{A_i} - X_{B_i})}$ — missing square on difference term. Correct: $\sqrt{\sum(X_{A_i} - X_{B_i})^2}$	Would produce incorrect inter-point distances and broken antibunching if implemented as printed	Julia uses correct squared Euclidean distance. See Section .

ID	Location in 2002 Paper	Type	Description	Effect on Implementation	Status in This Work
E2	Fig. 41, axis label	Typographical	Units label reads “nm” (nanometres). Intended: “NM” (nautical miles). Range values throughout paper are in nautical miles.	None	This work uses “NM” consistently. See Section .
E3	Fig. 19, caption	Typographical	“Hypershpere” — missing letter ‘e’.	None	Corrected throughout.
E4	Fig. 6, caption	Typographical	“Cross-sectionl” — missing letter ‘a’.	None	Corrected throughout.
E5	p. 20, body text	Typographical	“Aany” — double ‘a’, should be “Any”.	None	Corrected throughout.

Notes

- E1 is the only error with potential implementation consequences. All other entries are typographical and do not affect the mathematics or results.

- This errata list covers errors identified during the 2026 IV&V review. It is not claimed to be exhaustive.
- The full IV&V report is maintained separately at `ivv/ivv-bliss-rs-2002.md`.